Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– Hanoi, June 2026–

Table of Contents

I. Record of Changes 2

II. Software Design Document 4

1. System Design 4

1.1 System Architecture 4

1.1.1 Logical Architecture Diagram 5

1.1.2 Architectural Components Description 5

1.2 Package Diagram 7

2. Database Design 9

3. Detailed Design 12

3.1 Module 1: Authentication & Profile Management (UC01 - UC05) 12

3.2 Module 2: Tenancy & Roommate Management (UC06 - UC12) 16

3.3 Module 3: Billing & Payment Management (UC13 - UC17) 25

3.4 Module 4: Incident & Task Management (UC18 - UC23) 30

3.5 Module 5: Communication & Notifications (UC24 - UC28) 38

3.6 Module 6: Apartment Management (UC29 - UC34) 42

3.7 Module 7: Statistics & Dashboard (UC35) 48

3.8 Module 8: Operational Staff Management (UC36 - UC40) 51

3.9 Module 9: Manager Management (UC41 - UC45) 56

I. Record of Changes

Date	A* M, D	In charge	Change Description

*A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

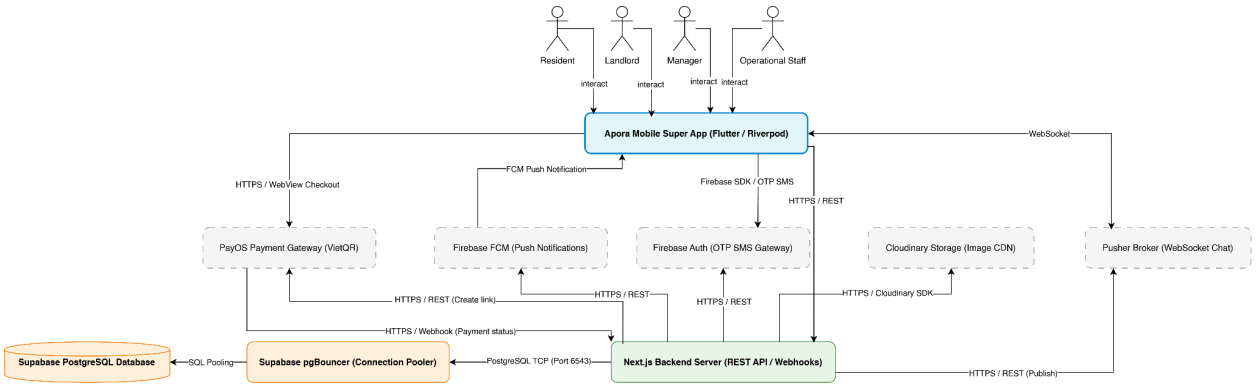
1.1 System Architecture

The Apora (Apartment Management System) system utilizes a modern, 4-tier layered architecture style (Presentation Layer, Application Layer, Data Layer, and Integration Layer) to build a robust, scalable, and secure mobile "Super App".

All user actors (Landlord, Manager, Resident, and Operational Staff) interact with the system through a single mobile application (Apora Mobile App), which dynamically renders role-specific workspaces and dashboards upon authentication. Client-side applications communicate with the central backend server via RESTful APIs and WebSockets, while the backend coordinates database transactions and offloads specialized workloads to third-party cloud services.

1.1.1 System Architecture Diagram

The diagram below illustrates the system components, their classification across the architectural layers, and the data flow interfaces connecting them:



1.1.2 Architectural Components Description

A. Presentation Layer (Sub-systems / Front-end Client)

This layer represents the unified mobile application running on the users' devices. Rather than deploying separate web portals, Apora utilizes a role-based modular architecture inside a single mobile client. Upon user login, a role-selection interface checks user permissions (e.g., if a user is both a Landlord and Resident) and initializes the corresponding workspace module.

1. **Apora Mobile Super App**
    - **Purpose**: The single entry point for all actors. Built on Flutter for cross-platform Android & iOS support, using Riverpod for state management to enforce a clean unidirectional data flow.
    - **Resident Module**: Allows tenants to view contracts, pay monthly invoices, submit repair tickets with photos, register roommates, request stay extensions, view announcements, and chat with management.
    - **Manager Module**: The workspace for property managers. Enables apartment configurations, check-in/check-out processing, In-app utility meter index readings input

and billing, roommate and stay extension requests approval, task delegation to staff, and news broadcasting.
- **Operational Staff Module**: The workspace for Security Guards, Janitors, and Technicians to view assigned maintenance tasks, update progress in real-time, and upload completion proof photos.
- **Landlord Module**: Provides the building owner with global dashboard statistics (occupancy data, revenue collections charts) and manager account provisioning features.
- **Core Features**: Configures apartment structures, manages manager accounts, and monitors global financial records (revenue collection charts), occupancy rates, and ticket statistics.

B. Application & Orchestration Layer (Core Backend)

This layer acts as the centralized brain of the system, abstracting business rules and validating operations before committing state changes.

1. **1. Next.js Backend Server**
    - **Purpose**: Serves RESTful API requests, exposes endpoints for the Flutter client, hosts background batch jobs, and processes webhook events.
    - **Core Functions**:
        - **Authentication & Role Authorization**: Decodes and verifies JWT tokens to authorize requests based on roles (Landlord, Manager, Resident, Staff).
        - **Billing Calculations**: Computes invoice totals combining base rent, water, and electricity consumption.
        - **Workflow Control**: Enforces valid state transitions (e.g., ticket lifecycle: PENDING → ASSIGNED → PROCESSING → RESOLVED).
        - **Webhook Handling**: Listens to server-to-server transaction status signals from PayOS.
        - **Data Anonymization Job**: Performs scheduled/triggered data masking and media deletion to comply with data privacy policies when tenants check out.

C. Data Persistence Layer (Database)

This layer guarantees the reliability, consistency, and security of stored business information.

1. **1. Supabase pgBouncer (Connection Pooler)**
    - **Purpose**: Manages database connection scaling. Since the backend executes serverless-compatible API routes, pgBouncer is configured on port 6543 to pool database connections, ensuring the database does not crash under concurrent load from hundreds of users.
2. **2. Supabase PostgreSQL Database**
    - **Purpose**: Stores relational records (Users, Apartments, Invoices, Contracts, Repair Tickets, Tasks, and Chat Messages).
    - **Core Features**: Enforces strict relational schema constraints, cascades delete/update operations, and encapsulates critical ACID transactions (such as atomic check-in/check-out routines).

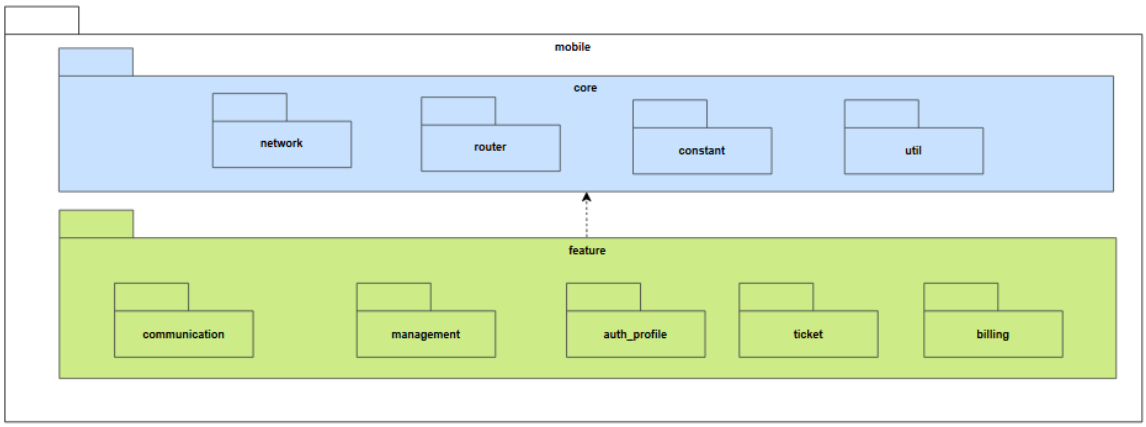
D. Integration & External Services Layer

To maintain a serverless-friendly, lightweight architecture and offload operational overhead, the system delegates specialized services to established external cloud platforms.

1. **1. PayOS System (Payment Gateway)**
    - **Role**: Facilitates secure online payments.
    - **Mechanism**: When a resident requests payment, the backend calls PayOS APIs to generate a unique VietQR code and secure checkout link. After the bank transfer completes, PayOS invokes the backend's webhook listener with an HMAC-SHA256 signature to verify and clear the invoice.
2. **2. Cloudinary Service (Image Storage)**
    - **Role**: Handles media asset storage and delivery.
    - **Mechanism**: Optimizes, compresses, and distributes resident identity cards, incident photos, and task completion proofs. The backend coordinates with Cloudinary via its SDK for uploads and handles the programmatic deletion of stored images when users check out.
3. **3. Pusher System (WebSocket Broker)**
    - **Role**: Powers the real-time Live Chat feature.
    - **Mechanism**: Acts as a managed, state-free WebSocket broker. It distributes chat messages instantly between residents and management, bypassing the socket connection constraints of the serverless backend.
4. **4. Firebase Cloud Messaging (FCM)**
    - **Role**: Delivers background real-time system alerts.
    - **Mechanism**: Dispatches push notifications regarding new invoices, ticket status updates, stay extension review verdicts, and community announcements to users' mobile device trays.
5. **5. Firebase Authentication System (OTP Gateway)**
    - **Role**: Manages SMS OTP generation and password reset authorization.
    - **Mechanism**: Verifies the identity of residents requesting password resets by routing a One-Time Password via SMS. Upon validation, it issues verified ID tokens used by the backend to securely update password hashes.

1.2 Package Diagram

1.2.1 Frontend Flutter package diagram

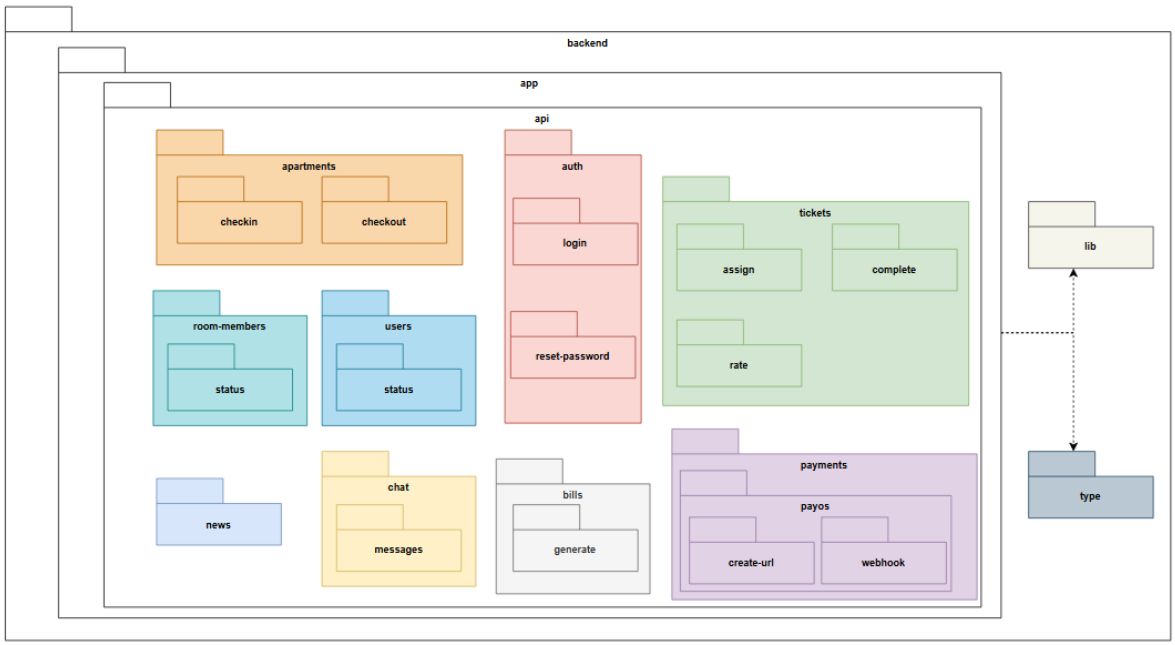


Package Descriptions

No	Package	Description
01	mobile	Contains the entire Frontend source code of the mobile application (Super App) for Residents, the Management Board, Landlords, and Operational Staff.
02	feature	A sub-package of mobile containing the application's main business feature modules and User Interfaces (UI).
03	management	Manages functions specific to Managers/Landlords, such as managing apartments, staff, and approving requests.
04	billing	Contains the UI and processing logic for monthly bills, transaction history, and payments.
05	auth profile	Handles authentication interfaces (login, forgot password), role-based authorization, and personal profile management.
06	communication	Manages communication features, including the community news board and real-time Live Chat.
07	ticket	Contains interfaces for creating, tracking, assigning, and updating the progress of incidents and repair tickets.
08	core	A sub-package of mobile containing core components, base configurations, and shared code used throughout the application.
09	constant	Contains constants, error messages, colors, fonts, and fixed configuration values for the mobile app.

10	router	Defines and manages navigation flows and screen routing within the application.
11	network	Handles network configuration, interceptors, and API calling functions to communicate with the Backend.
12	util	Contains shared utility functions (e.g., date formatting, currency formatting, input data validation).

1.2.2 Backend NextJS package diagram



Package Descriptions

No	Package	Description
01	backend	Contains the entire server-side source code (API Server), handling business logic and database interactions.
02	app	The main sub-package of backend where the system's execution modules are centralized.
03	api	Defines the endpoints (API routes) that the Frontend calls to communicate with the system.
04	apartments	Handles logic related to the apartment lifecycle, including APIs for check-in and check-out operations.

05	users	Manages user data, account configurations, and user status modifications.
06	room-members	Handles the approval logic and management of roommate lists within an apartment.
07	auth	Manages user authentication logic, including login processes, token issuance, and password resets.
08	tickets	Manages the entire lifecycle of repair requests: assignment, completion, and rating.
09	bills	Handles the calculation and automatic generation of periodic monthly invoices for apartments.
10	chat	Manages real-time message synchronization logic between users via WebSocket connections.
11	news	APIs for handling the creation, editing, and publishing of announcements on the community board.
12	payments	Handles integration with the PayOS payment gateway, generating payment URLs, and receiving Webhook signals to update invoice statuses.
13	lib	Contains libraries and configurations for integrating with third-party services (PayOS, Cloudinary, Firebase, Pusher).
14	type	Contains data type definitions (Interfaces, DTOs) to ensure data structure consistency across the Backend.

2. Database Design

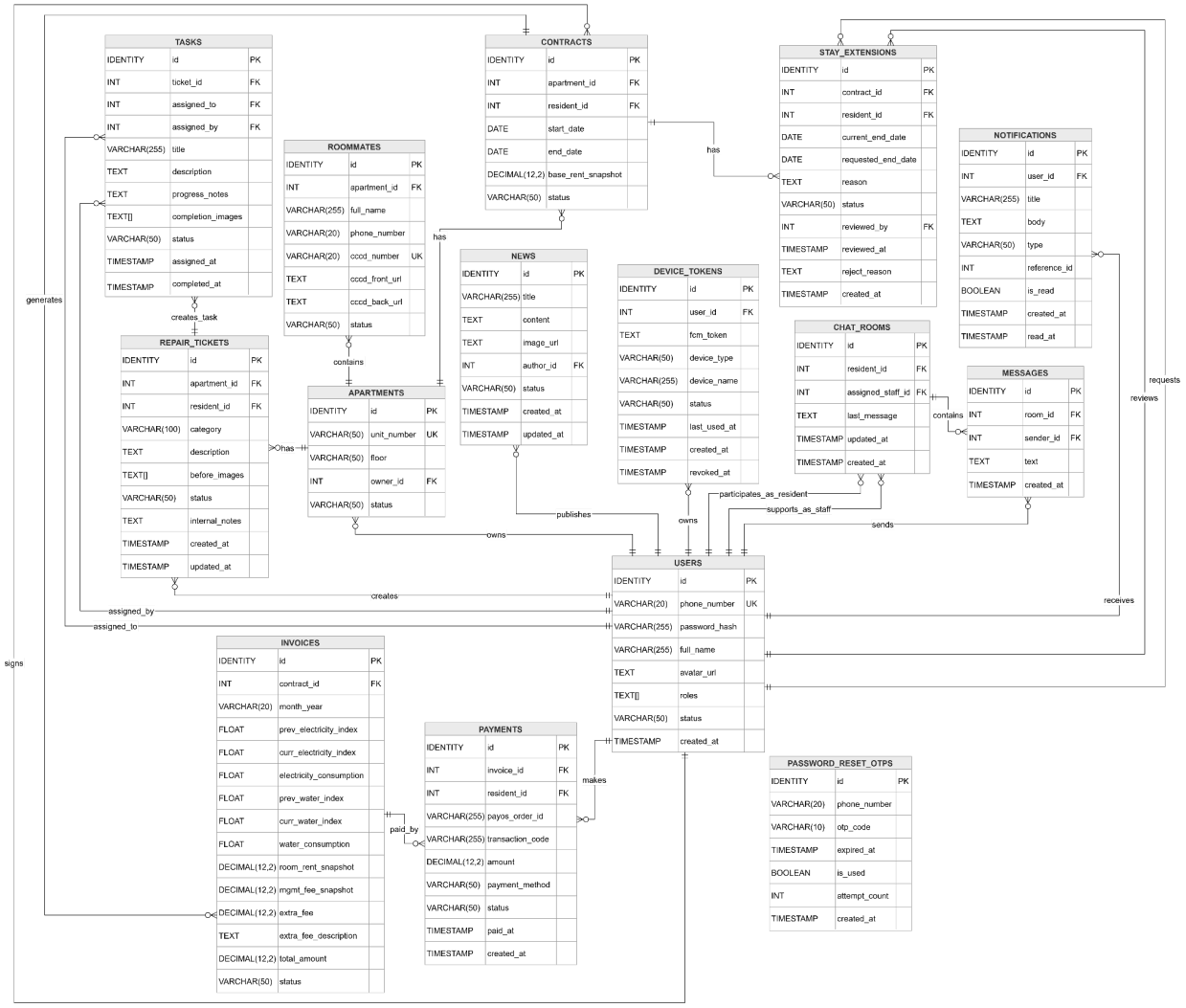


Table Descriptions

No	Table	Description
01	USERS	Stores personal information, identity details, roles, and login credentials of system users. - Primary keys: id - Foreign keys: None
02	APARTMENTS	Stores detailed information about apartment units. - Primary keys: id - Foreign keys: owner_id
03	CONTRACTS	Manages rental contracts between residents and apartment owners, including lease durations and fixed price snapshots. - Primary keys: id - Foreign keys: apartment_id, resident_id

04	INVOICES	Manages monthly billing invoices for residents, recording electricity and water consumption along with related rental fees. - Primary keys: id - Foreign keys: contract_id
05	ROOMMATES	Stores information about individuals sharing an apartment with the main resident, including their identity profiles (CCCD). - Primary keys: id - Foreign keys: apartment_id
06	REPAIR_TICKETS	Records repair and maintenance requests reported by residents regarding issues in their respective apartments. - Primary keys: id - Foreign keys: apartment_id, resident_id
07	NEWS	Manages news and announcements published by the management board or landlords to keep residents informed. - Primary keys: id - Foreign keys: author_id
08	CHAT_ROOMS	Manages direct support chat sessions between residents and assigned staff members (management or security guards). - Primary keys: id - Foreign keys: resident_id, assigned_staff_id
09	MESSAGES	Stores the detailed text content and timestamps of messages sent within the chat rooms. - Primary keys: id - Foreign keys: room_id, sender_id
10	TASKS	Manages work assignments (e.g., repairs, inspections) allocated to staff members to resolve repair tickets. - Primary keys: id - Foreign keys: ticket_id, assigned_to, assigned_by
11	DEVICE_TOKENS	Stores Firebase Cloud Messaging (FCM) device tokens used to send push notifications to users' devices. - Primary keys: id - Foreign keys: user_id
12	PASSWORD_RESET_OTPS	Records and manages One-Time Password (OTP) codes sent to users during the password recovery process. - Primary keys: id - Foreign keys: None
13	STAY_EXTENSIONS	Stores requests submitted by residents to extend their current rental contracts and tracks the approval status.

		- Primary keys: id - Foreign keys: contract_id, resident_id, reviewed_by
14	PAYMENTS	Records the history and status of payment transactions made by residents via the payment gateway. - Primary keys: id - Foreign keys: invoice_id, resident_id
15	NOTIFICATIONS	Stores the list of system notifications generated and sent to users, tracking their read status. - Primary keys: id - Foreign keys: user_id

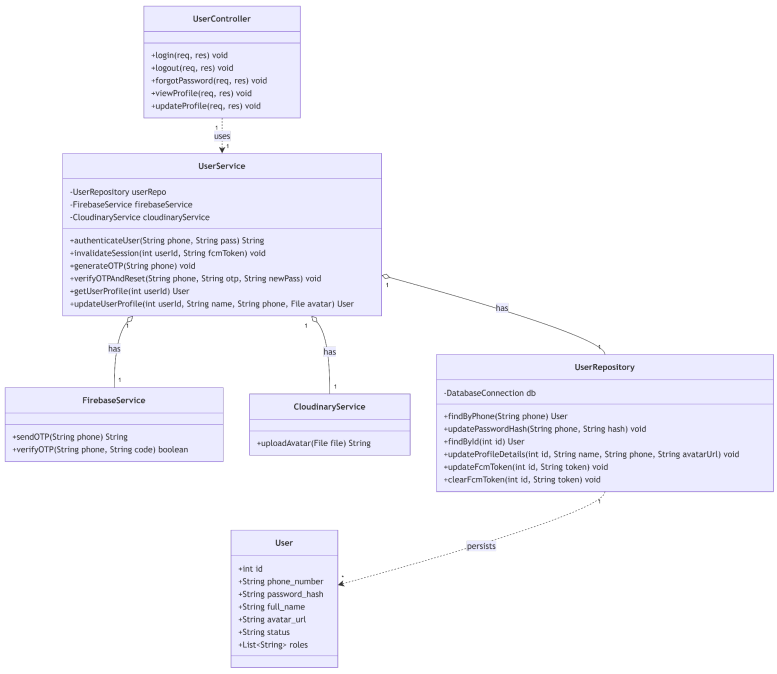
3. Detailed Design

3.1 Module 1: Authentication & Profile Management (UC01 - UC05)

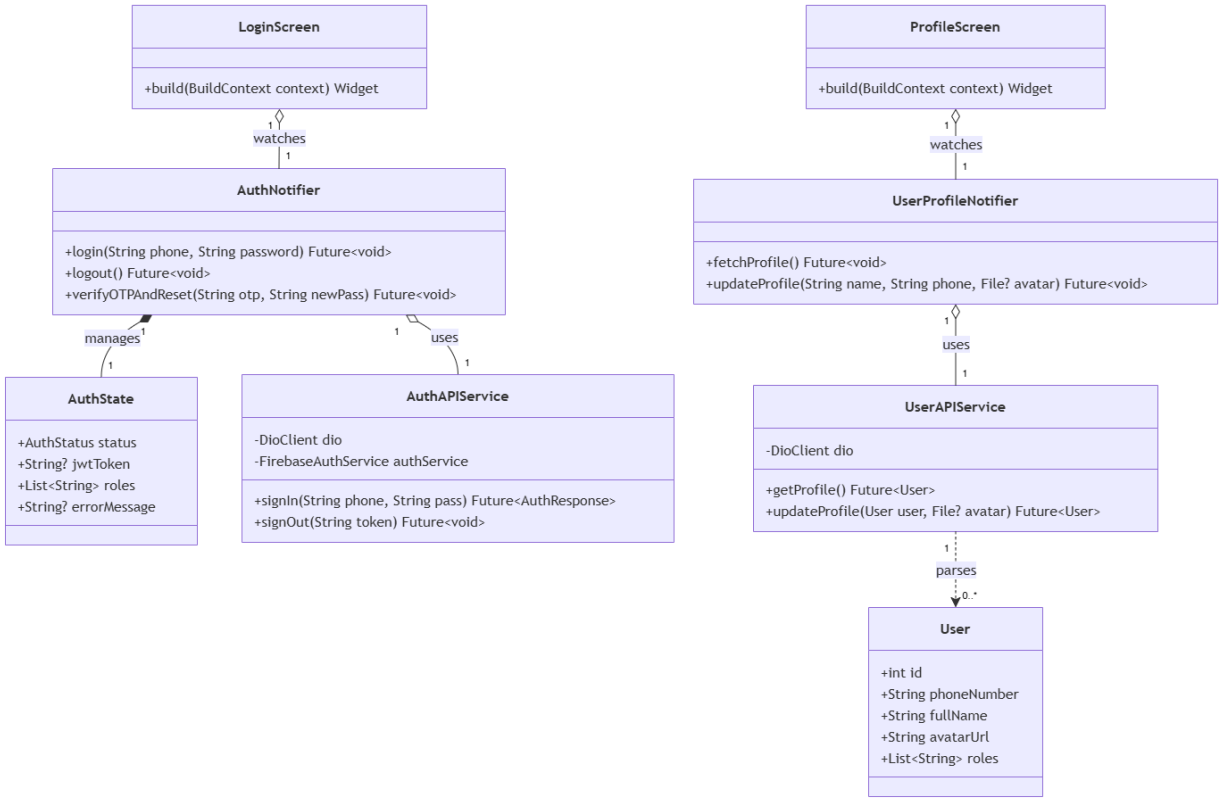
This module handles secure user authentication, password recovery via SMS OTP, session management, and profile customization.

3.1.1 Class Diagram

3.1.1.1. NextJS Backend Class Diagram



3.1.1.2. Flutter Client Class Diagram



3.1.2 Class Specifications

3.1.2.1 NextJS Backend Class Specifications

A. UserController

- **Purpose**: Exposes HTTP endpoints for session handling, OTP requests, profile reading, and profile updates. Depends on UserService to execute business logic.
- **Attributes:**
    - userService: UserService
- **Methods**:
    - login(req, res): POST /api/auth/login. Extracts credentials, calls UserService.authenticateUser(), and returns a JWT token with user roles.
    - logout(req, res): POST /api/auth/logout. Extracts session data and calls UserService.invalidateSession() to clear active tokens.
    - forgotPassword(req, res): POST /api/auth/forgot-password. Handles OTP generation requests or final password resets via UserService.
    - viewProfile(req, res): GET /api/users/profile. Fetches active user data using UserService.getUserProfile().
    - updateProfile(req, res): PUT /api/users/profile. Receives updated text fields and multi-part files, forwarding them to UserService.updateUserProfile().

B. UserService

- **Purpose**: Orchestrates core business logic. **Aggregates UserRepository, FirebaseService, and CloudinaryService** to manage authentication states, security validations, and asset uploads.
- **Attributes:**
    - userRepository: UserRepository
    - firebaseService: FirebaseService
    - cloudinaryService: CloudinaryService
- **Methods**:
    - authenticateUser(phone, pass): Validates active status, verifies password with bcrypt via UserRepository.findByPhone(), saves the new FCM token, and returns a generated JWT.
    - invalidateSession(userId, fcmToken): Clears active sessions and invokes UserRepository.clearFcmToken() to stop push alerts.
    - generateOTP(phone): Checks user existence via UserRepository, then triggers FirebaseService.sendOTP().
    - verifyOTPAndReset(phone, otp, newPass): Validates the OTP with FirebaseService, hashes newPass using bcrypt, and updates the DB via UserRepository.updatePasswordHash().
    - getUserProfile(userId): Fetches a User entity from UserRepository.findById() and masks sensitive fields before returning it.
    - updateUserProfile(userId, name, phone, avatar): Compresses the avatar file (<500KB), uploads it via CloudinaryService.uploadAvatar(), updates profile info via UserRepository.updateProfileDetails(), and returns the updated User object.

C. UserRepository

- **Purpose**: Direct database communicator executing CRUD operations on the users table. **Persists and maps raw database rows into User entities**.
- **Attributes:**
    - db: DatabaseConnection
- **Methods**:
    - findByPhone(phone): Queries the database and returns a corresponding User entity or null.
    - updatePasswordHash(phone, hash): Commits the newly encrypted password hash to the database.
    - findById(id): Queries and returns a User entity by its primary key.
    - updateProfileDetails(id, name, phone, avatarUrl): Updates data fields for a specific user profile.
    - updateFcmToken(id, token): Updates or appends a registration token for Firebase Cloud Messaging (FCM).
    - clearFcmToken(id, token): Removes a specific FCM token upon user logout.

3.1.2.2 Flutter Client Class Specifications

A. AuthNotifier

- **Purpose**: A Riverpod notifier managing authentication flows, SMS OTP verification, and JWT session caching on the device.

- **Attributes**:
    - authAPIService: AuthAPIService
    - state: AuthState
- **Methods**:
    - login(phone: String, password: String): Future<void>
    - logout(): Future<void>
    - verifyOTPAndReset(otp: String, newPass: String): Future<void>

B. AuthState

- **Purpose**: Immutable state object representing the current user session and authentication status.
- **Attributes**:
    - status: AuthStatus
    - jwtToken: String?
    - roles: List<String>
    - errorMessage: String?

C. AuthAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Authentication REST API endpoints.
- **Attributes**:
    - dio: DioClient
    - authService: FirebaseAuthService
- **Methods**:
    - signIn(phone: String, pass: String): Future<AuthResponse>
    - signOut(token: String): Future<void>

D. UserProfileNotifier

- **Purpose**: A Riverpod notifier managing active user profiles and metadata updates.

- **Attributes**:
    - userAPIService: UserAPIService
    - state: AsyncValue<User>
- **Methods**:
    - fetchProfile(): Future<void>
    - updateProfile(name: String, phone: String, avatar: File?): Future<void>

E. UserAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js User REST API endpoints.
- **Attributes**:
    - dio: DioClient
- **Methods**:
    - getProfile(): Future<User>
    - updateProfile(user: User, avatar: File?): Future<User>

F. User (Model)

- **Purpose**: Client-side data model representing a user entity with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - phoneNumber: String
    - fullName: String
    - avatarUrl: String
    - roles: List<String>
- **Methods**:
    - fromJson(json: Map<String, dynamic>): User
    - toJson(): Map<String, dynamic>

G. LoginScreen

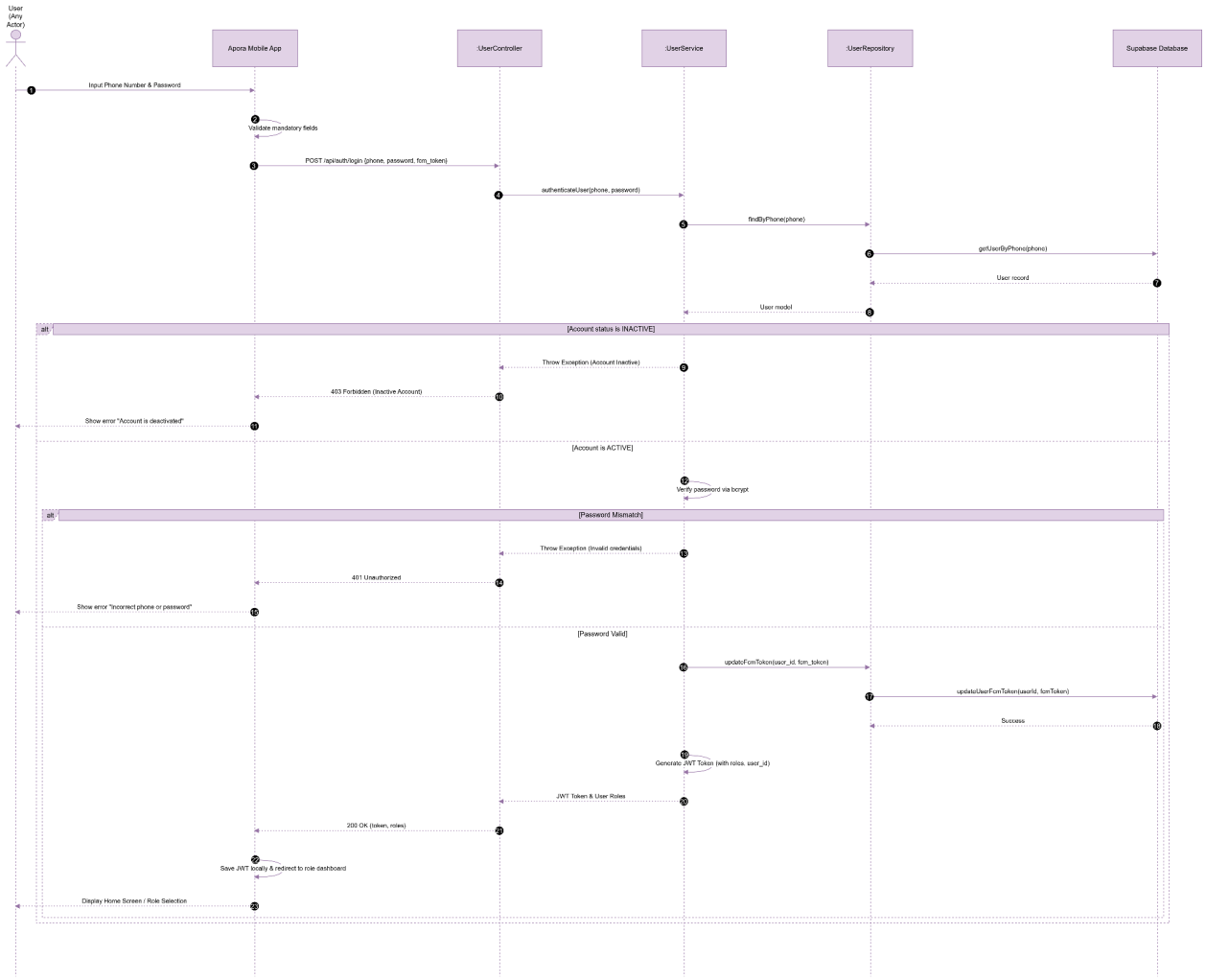
- **Purpose**: Flutter screen handling user credential input and authentication redirects.
- **Methods**:
    - build(context: BuildContext): Widget

H. ProfileScreen

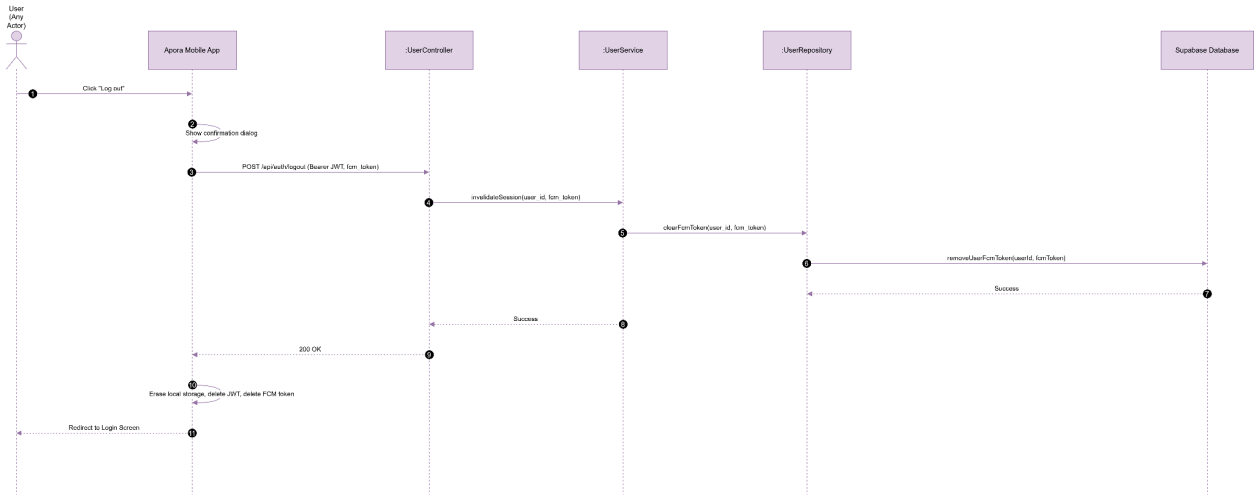
- **Purpose**: Flutter screen displaying active user details and profile modification forms.
- **Methods**:
    - build(context: BuildContext): Widget

3.1.3 Sequence Diagrams

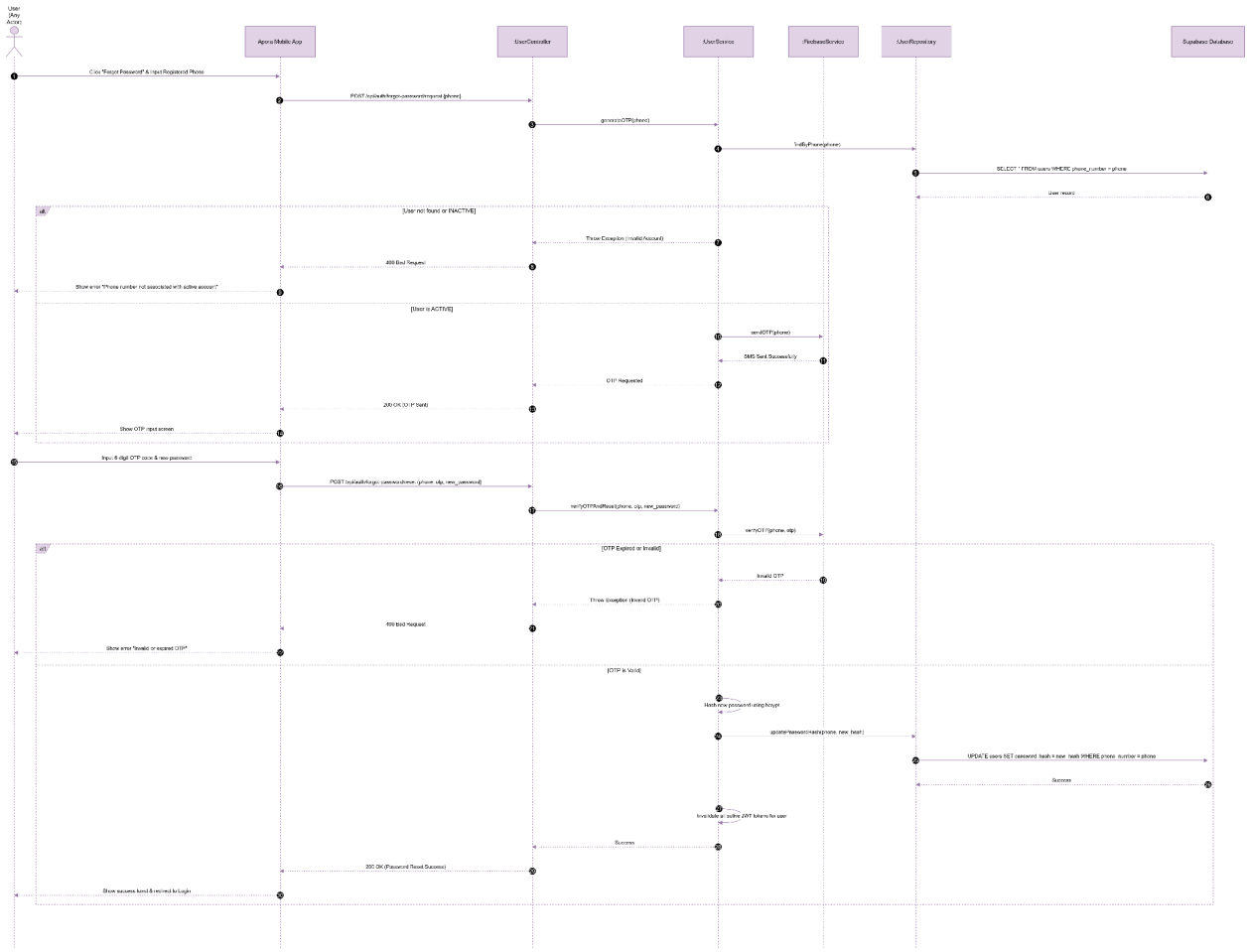
UC01 : Login


User (Any Actor)
Apora Mobile App
:UserController
:UserService
:UserRepository
Supabase Database
1 Input Phone Number & Password
2 Validate mandatory fields
3 POST /api/auth/login (phone, password, fcm_token)
4 authenticateUser(phone, password)
5 findByPhone(phone)
6 getUserByPhone(phone)
7 User record
8 User model
alt [Account status is INACTIVE]
9 Throw Exception (Account Inactive)
10 403 Forbidden (Inactive Account)
11 Show error "Account is deactivated"
[Account is ACTIVE]
12 Verify password via bcrypt
alt [Password Mismatch]
13 Throw Exception (Invalid credentials)
14 401 Unauthorized
15 Show error "Incorrect phone or password"
[Password Valid]
16 updateFcmToken(user_id, fcm_token)
17 updateUserFcmToken(userId, fcmToken)
18 Success
19 Generate JWT Token (with roles, user_id)
20 JWT Token & User Roles
21 200 OK (token, roles)
22 Save JWT locally & redirect to role dashboard
23 Display Home Screen / Role Selection

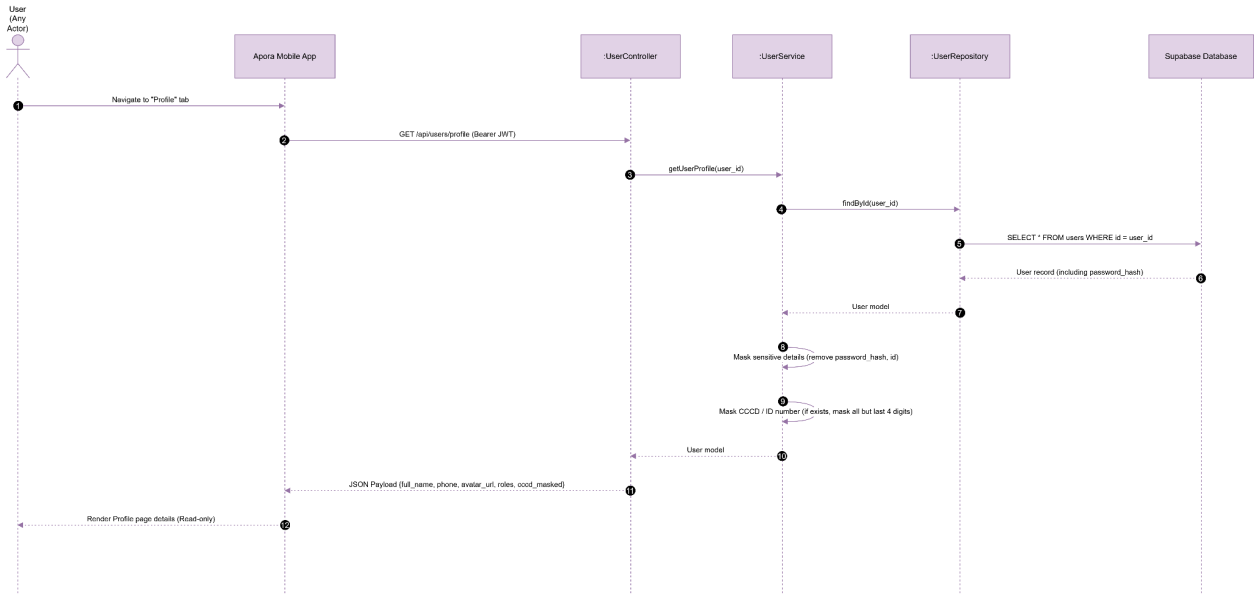
UC02 : Log out



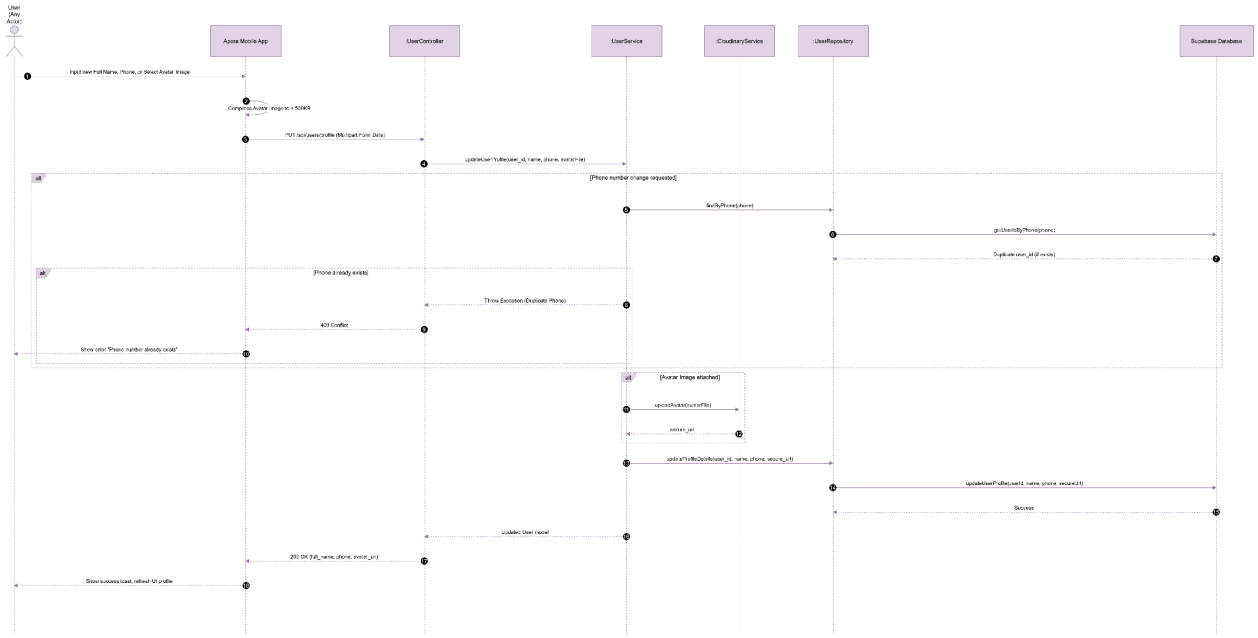
UC03 : Forgot Password



UC04 : View Personal Profile



UC05 : Update Personal Profile

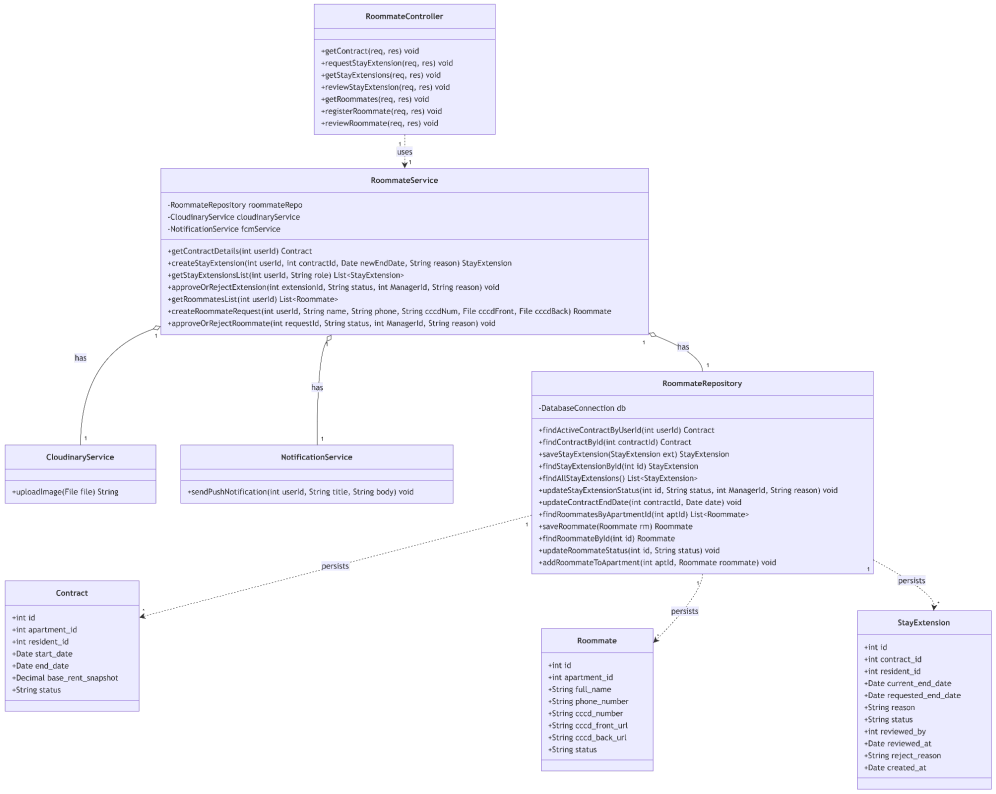


3.2 Module 2: Tenancy & Roommate Management (UC06 - UC12)

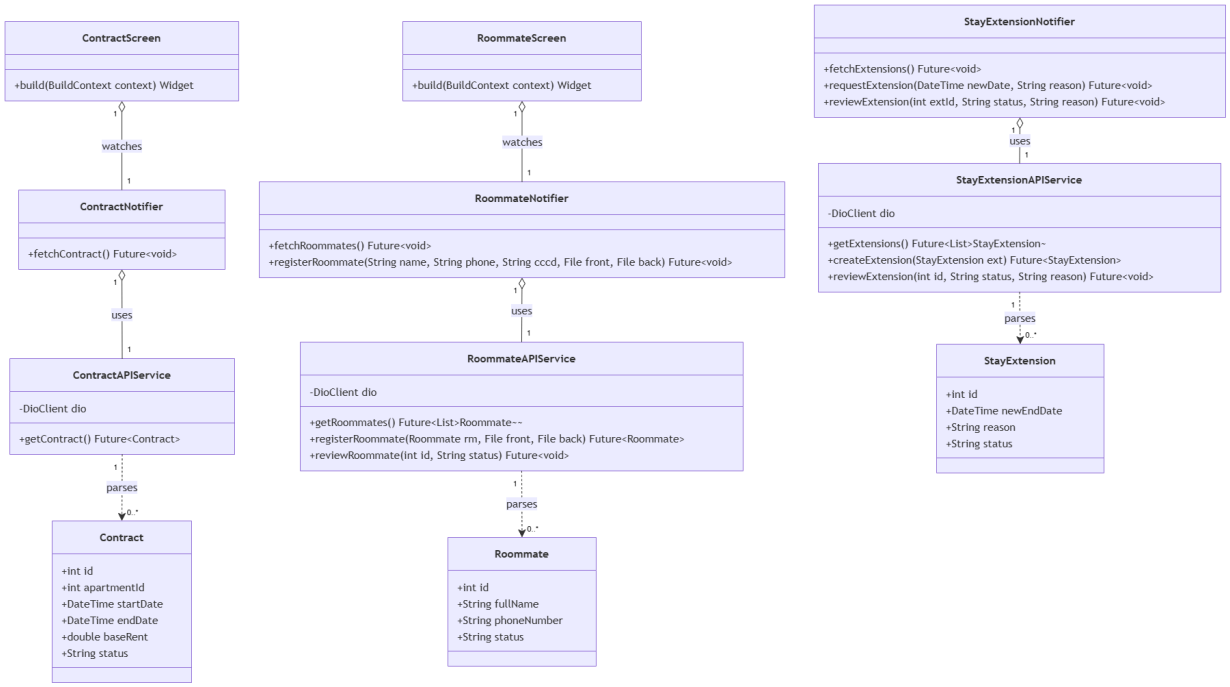
This module handles tenancy agreements, resident registration, and roommate management workflows.

3.2.1 Class Diagram

3.2.1.1. NextJS Backend Class Diagram



3.2.1.2. Flutter Client Class Diagram



3.2.2 Class Specifications

3.2.2.1 NextJS Backend Class Specifications

A. RoommateController

- **Purpose**: Exposes HTTP endpoints for contract detail retrieval, roommate management, and stay extension workflows. **Depends on RoommateService** to process requests.
- **Attributes:**
    - roommateService: RoommateService
- **Methods**:
    - getContract(req, res): GET /api/tenancy/contract. Fetches active contract data via RoommateService.getContractDetails().
    - requestStayExtension(req, res): POST /api/tenancy/extensions. Submits an extension request via RoommateService.createStayExtension().
    - getStayExtensions(req, res): GET /api/tenancy/extensions. Retrieves a list of extensions filtered by role via RoommateService.getStayExtensionsList().
    - reviewStayExtension(req, res): PUT /api/tenancy/extensions/:id/status. Managers approve/reject extension requests via RoommateService.approveOrRejectExtension().
    - getRoommates(req, res): GET /api/tenancy/roommates. Fetches the roommate roster via RoommateService.getRoommatesList().
    - registerRoommate(req, res): POST /api/tenancy/roommates/register. Handles multi-part file uploads and data registration via RoommateService.createRoommateRequest().
    - reviewRoommate(req, res): PUT /api/tenancy/roommates/:id/status. manager approve/reject pending roommates via RoommateService.approveOrRejectRoommate().

B. RoommateService

- **Purpose**: Encapsulates core business rules for tenancy durations, occupant limit counts, image compression, and push notification triggers. **Aggregates RoommateRepository, CloudinaryService, and NotificationService**.
- **Attributes:**
    - roommateRepository: RoommateRepository
    - cloudinaryService: CloudinaryService
    - notificationService: NotificationService
- **Methods**:
    - getContractDetails(userId): Fetches the active Contract from RoommateRepository. Dynamically calculates and appends remaining lease days.
    - createStayExtension(userId, contractId, newEndDate, reason): Validates active lease existence, ensures newEndDate is in the future, and saves a new StayExtension entity.
    - getStayExtensionsList(userId, role): Fetches all extensions via RoommateRepository for managers, or filters by userId for residents.
    - approveOrRejectExtension(extensionId, status, managerId, reason): Runs within a transaction block. Updates extension status, updates the Contract end date if approved, and calls NotificationService to alert the resident.
    - getRoommatesList(userId): Isolates data based on the user's active apartment. Returns a List<Roommate> and calculates the final occupancy headcount.

    - createRoommateRequest(userId, name, phone, cccdNum, cccdFront, cccdBack): Verifies apartment existence, compresses CCCD images, uploads them via CloudinaryService, and persists the Roommate entity with a PENDING status.
    - approveOrRejectRoommate(requestId, status, managerId, reason): Performs status change via RoommateRepository. If approved, links the roommate to the apartment and dispatches an FCM message via NotificationService.

C. RoommateRepository

- **Purpose**: Direct database communicator handling CRUD operations across contracts, roommates, and stay_extensions tables. **Maps query results into domain entities**.
- **Attributes:**
    - db: DatabaseConnection
- **Methods**:
    - findActiveContractByUserId(userId): Queries the active lease contract belonging to a resident.
    - findContractById(contractId): Returns a concrete Contract instance.
    - saveStayExtension(ext): Inserts a new stay extension record into the database.
    - findStayExtensionById(id): Fetches details of a specific extension request.
    - findAllStayExtensions(): Returns all submitted stay extension rows for Manager management.
    - updateStayExtensionStatus(id, status, managerId, reason): Commits approval/rejection audits to an extension record.
    - updateContractEndDate(contractId, date): Modifies the official end_date of a specific contract.
    - findRoommatesByApartmentId(aptId): Fetches all approved or registered occupants inside a given apartment unit.
    - saveRoommate(rm): Persists a new roommate entry.
    - findRoommateById(id): Returns a concrete Roommate instance.
    - updateRoommateStatus(id, status): Updates a roommate request state (e.g., APPROVED, REJECTED).
    - addRoommateToApartment(aptId, roommate): Internally associates an approved roommate to an active apartment profile.

3.2.2.2 Flutter Client Class Specifications

A. ContractNotifier

- **Purpose**: A Riverpod notifier managing the active lease contract state for the resident.
- **Attributes**:
    - contractAPIService: ContractAPIService
    - state: AsyncValue<Contract>
- **Methods**:
    - fetchContract(): Future<void>

B. RoommateNotifier

- **Purpose**: A Riverpod notifier managing roommate directories, registration submissions, and approval statuses.
- **Attributes**:
    - roommateAPIService: RoommateAPIService
    - state: AsyncValue<List<Roommate>>
- **Methods**:
    - fetchRoommates(): Future<void>
    - registerRoommate(name: String, phone: String, cccd: String, front: File, back: File): Future<void>

C. StayExtensionNotifier

- **Purpose**: A Riverpod notifier managing lease stay extension requests and approvals.
- **Attributes**:
    - stayExtensionAPIService: StayExtensionAPIService
    - state: AsyncValue<List<StayExtension>>
- **Methods**:
    - fetchExtensions(): Future<void>
    - requestExtension(newDate: DateTime, reason: String): Future<void>
    - reviewExtension(extId: int, status: String, reason: String): Future<void>

D. RoommateAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Roommate REST API endpoints.
- **Attributes**:
    - dio: DioClient
- **Methods**:
    - getRoommates(): Future<List<Roommate>>
    - registerRoommate(rm: Roommate, front: File, back: File): Future<Roommate>

- reviewRoommate(id: int, status: String): Future<void>

E. ContractAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Contract REST API endpoints.
- **Attributes**:
  - dio: DioClient
- **Methods**:
  - getContract(): Future<Contract>

F. StayExtensionAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Stay Extension REST API endpoints.
- **Attributes**:
  - dio: DioClient
- **Methods**:
  - getExtensions(): Future<List<StayExtension>>
  - createExtension(ext: StayExtension): Future<StayExtension>
  - reviewExtension(id: int, status: String, reason: String): Future<void>

G. Contract (Model)

- **Purpose**: Client-side data model representing a tenancy contract with local JSON serialization capabilities.
- **Attributes**:
  - id: int
  - apartmentId: int
  - startDate: DateTime
  - endDate: DateTime
  - baseRent: double
  - status: String

- **Methods**:
    - fromJson(json: Map<String, dynamic>): Contract
    - toJson(): Map<String, dynamic>

H. Roommate (Model)

- **Purpose**: Client-side data model representing a registered roommate entity with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - fullName: String
    - phoneNumber: String
    - status: String
- **Methods**:
    - fromJson(json: Map<String, dynamic>): Roommate
    - toJson(): Map<String, dynamic>

I. StayExtension (Model)

- **Purpose**: Client-side data model representing a lease stay extension request with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - newEndDate: DateTime
    - reason: String
    - status: String
- **Methods**:
    - fromJson(json: Map<String, dynamic>): StayExtension
    - toJson(): Map<String, dynamic>

J. ContractScreen

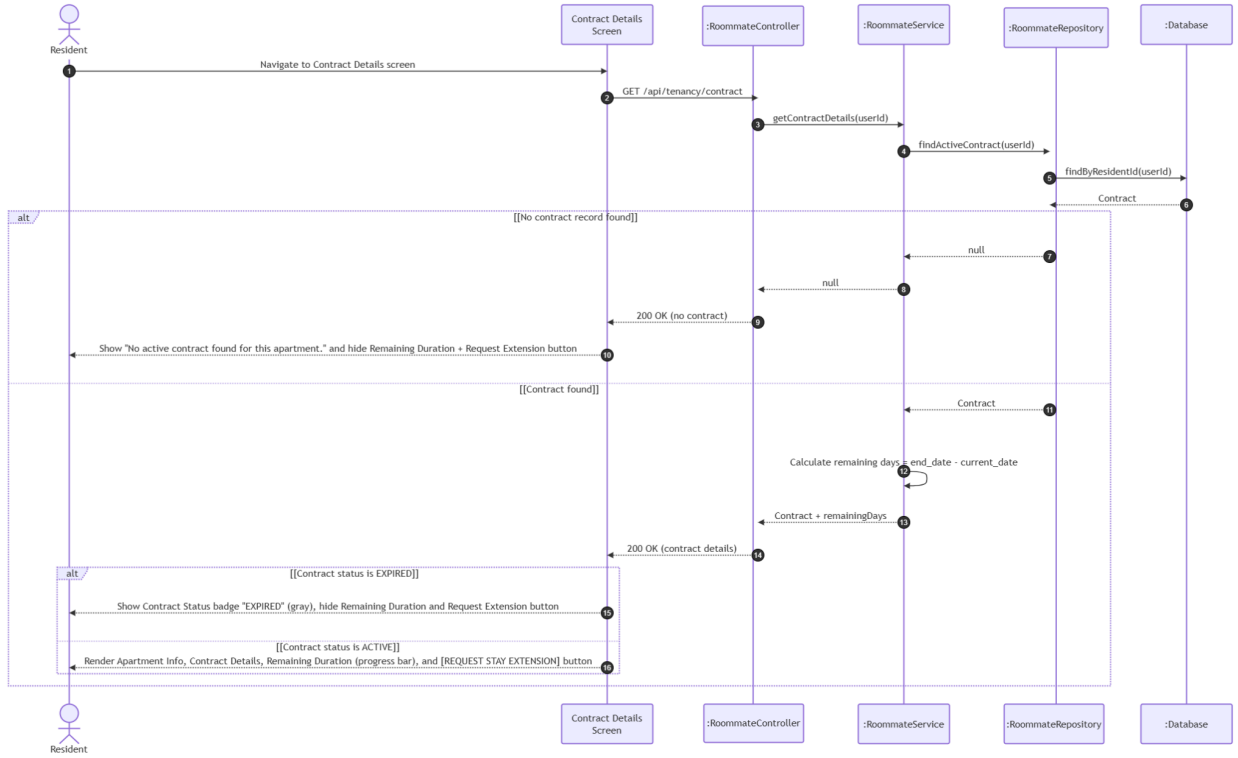
- **Purpose**: Flutter screen displaying active lease details, base rent snapshots, and duration counters.
- **Methods**:
    - build(context: BuildContext): Widget

K. RoommateScreen

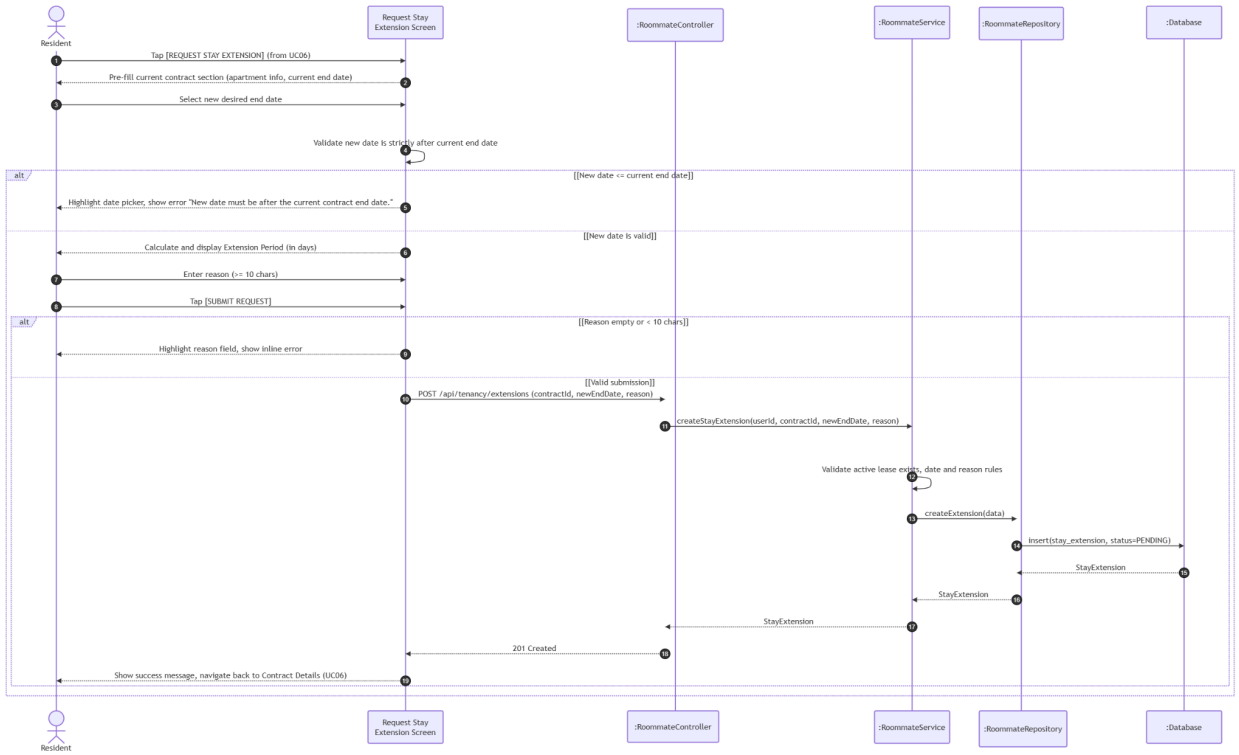
- **Purpose**: Flutter screen displaying the roommate list directory and registration forms.
- **Methods**:
    - build(context: BuildContext): Widget

3.2.3 Sequence Diagrams

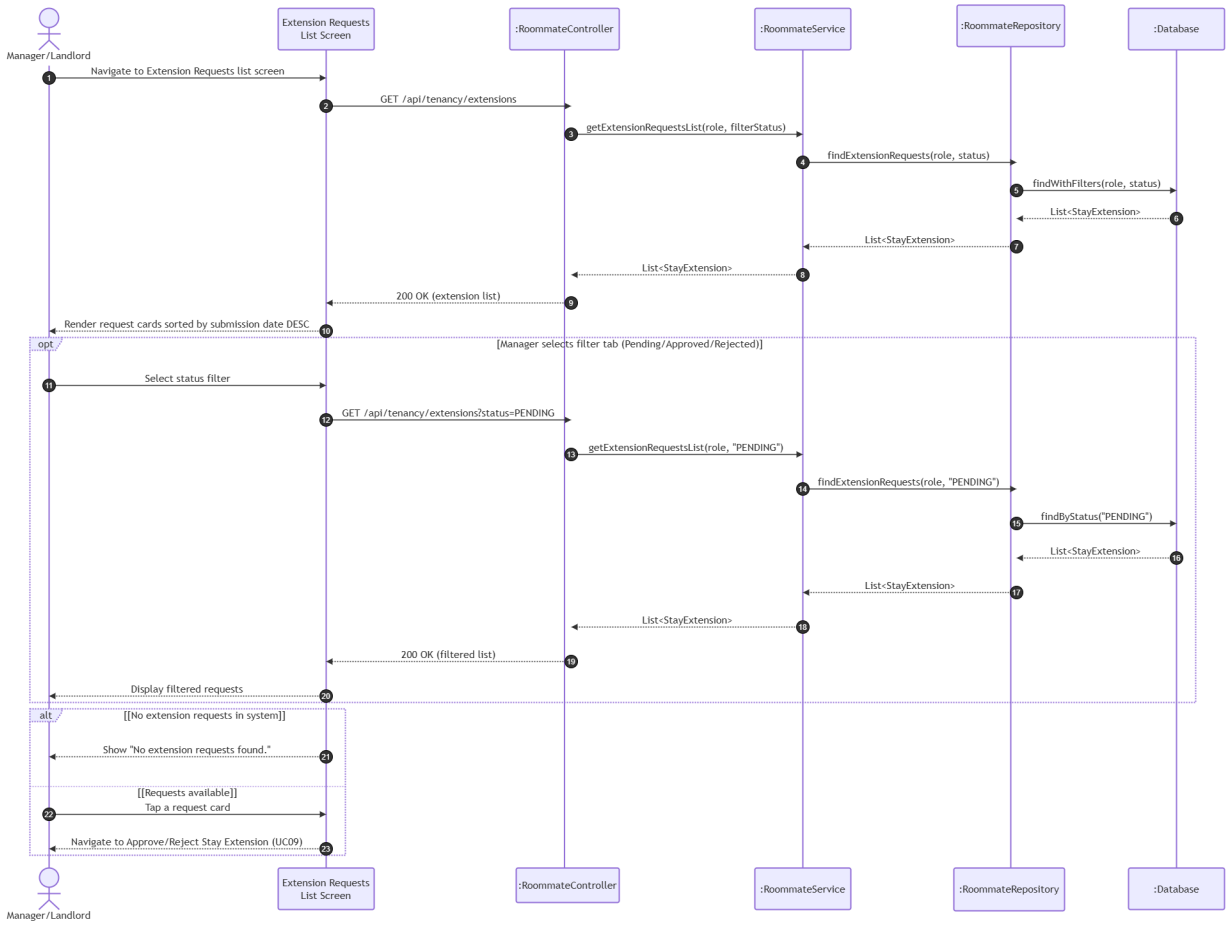
UC06: View Contract (FID-09)



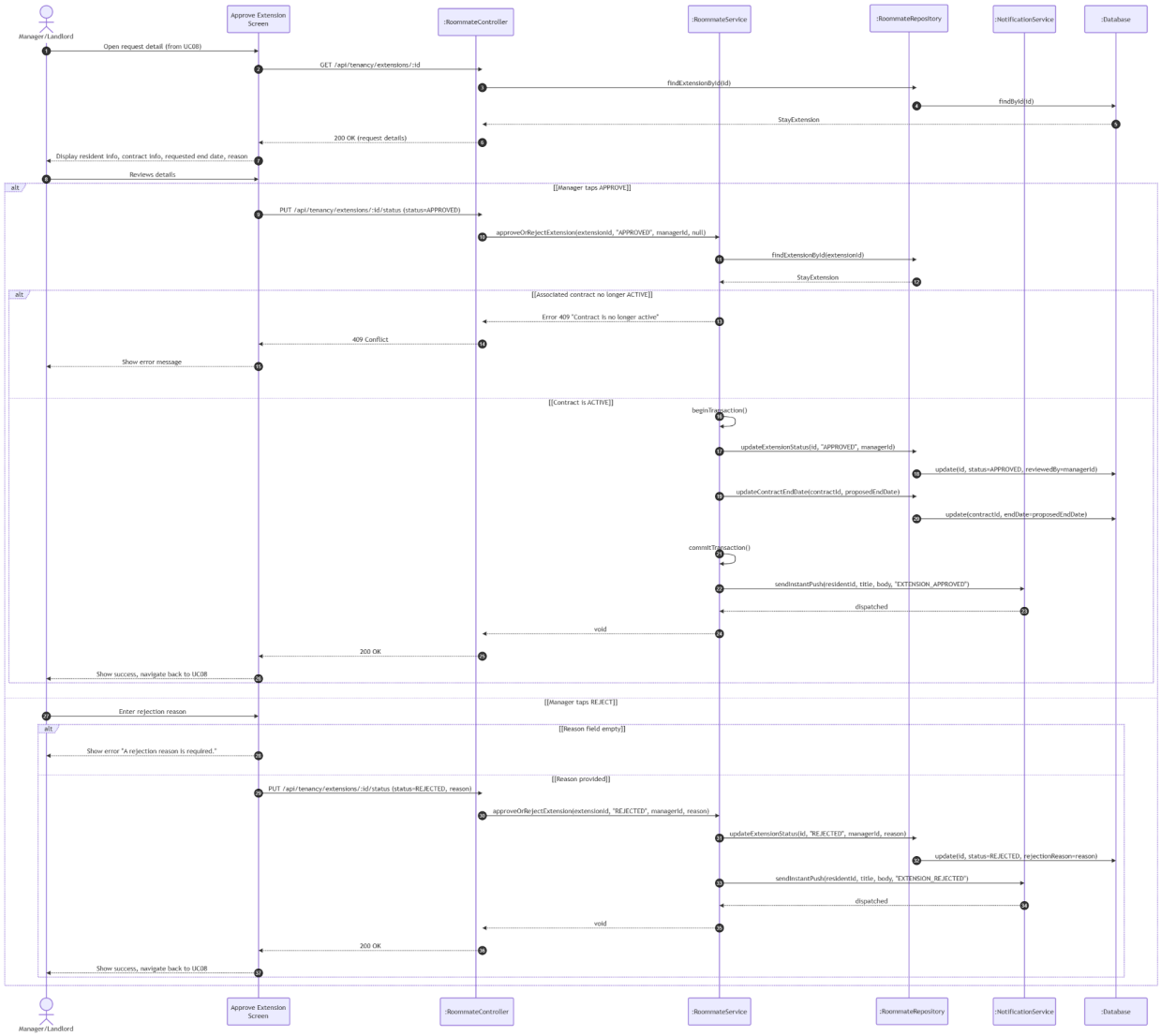
UC07: Request Stay Extension (FID-10)



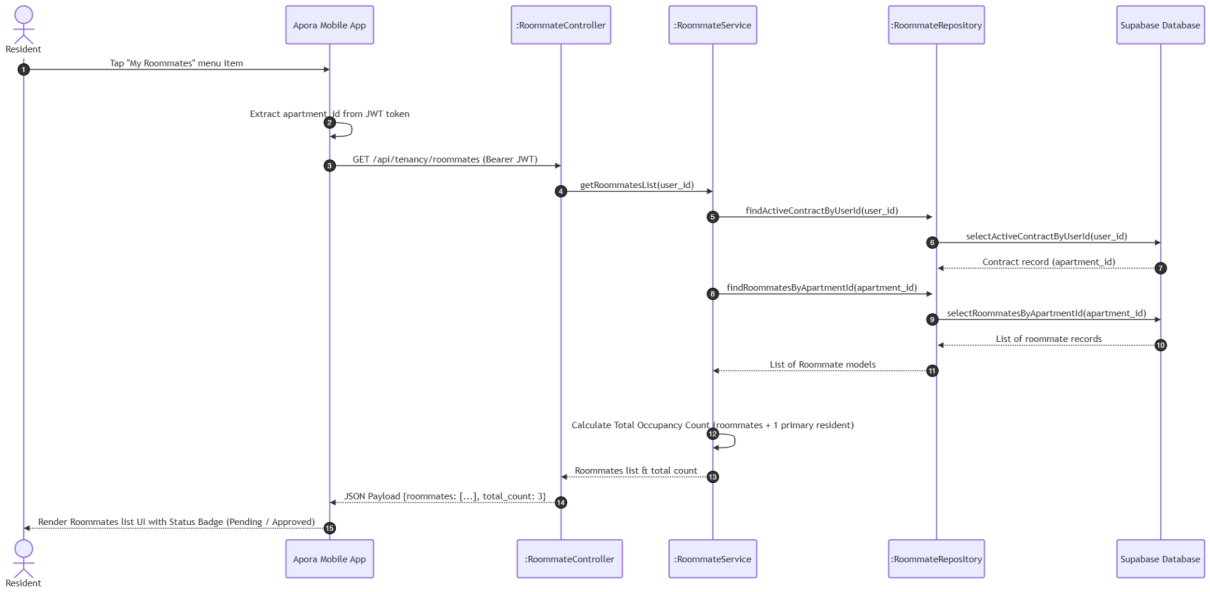
UC08: View Stay Extension Requests (FID-11)



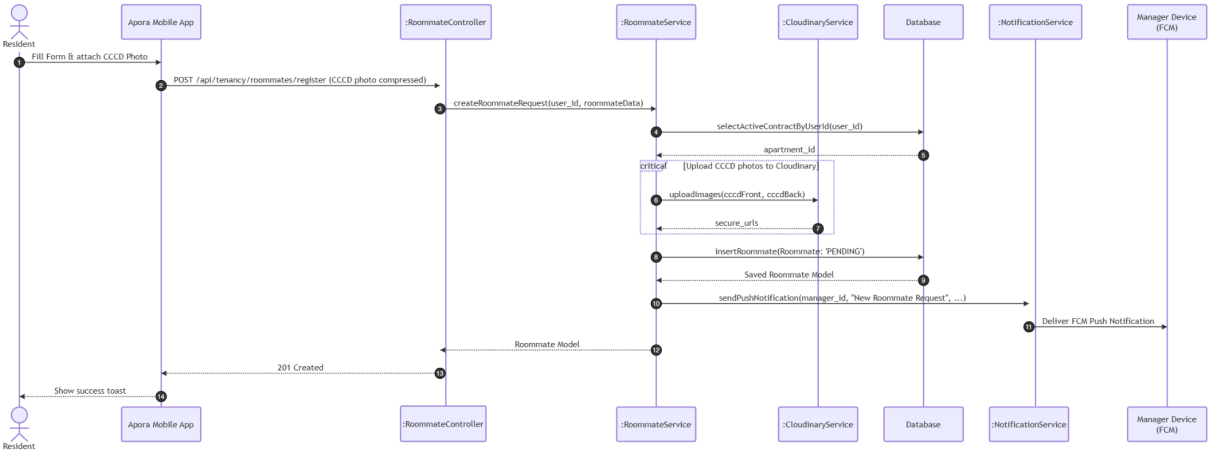
UC09: Approve/Reject Stay Extension Request (FID-12)



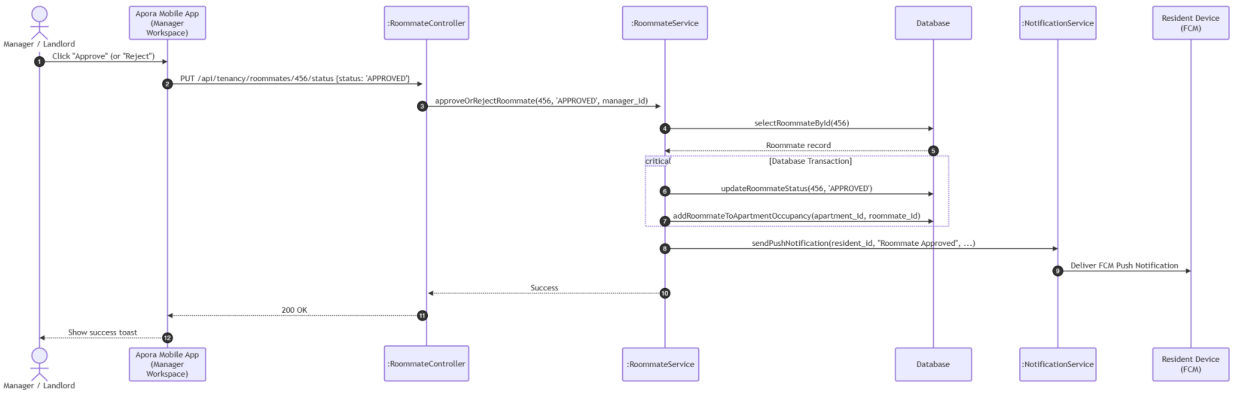
UC10: View Roommate List



UC11: Register Roommate



UC12: Approve Roommate Request

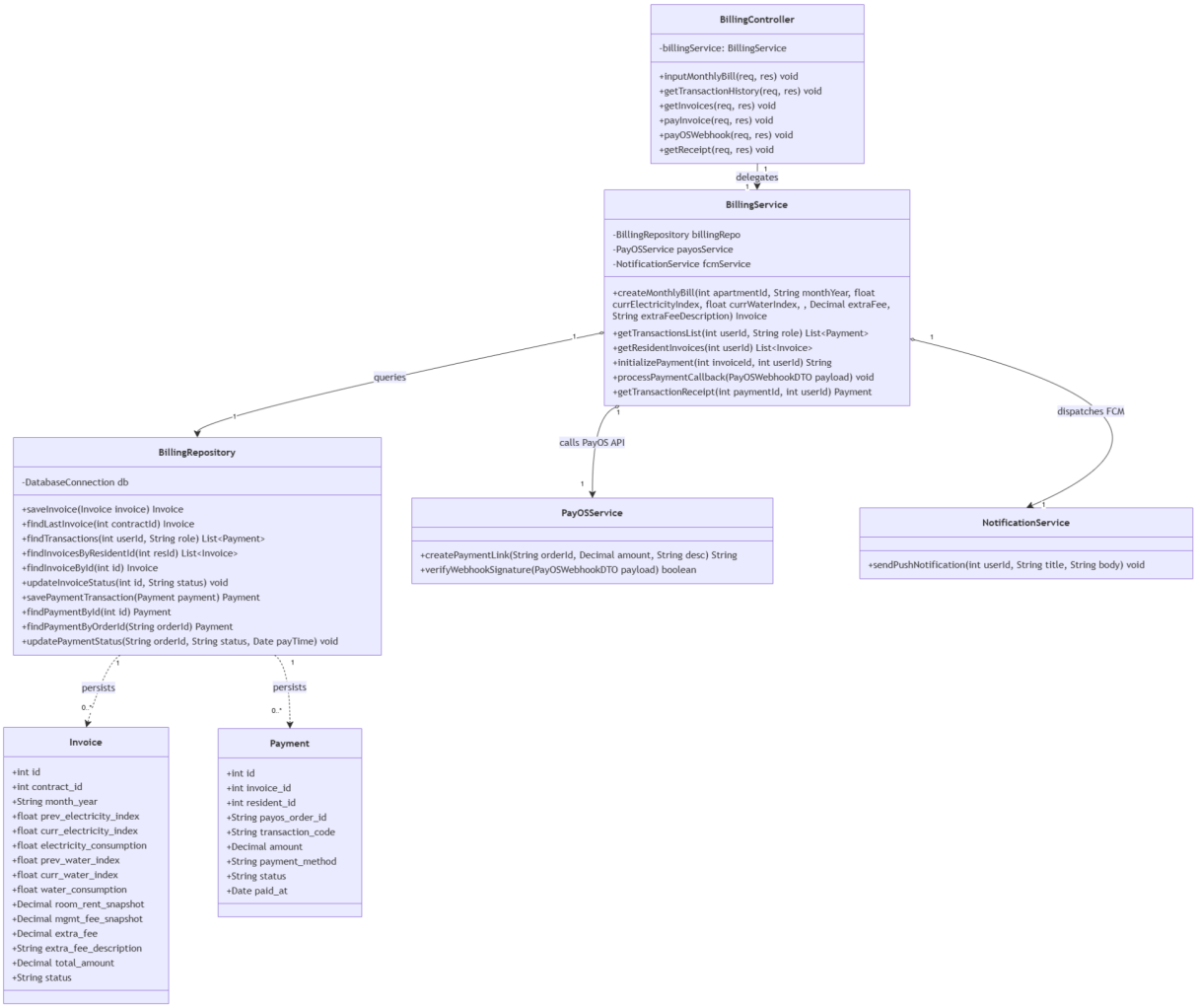


3.3 Module 3: Billing & Payment Management (UC13 - UC17)

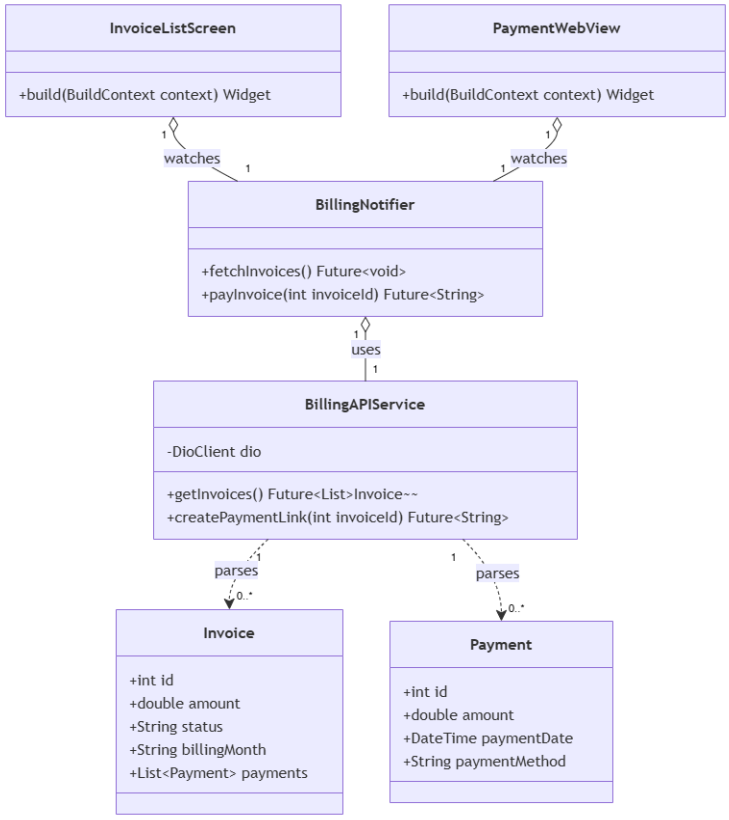
3.3.1 Class Diagram

This module handles monthly billing generation, transaction history tracking, and VietQR-based online payment integration.

3.3.1.1. NextJS Backend Class Diagram



3.3.1.2. Flutter Client Class Diagram



3.3.2 Class Specifications

3.3.2.1 NextJS Backend Class Specifications

A. BillingController

- **Purpose:** Exposes HTTP endpoints for bill generation, invoices, online payments, and webhook callbacks.
- **Attributes:**
    - billingService: BillingService
- **Methods:**
    - inputMonthlyBill(req, res): void
    - getTransactionHistory(req, res): void
    - getInvoices(req, res): void
    - payInvoice(req, res): void
    - payOSWebhook(req, res): void
    - getReceipt(req, res): void

B. BillingService

- **Purpose:** Orchestrates business logic for billing validation, database transactions, and payment integration.
- **Attributes:**

    - billingRepo: BillingRepository
    - payosService: PayOSService
    - fcmService: NotificationService
- **Methods:**
    - createMonthlyBill(apartmentId: int, monthYear: String, currElectricityIndex: float, currWaterIndex: float, extraFee: float, extraFeeDescription: String): Invoice
    - getTransactionsList(userId: int, role: String): List<Payment>
    - getResidentInvoices(userId: int): List<Invoice>
    - initializePayment(invoiceId: int, userId: int): String
    - processPaymentCallback(payload: PayOSWebhookDTO): void
    - getTransactionReceipt(paymentId: int, userId: int): Payment

C. BillingRepository

- **Purpose:** Handles persistent data access layer operations for billing tables.
- **Attributes:**
    - db: DatabaseConnection
- **Methods:**
    - saveInvoice(invoice: Invoice): Invoice
    - findLastInvoice(contractId: int): Invoice
    - findTransactions(userId: int, role: String): List<Payment>
    - findInvoicesByResidentId(resId: int): List<Invoice>
    - findInvoiceById(id: int): Invoice
    - updateInvoiceStatus(id: int, status: String): void
    - savePaymentTransaction(payment: Payment): Payment
    - findPaymentById(id: int): Payment
    - findPaymentByOrderId(orderId: String): Payment
    - updatePaymentStatus(orderId: String, status: String, payTime: Date): void

D. PayOSService

- **Purpose:** Integrates and communication with the external PayOS gateway.
- **Methods:**
    - createPaymentLink(orderId: String, amount: Decimal, desc: String): String
    - verifyWebhookSignature(payload: PayOSWebhookDTO): boolean

E. NotificationService

- **Purpose:** Dispatches event-driven push notifications to users via FCM.
- **Methods:**
    - sendPushNotification(userId: int, title: String, body: String): void

F. Invoice (Entity)

- **Purpose:** Data model representing a customer billing invoice.
- **Attributes:**
    - id: int
    - contract_id: int
    - month_year: String

- electricity_consumption: float
- water_consumption: float
- room_rent_snapshot: Decimal
- mgmt_fee_snapshot: Decimal
- extra_fee: Decimal
- extra_fee_description: String
- total_amount: Decimal
- status: String

G. Payment (Entity)

- **Purpose:** Data model representing a financial transaction log.
- **Attributes:**
  - id: int
  - invoice_id: int
  - resident_id: int
  - payos_order_id: String
  - transaction_code: String
  - amount: Decimal
  - payment_method: String
  - status: String
  - paid_at: Date

3.3.2.2 Flutter Client Class Specifications

A. BillingNotifier

- **Purpose**: A Riverpod notifier managing the state of billing lists, checkout URL creation, and payment statuses for residents.
- **Attributes**:
  - billingAPIService: BillingAPIService
  - state: AsyncValue<List<Invoice>>
- **Methods**:
  - fetchInvoices(): Future<void>
  - payInvoice(invoiceId: int): Future<String>

B. BillingAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Billing REST API endpoints.
- **Attributes**:
  - dio: DioClient

- **Methods**:
    - getInvoices(): Future<List<Invoice>>
    - createPaymentLink(invoiceId: int): Future<String>

C. Invoice (Model)

- **Purpose**: Client-side data model representing a customer billing invoice with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - amount: double
    - status: String
    - billingMonth: String
    - payments: List<Payment>
- **Methods**:
    - fromJson(json: Map<String, dynamic>): Invoice
    - toJson(): Map<String, dynamic>

D. Payment (Model)

- **Purpose**: Client-side data model representing a financial transaction log with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - amount: double
    - paymentDate: DateTime
    - paymentMethod: String
- **Methods**:
    - fromJson(json: Map<String, dynamic>): Payment
    - toJson(): Map<String, dynamic>

E. InvoiceListScreen

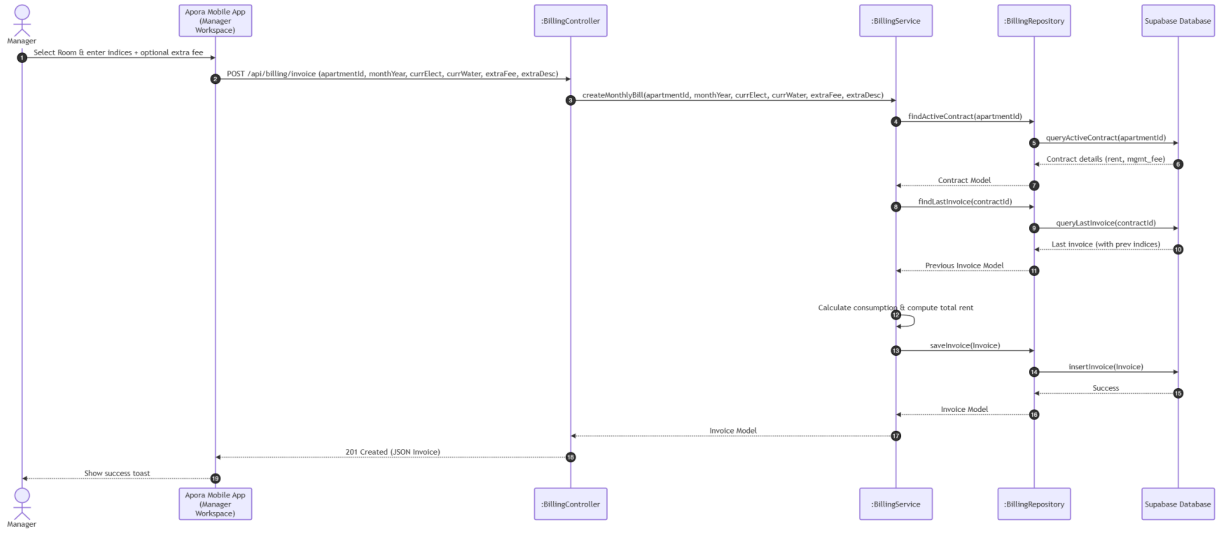
- **Purpose**: Stateful Flutter screen displaying active invoices, billing details, and checkout redirections for residents.
- **Methods**:
    - build(context: BuildContext): Widget

F. PaymentWebView

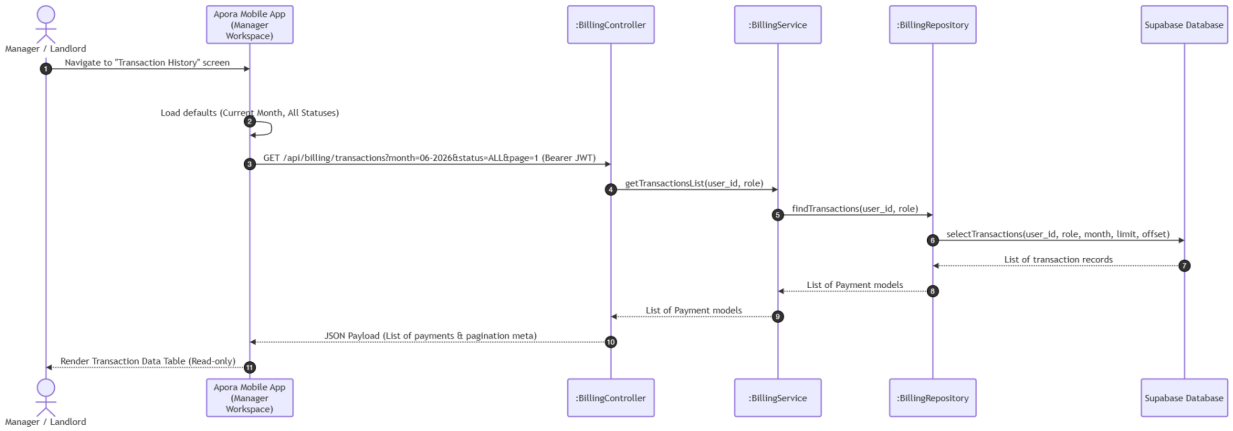
- **Purpose**: Webview screen handling secure payment checkout page redirection from PayOS.
- **Methods**:
    - build(context: BuildContext): Widget

3.3.3 Sequence Diagrams

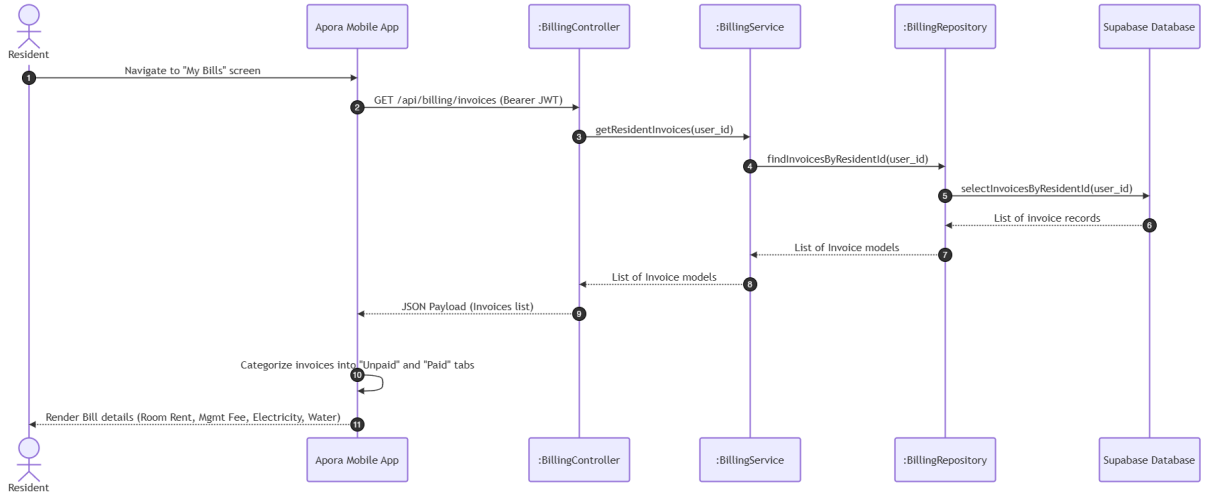
UC13: Input Monthly Bills



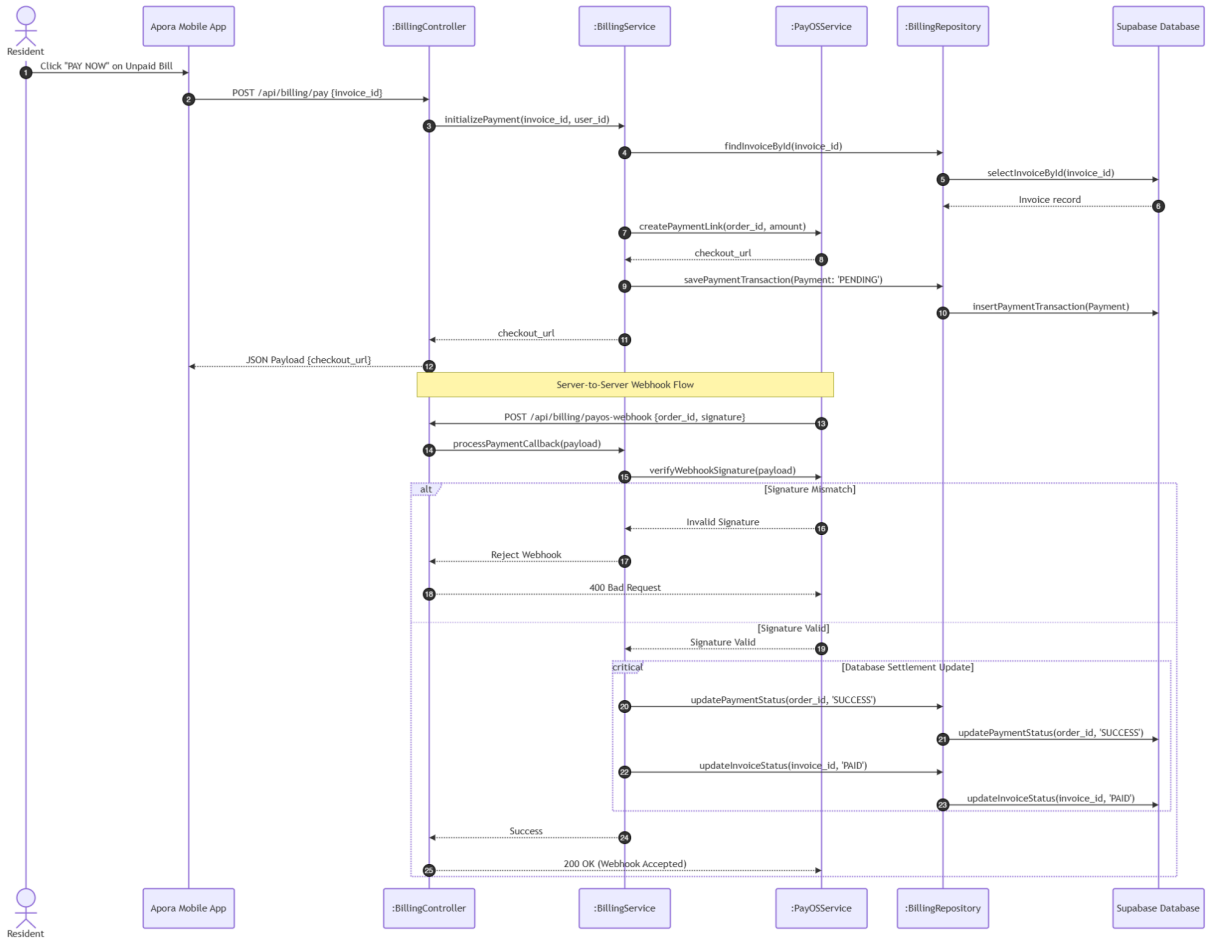
UC14: View Transaction History



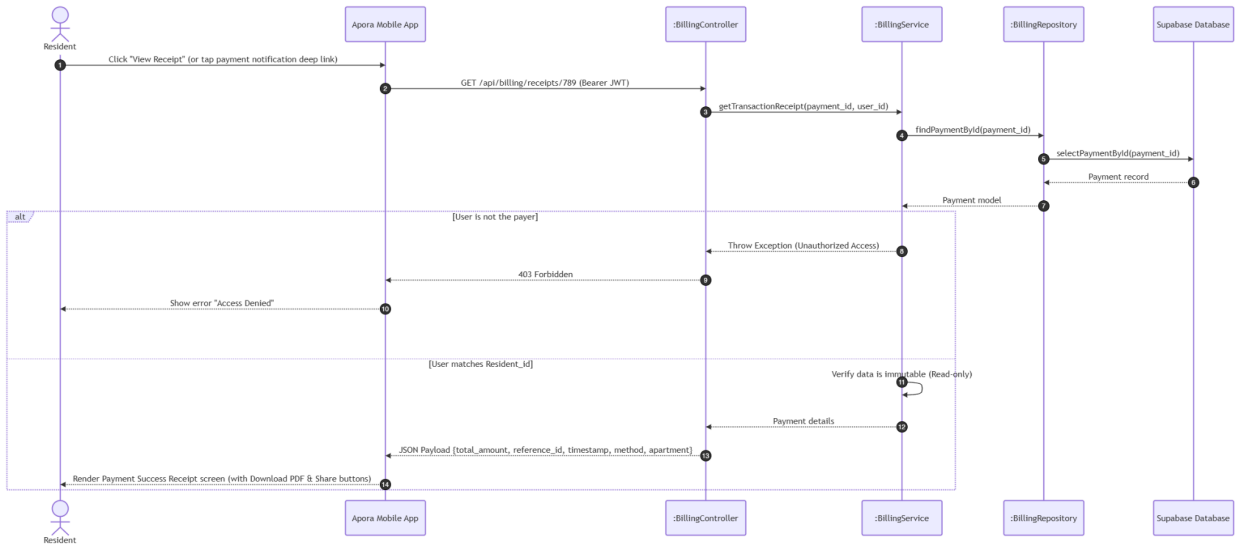
UC15: View Invoice List & Details



UC16: Pay Bills



UC17: View Payment Receipt

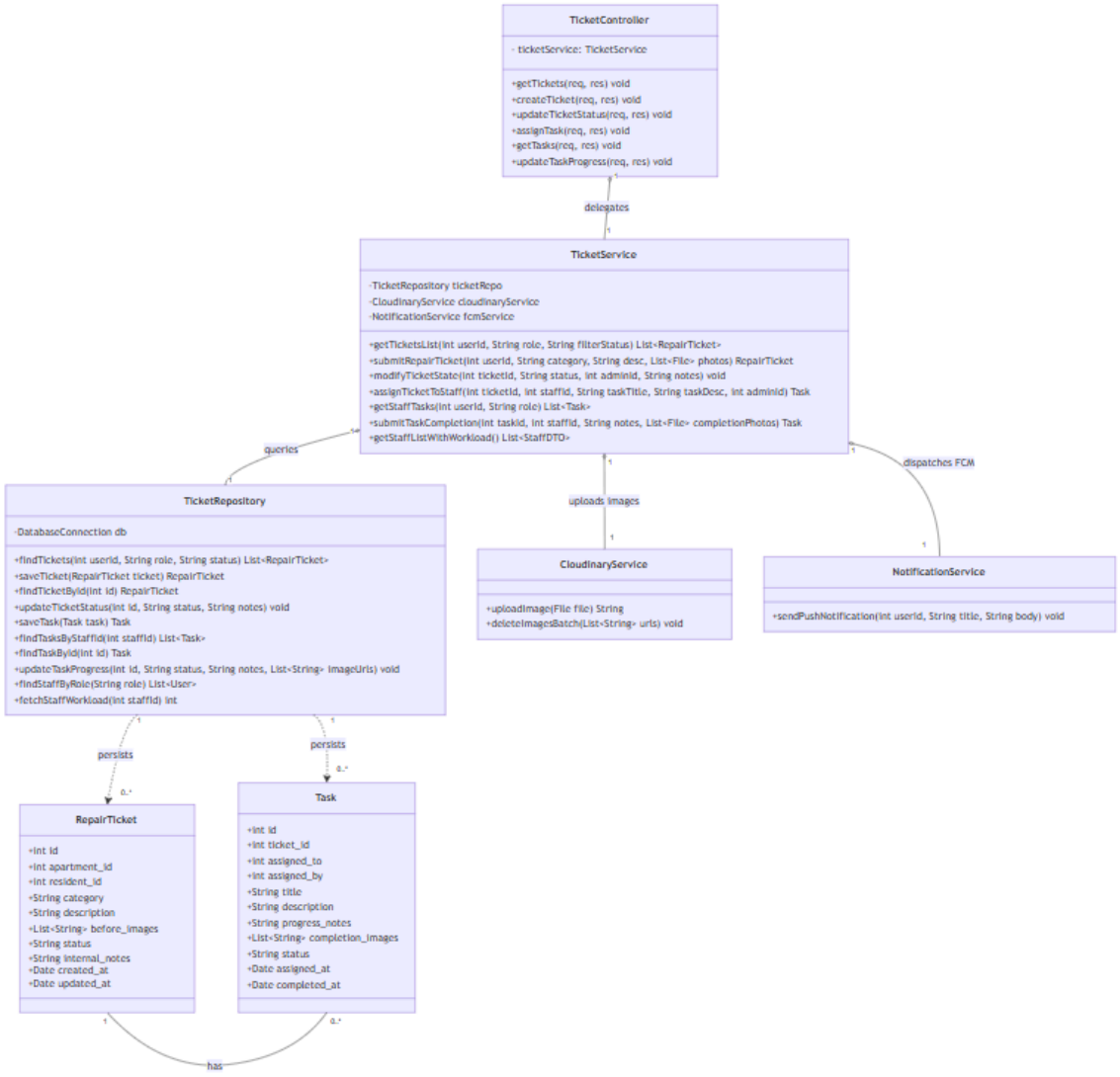


3.4 Module 4: Incident & Task Management (UC18 - UC23)

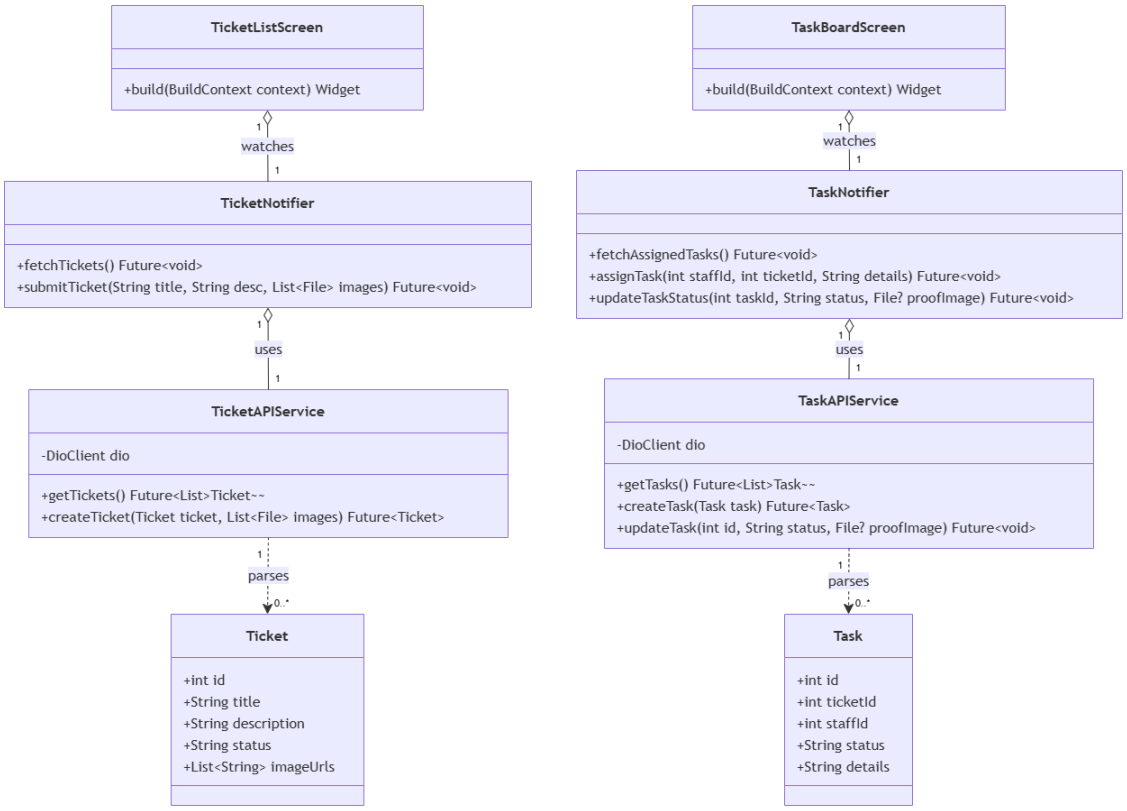
3.4.1 Class Diagram

This module handles resident incident reporting, ticket management, task assignment to staff, and work progress updates.

3.4.1.1. NextJS Backend Class Diagram



3.4.1.2. Flutter Client Class Diagram



3.4.2 Class Specifications

3.4.2.1 NextJS Backend Class Specifications

A. TicketController

Purpose: Exposes endpoints for residents to report issues, and for Manager/Landlord/Staff to manage the ticket lifecycle and task assignments.

Attributes:
-ticketService: TicketService

Methods:

- getTickets(req, res): GET /api/tickets. Retrieves the list of repair tickets, filterable by status.
- createTicket(req, res): POST /api/tickets. Creates a new repair ticket submitted by a resident.
- updateTicketStatus(req, res): PUT /api/tickets/:id/status. Updates the status and internal notes of a ticket.
- assignTask(req, res): POST /api/tickets/:id/assign. Assigns a pending ticket to a staff member by creating a new task.
- getTasks(req, res): GET /api/tasks. Retrieves the list of tasks assigned to the logged-in staff member.

- updateTaskProgress(req, res): PUT /api/tasks/:id/progress. Updates task status, progress notes, and completion photos.

B. TicketService

Purpose: Encapsulates business logic for ticket lifecycle management, task assignment, staff workload balancing, and resolution proof validation.

Attributes:
-ticketRepo: TicketRepository
-cloudinaryService: CloudinaryService
-fcmService: NotificationService

Methods:

- getTicketsList(userId, role, filterStatus): Applies role-based access filtering (BR-39) and optional status filter; returns the ticket list sorted by creation date.
- submitRepairTicket(userId, category, desc, photos): Validates mandatory category and description length (BR-36), enforces the 3-image/5MB limit (BR-37), compresses images on upload (BR-10), and creates a new ticket with status PENDING.
- modifyTicketState(ticketId, status, managerId, notes): Validates the requested transition against the allowed workflow PENDING → ASSIGNED → PROCESSING → RESOLVED (BR-40); if status is CANCELLED, purges ticket images via CloudinaryService (BR-38); dispatches an FCM notification on success (BR-18).
- assignTicketToStaff(ticketId, staffId, taskTitle, taskDesc, managerId): Validates the ticket is still PENDING; creates a new Task linked to the ticket and selected staff member; updates ticket status to ASSIGNED; notifies the assigned staff.
- getStaffTasks(userId, role): Retrieves the tasks assigned to the requesting staff member.
- submitTaskCompletion(taskId, staffId, notes, completionPhotos): Validates that at least one completion photo is attached before allowing completion (BR-43); uploads photos via CloudinaryService; updates the task status to COMPLETED and synchronizes the related ticket status to RESOLVED.
- getStaffListWithWorkload(): Retrieves all Security Guard/Technician accounts and computes each member's current active task count to populate the Assign Task staff selection list.

C. TicketRepository

Purpose: Direct database communicator for the repair_tickets and tasks tables.

Attributes:
-db: DatabaseConnection

Methods:

- findTickets(userId, role, status): Queries tickets filtered by role-based access and optional status.
- saveTicket(ticket): Inserts a new repair ticket record.

- findTicketById(id): Reads a single ticket record by ID.
- updateTicketStatus(id, status, notes): Updates the status and internal notes fields of a ticket.
- saveTask(task): Inserts a new task record.
- findTasksByStaffId(staffId): Queries tasks assigned to a specific staff member.
- findTaskById(id): Reads a single task record by ID.
- updateTaskProgress(id, status, notes, imageUrls): Updates the status, progress notes, and completion images of a task.
- findStaffByRole(role): Queries all active user accounts matching the given staff role.
- fetchStaffWorkload(staffId): Counts the number of currently active (non-completed) tasks assigned to a staff member.

D. CloudinaryService *(Shared Service)*

Purpose: Handles media asset storage and deletion for ticket and task images.

Methods:

- uploadImage(file): Compresses and uploads an image file, returning the resulting URL.
- deleteImagesBatch(urls): Permanently removes a batch of image files from cloud storage.

E. NotificationService *(Shared Service — defined in Module 5)*

Purpose: Dispatches FCM push notifications to relevant users upon ticket or task status changes.

Methods:

- sendPushNotification(userId, title, body): Retrieves the user's active device token and sends a push notification payload via Firebase Cloud Messaging.

F. RepairTicket (Entity)

Purpose: Data model representing an incident or maintenance issue reported by a resident.

Attributes:
+id: int, +apartment_id: int, +resident_id: int, +category: String, +description: String, +before_images: List<String>, +status: String, +internal_notes: String, +created_at: Date, +updated_at: Date

G. Task (Entity)

Purpose: Data model representing a work assignment created from a repair ticket and assigned to a staff member.

Attributes:
+id: int, +ticket_id: int, +assigned_to: int, +assigned_by: int, +title: String, +description: String, +progress_notes: String, +completion_images: List<String>, +status: String, +assigned_at: Date, +completed_at: Date

3.4.2.2 Flutter Client Class Specifications

A. TicketNotifier

- **Purpose**: A Riverpod notifier managing active incident tickets, status updates, and new ticket submissions.
- **Attributes**:
    - ticketAPIService: TicketAPIService
    - state: AsyncValue<List<Ticket>>
- **Methods**:
    - fetchTickets(): Future<void>
    - submitTicket(title: String, desc: String, images: List<File>): Future<void>

B. TaskNotifier

- **Purpose**: A Riverpod notifier managing operational tasks assigned to staff and status updates.
- **Attributes**:
    - taskAPIService: TaskAPIService
    - state: AsyncValue<List<Task>>
- **Methods**:
    - fetchAssignedTasks(): Future<void>
    - assignTask(staffId: int, ticketId: int, details: String): Future<void>
    - updateTaskStatus(taskId: int, status: String, proofImage: File?): Future<void>

C. TicketAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Ticket REST API endpoints.
- **Attributes**:
    - dio: DioClient
- **Methods**:
    - getTickets(): Future<List<Ticket>>
    - createTicket(ticket: Ticket, images: List<File>): Future<Ticket>

D. TaskAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Task REST API endpoints.
- **Attributes**:
    - dio: DioClient
- **Methods**:
    - getTasks(): Future<List<Task>>
    - createTask(task: Task): Future<Task>
    - updateTask(id: int, status: String, proofImage: File?): Future<void>

E. Ticket (Model)

- **Purpose**: Client-side data model representing an incident ticket with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - title: String
    - description: String
    - status: String
    - imageUrls: List<String>
- **Methods**:
    - fromJson(json: Map<String, dynamic>): Ticket
    - toJson(): Map<String, dynamic>

F. Task (Model)

- **Purpose**: Client-side data model representing a staff task with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - ticketId: int
    - staffId: int
    - status: String
    - details: String
- **Methods**:
    - fromJson(json: Map<String, dynamic>): Task
    - toJson(): Map<String, dynamic>

G. TicketListScreen

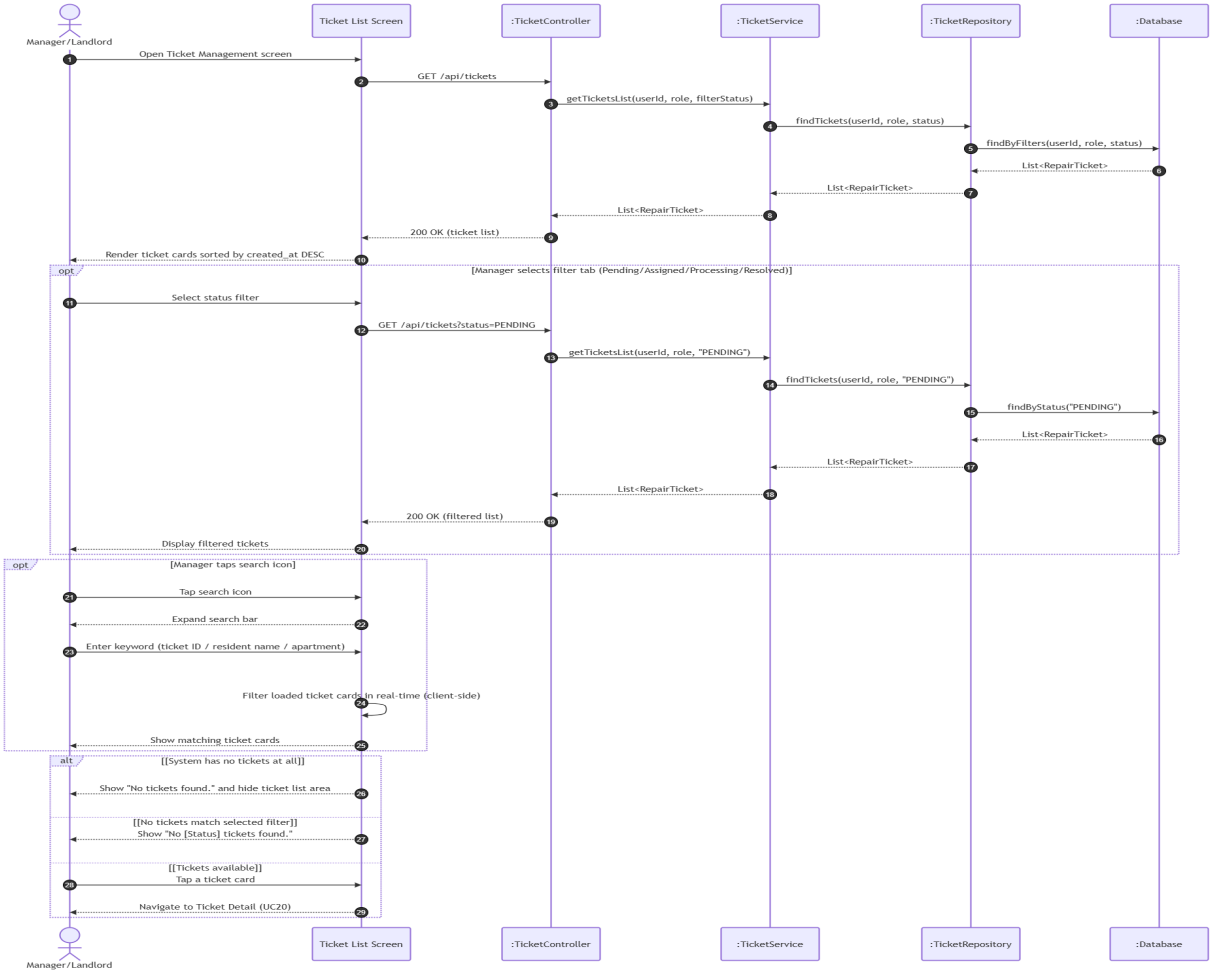
- **Purpose**: Flutter screen displaying tenant incident reports and status trackers.
- **Methods**:
    - build(context: BuildContext): Widget

H. TaskBoardScreen

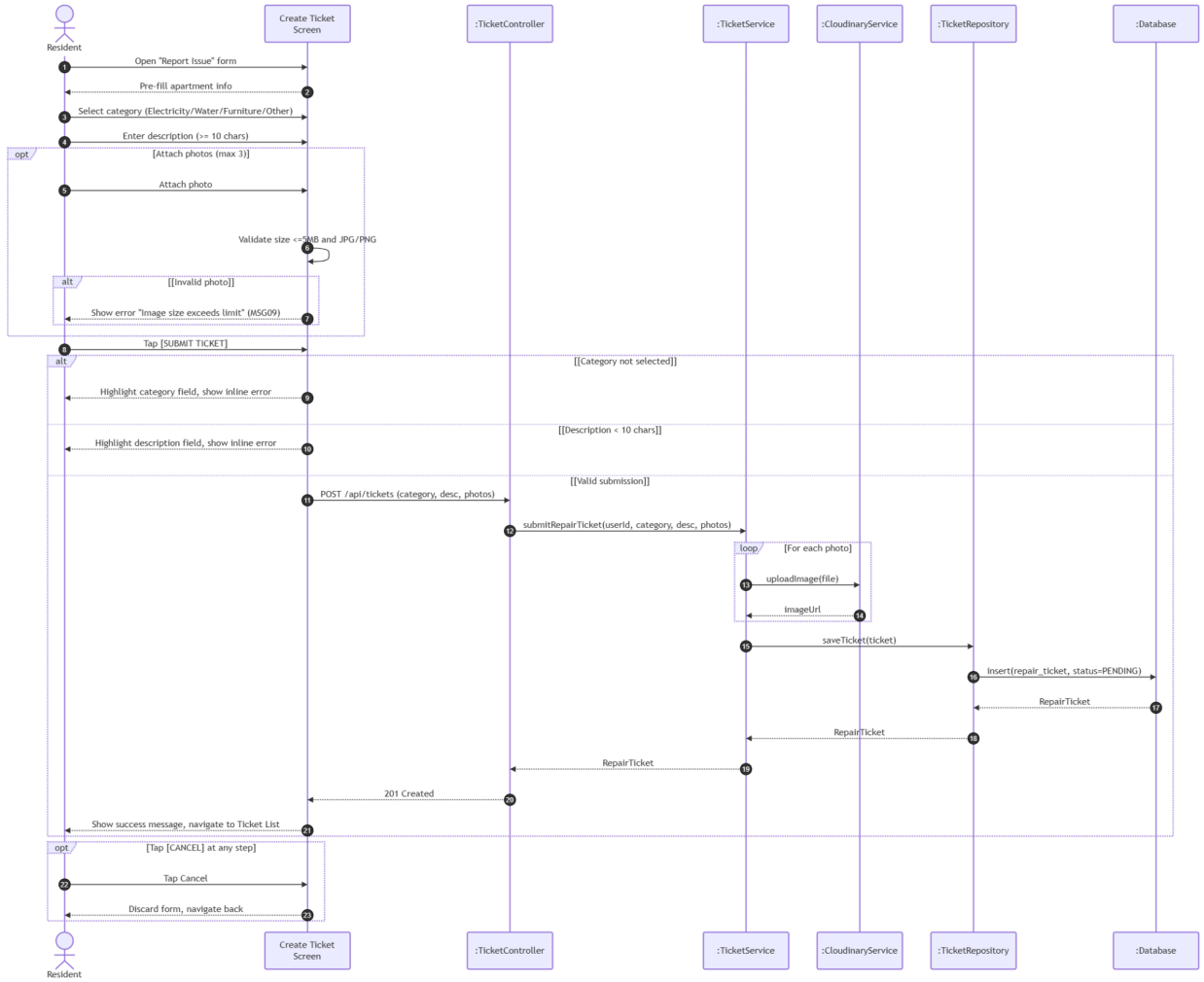
- **Purpose**: Flutter screen displaying assigned tasks for operational staff (janitor, technician, guard).
- **Methods**:
    - build(context: BuildContext): Widget

3.4.3 Sequence Diagrams

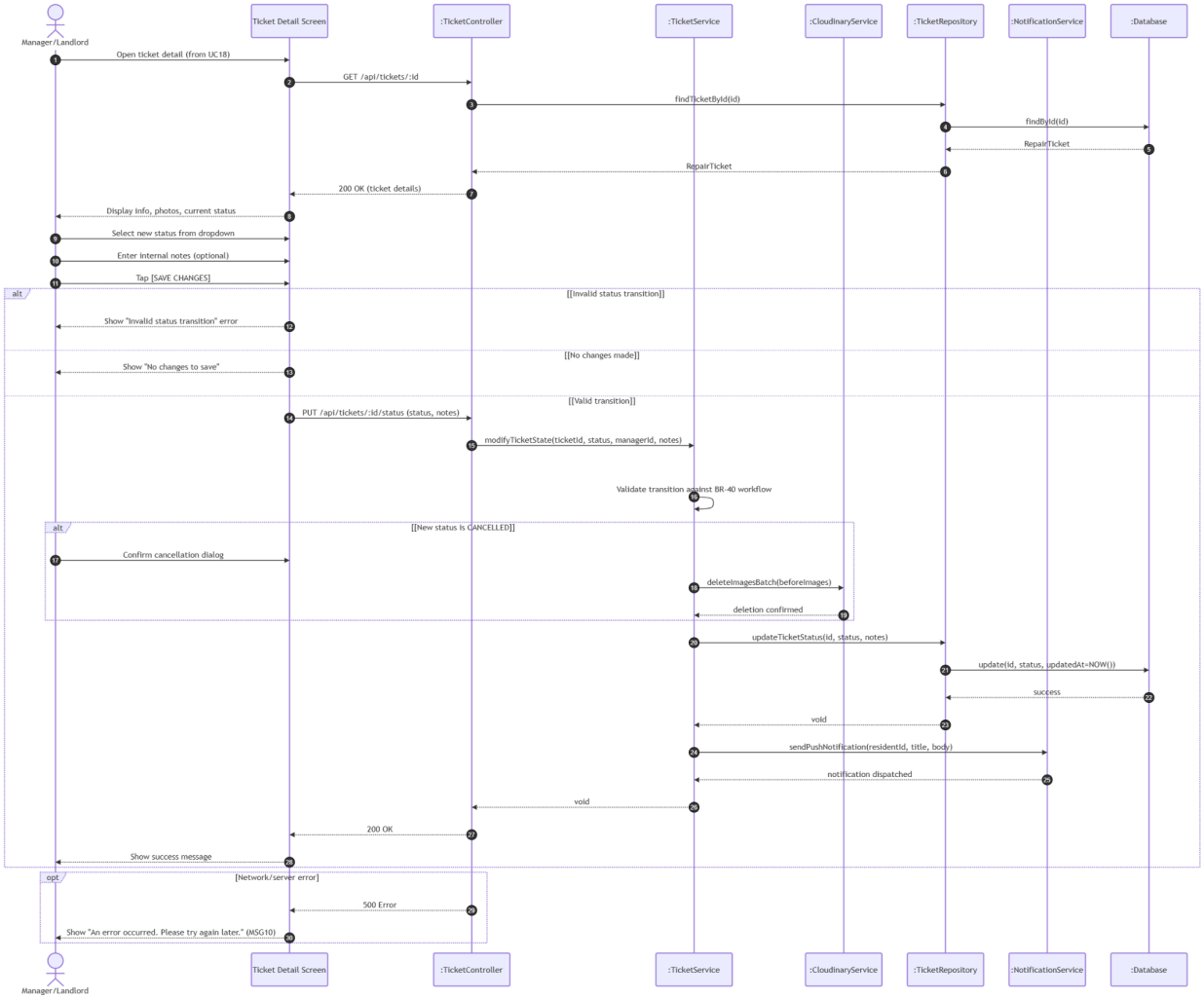
UC18: View Ticket List (FID-18)



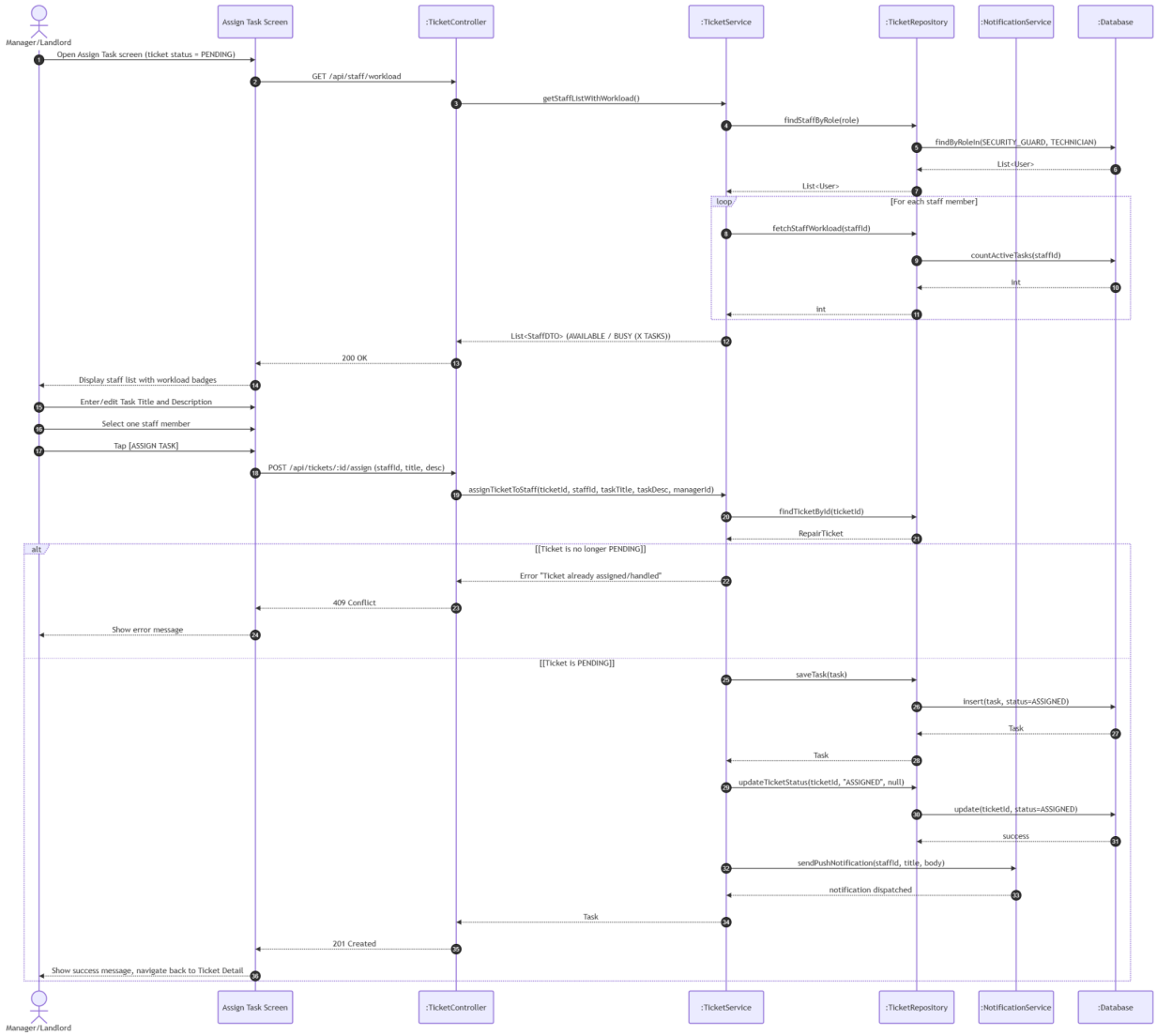
UC19: Create Ticket (FID-19)



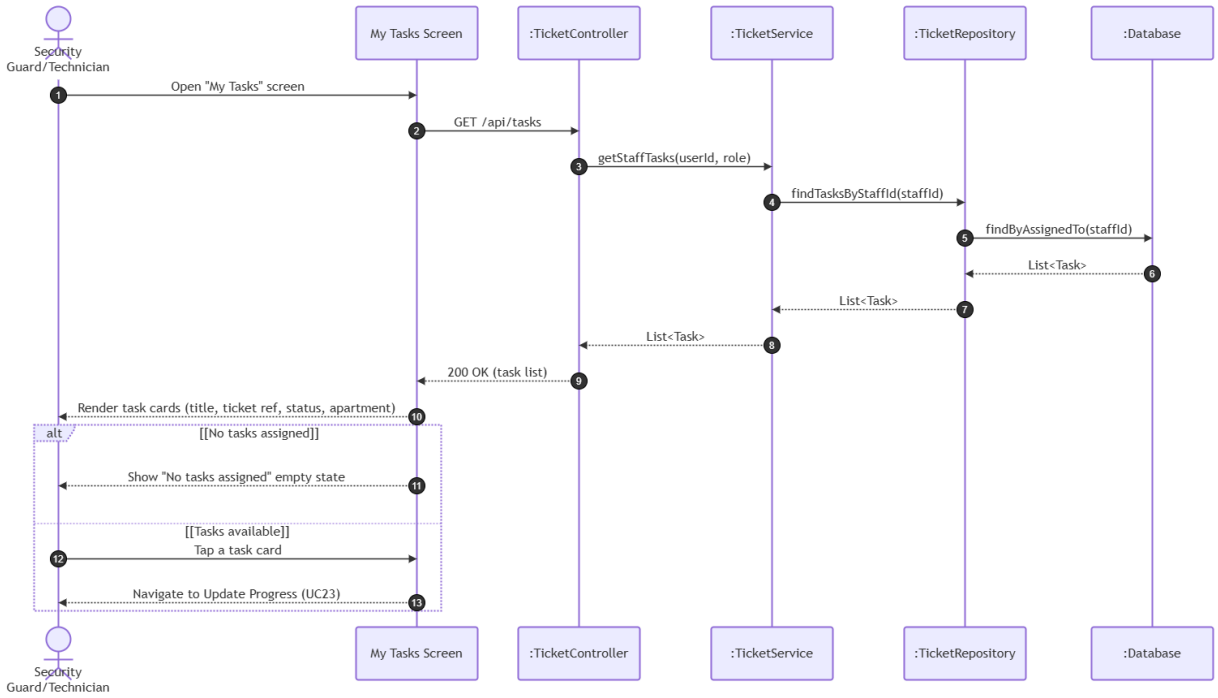
UC20: Update Ticket Status (FID-20)



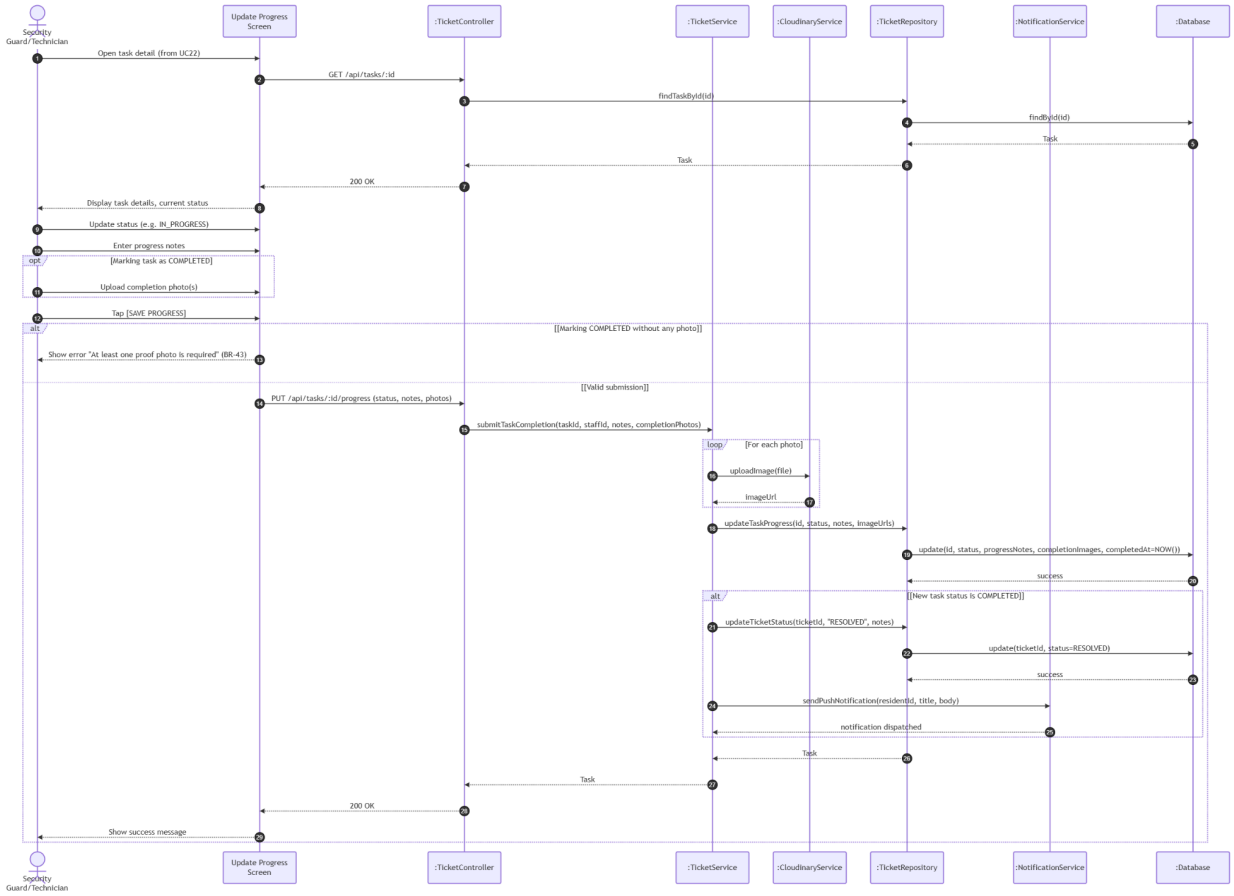
UC21: Assign Task (FID-21)



UC22: View Task List (FID-22)



UC23: Update Progress (FID-23)

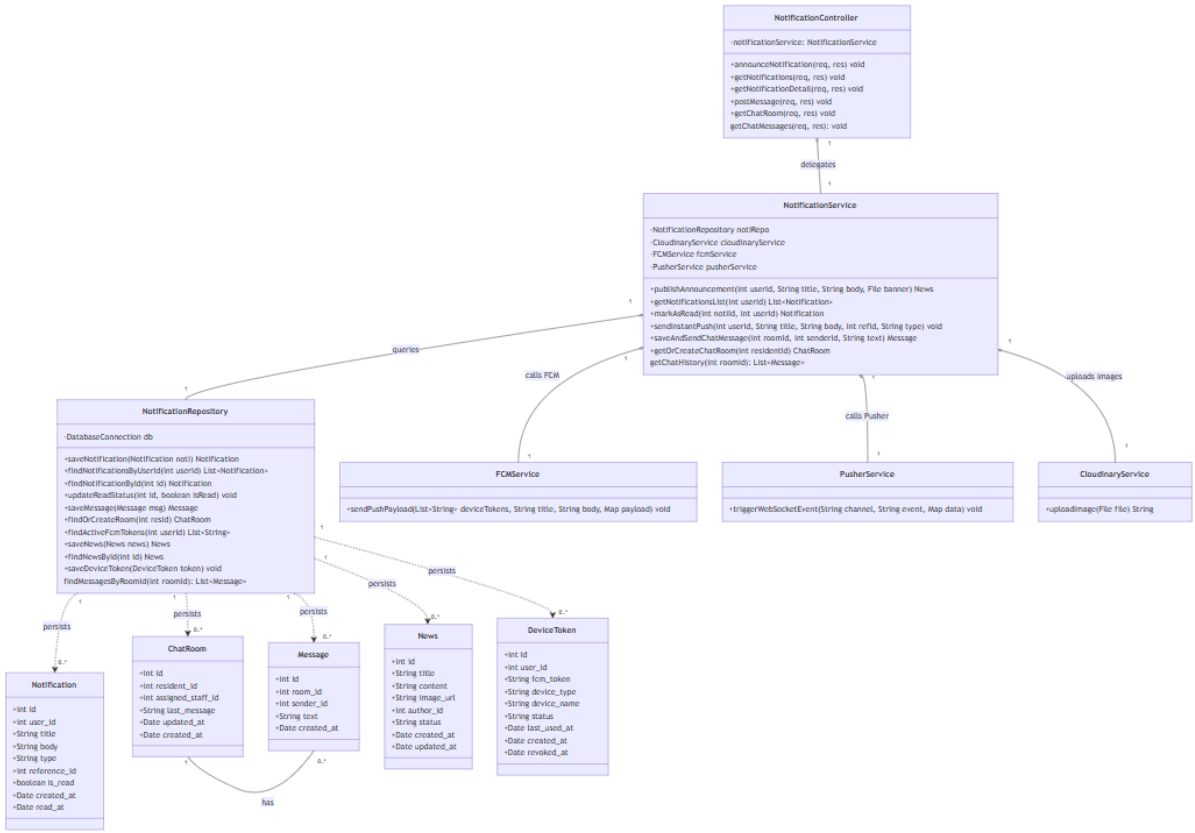


3.5 Module 5: Communication & Notifications (UC24 - UC28)

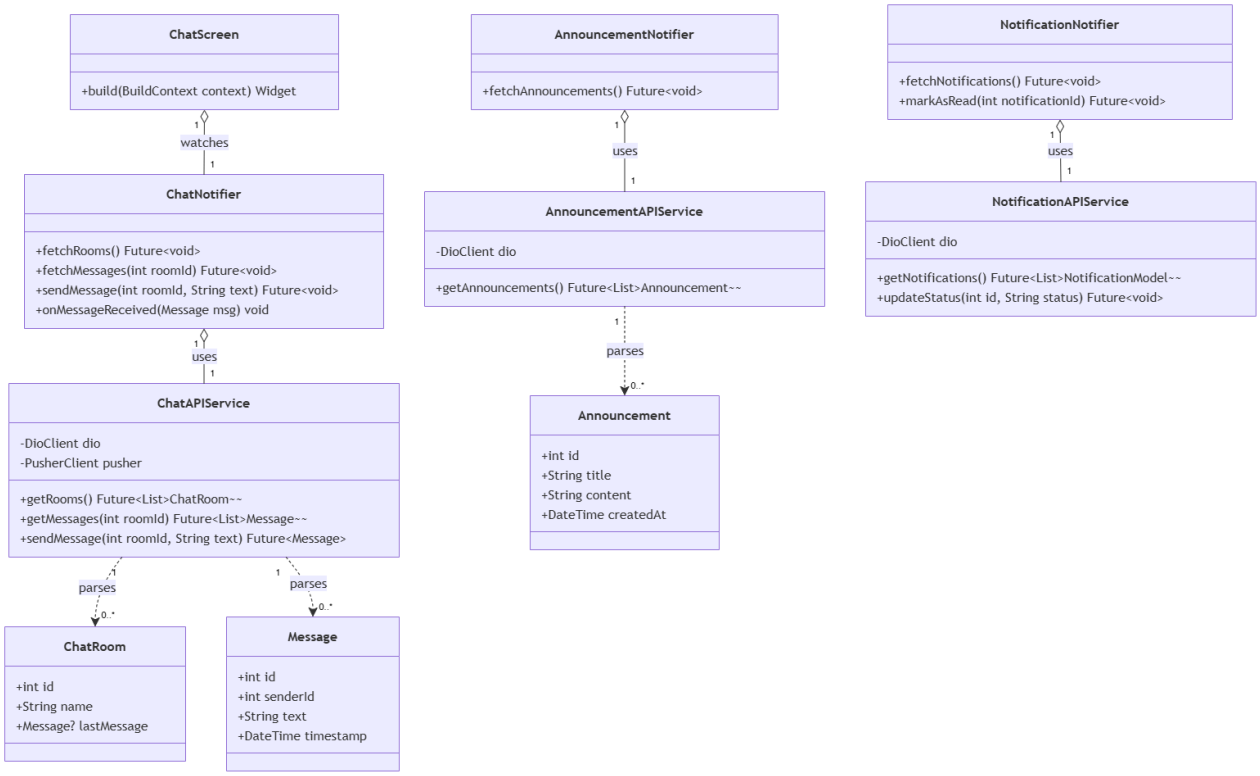
This module handles building announcements, notification lists, background push notifications via FCM, and real-time support chat via WebSocket (Pusher).

3.5.1 Class Diagram

3.5.1.1. NextJS Backend Class Diagram



3.5.1.2. Flutter Client Class Diagram



3.5.2 Class Specifications

3.5.2.1 NextJS Backend Class Specifications

A. NotificationController

- **Purpose**: Exposes API endpoints for alerts, notifications, and chat messages.
- **Methods**:
    - announceNotification(req, res): Handles POST /api/notifications/announce (UC24).
    - getNotifications(req, res): Handles GET /api/notifications (UC25).
    - getNotificationDetail(req, res): Handles GET /api/notifications/:id (UC26).
    - postMessage(req, res): Handles POST /api/chat/messages (UC28 chat).
    - getChatRoom(req, res): Handles GET /api/chat/room (UC28 room).

B. NotificationService

- **Purpose**: Coordinates FCM background delivery, real-time Pusher WebSockets, and image uploading.
- **Methods**:
    - publishAnnouncement(userId, title, body, banner): Verifies MANAGER role. Uploads banner to Cloudinary. Saves to DB and triggers asynchronous FCM broadcast payload.
    - getNotificationsList(userId): Enforces notification data isolation and pagination/sorting.

- markAsRead(notiId, userId): Processes status updates asynchronously in the backend and returns details. Enforces read-only immutability and supports deep link routing endpoints.
- sendInstantPush(userId, title, body, refId, type): Retrieves active user device tokens and invokes FCM API payload.
- saveAndSendChatMessage(roomId, senderId, text): Persists chat log and triggers Pusher client WebSocket broadcast. Enforces room isolation.

3.5.2.2 Flutter Client Class Specifications

A. ChatNotifier

- **Purpose**: A Riverpod notifier managing chat rooms, messages, real-time message receiving, and sending messages.
- **Attributes**:
  - chatAPIService: ChatAPIService
  - state: AsyncValue<List<Message>>
- **Methods**:
  - fetchRooms(): Future<void>
  - fetchMessages(roomId: int): Future<void>
  - sendMessage(roomId: int, text: String): Future<void>
  - onMessageReceived(msg: Message): void

B. AnnouncementNotifier

- **Purpose**: A Riverpod notifier managing building announcements and system broadcasts.
- **Attributes**:
  - announcementAPIService: AnnouncementAPIService
  - state: AsyncValue<List<Announcement>>
- **Methods**:
  - fetchAnnouncements(): Future<void>

C. NotificationNotifier

- **Purpose**: A Riverpod notifier managing user notifications, push alerts, and read status updates.
- **Attributes**:

    - notificationAPIService: NotificationAPIService
    - state: AsyncValue<List<NotificationModel>>
- **Methods**:
    - fetchNotifications(): Future<void>
    - markAsRead(notificationId: int): Future<void>

D. ChatAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Chat REST API endpoints and managing Pusher subscriptions.
- **Attributes**:
    - dio: DioClient
    - pusher: PusherClient
- **Methods**:
    - getRooms(): Future<List<ChatRoom>>
    - getMessages(roomId: int): Future<List<Message>>
    - sendMessage(roomId: int, text: String): Future<Message>

E. AnnouncementAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Announcement REST API endpoints.
- **Attributes**:
    - dio: DioClient
- **Methods**:
    - getAnnouncements(): Future<List<Announcement>>

F. NotificationAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Notification REST API endpoints.
- **Attributes**:
    - dio: DioClient

- **Methods**:
    - getNotifications(): Future<List<NotificationModel>>
    - updateStatus(id: int, status: String): Future<void>

G. ChatRoom (Model)

- **Purpose**: Client-side data model representing a chat room entity with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - name: String
    - lastMessage: Message?
- **Methods**:
    - fromJson(json: Map<String, dynamic>): ChatRoom
    - toJson(): Map<String, dynamic>

H. Message (Model)

- **Purpose**: Client-side data model representing a single chat message with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - senderId: int
    - text: String
    - timestamp: DateTime
- **Methods**:
    - fromJson(json: Map<String, dynamic>): Message
    - toJson(): Map<String, dynamic>

I. Announcement (Model)

- **Purpose**: Client-side data model representing a building announcement with local JSON serialization capabilities.

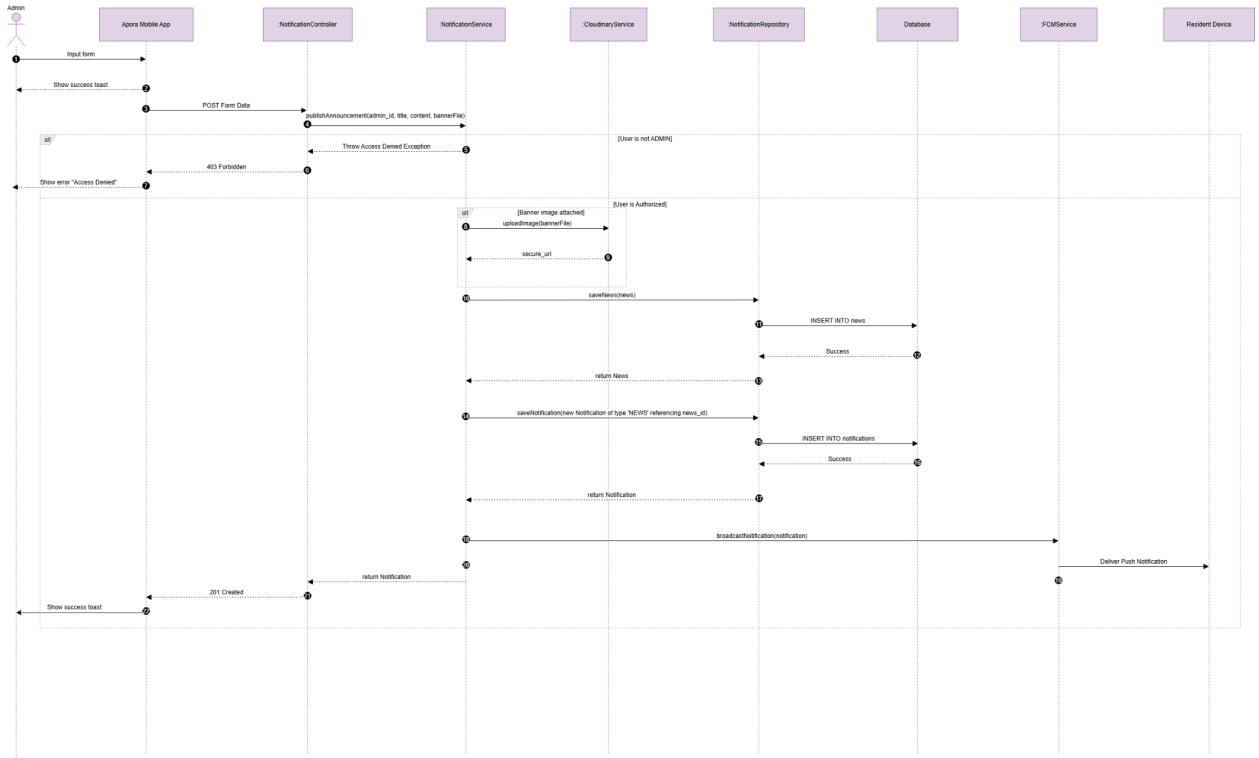
- **Attributes**:
    - id: int
    - title: String
    - content: String
    - createdAt: DateTime
- **Methods**:
    - fromJson(json: Map<String, dynamic>): Announcement
    - toJson(): Map<String, dynamic>

J. ChatScreen

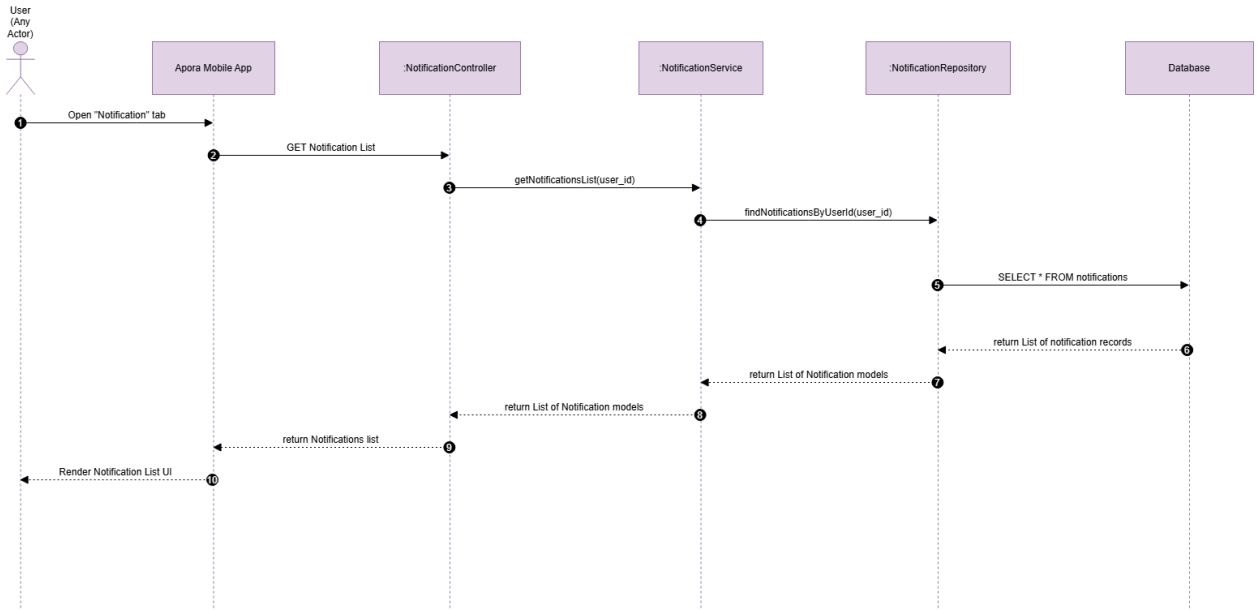
- **Purpose**: Flutter screen displaying real-time chat conversations, messages, and input boxes.
- **Methods**:
    - build(context: BuildContext): Widget

3.5.3 Sequence Diagrams

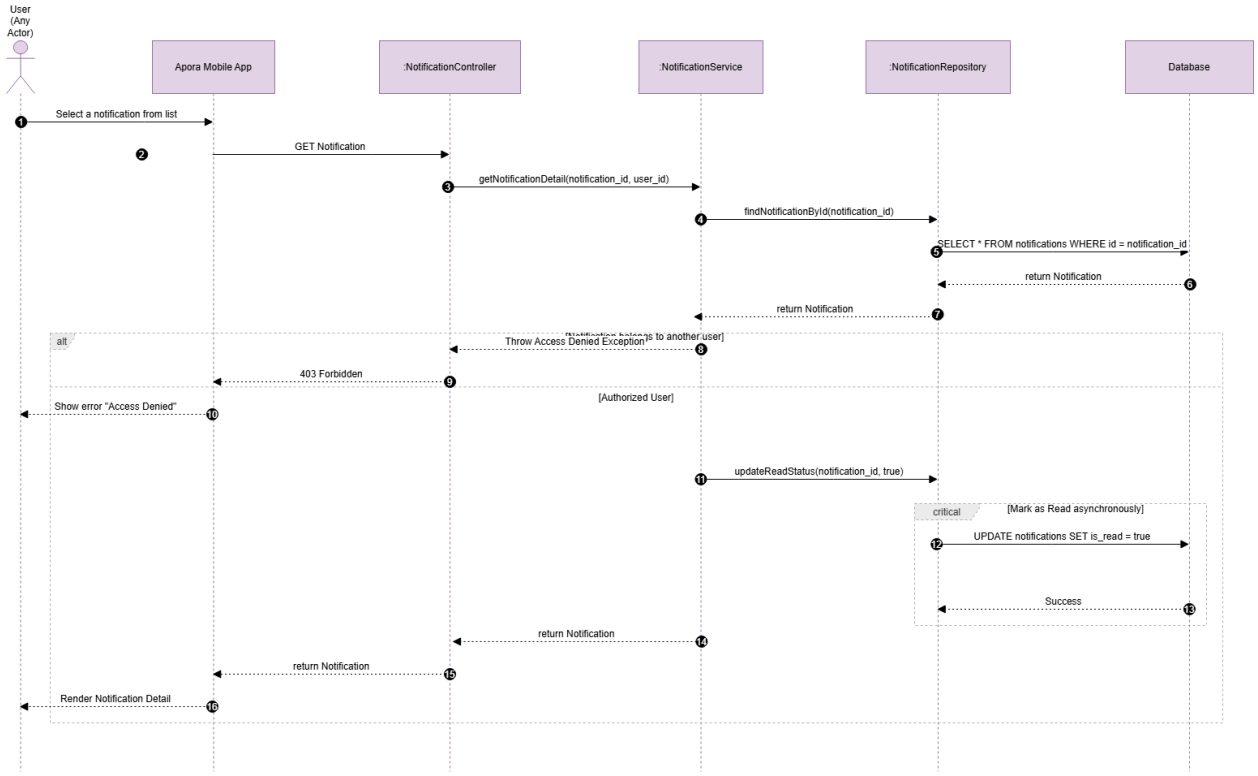
UC24: Announce Notification



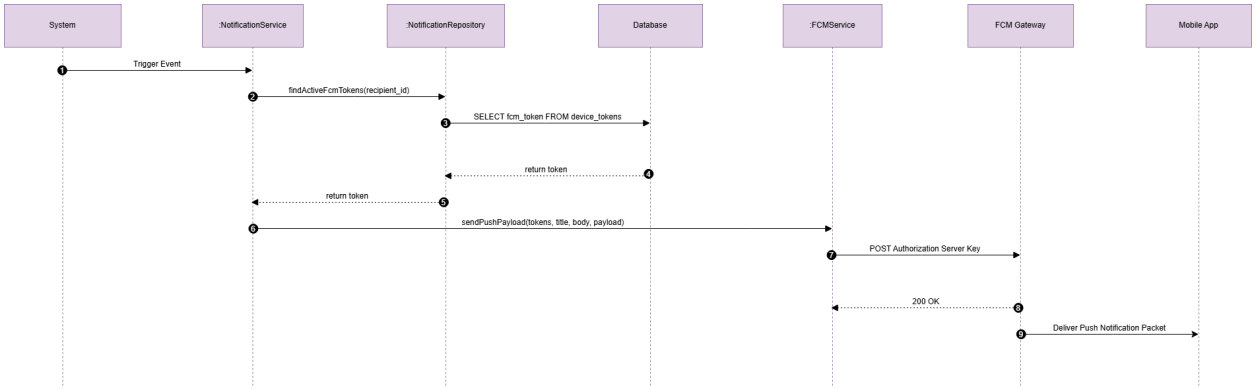
UC25: View Notification List



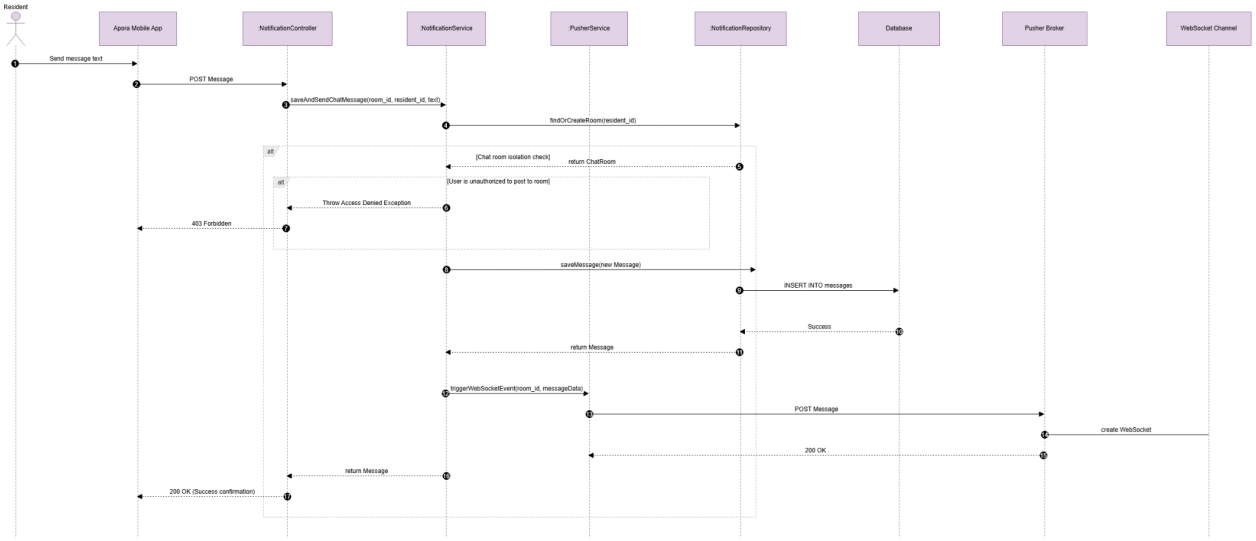
UC26: View Notification Detail



UC27: Receive Notification



UC28: Use Live Chat



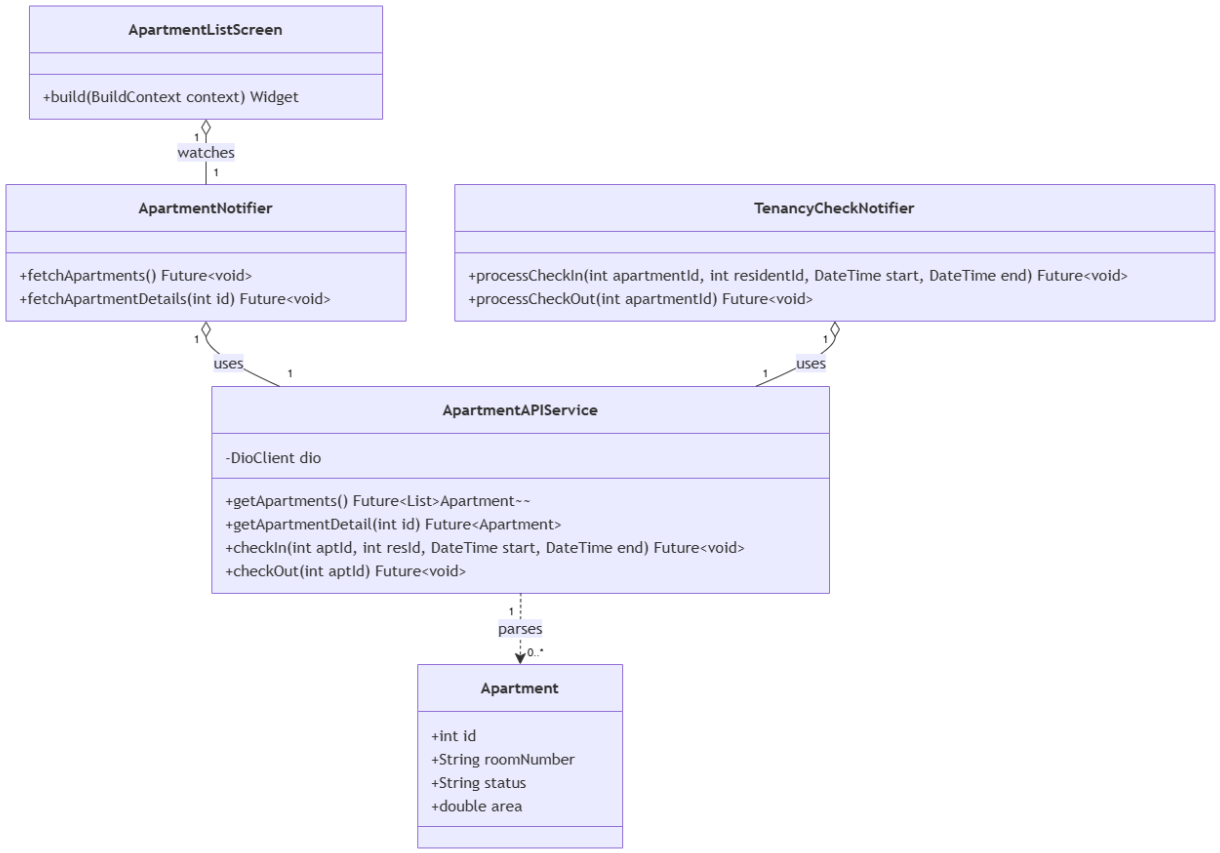
3.6 Module 6: Apartment Management (UC29 - UC34)

3.6.1 Class Diagram

This module handles building apartment directories, rental contract processing, tenant check-in operations, and soft-delete checkout procedures.

3.6.1.1. NextJS Backend Class Diagram

3.6.1.2. Flutter Client Class Diagram



3.6.2 Class Specifications

3.6.2.1 NextJS Backend Class Specifications

A. ApartmentController

- **Purpose:** Exposes HTTP endpoints for apartment directories, rental contract processing, tenant check-in operations, and checkout procedures.
- **Attributes:**
    - -apartmentService: ApartmentService
- **Methods:**
    - +getApartments(req, res): void
    - +getApartmentDetail(req, res): void
    - +createApartment(req, res): void
    - +updateApartment(req, res): void
    - +checkinTenant(req, res): void
    - +checkoutTenant(req, res): void

B. ApartmentService

- **Purpose:** Manages apartment lifecycles, role access boundaries, and transactional checks for check-in/checkout operations.
- **Attributes:**
    - -aptRepo: ApartmentRepository
    - -userRepo: UserRepository
    - -contractRepo: ContractRepository
    - -cloudinaryService: CloudinaryService
    - -notiService: NotificationService
- **Methods:**
    - +getApartmentsList(userId: int, role: String, filter: String): List<Apartment>
    - +getApartmentDetailById(id: int, userId: int, role: String): Apartment
    - +createNewApartment(data: ApartmentDTO): Apartment
    - +modifyApartment(id: int, data: ApartmentDTO): Apartment
    - +processCheckin(aptId: int, name: String, phone: String, start: Date, end: Date, rent: Decimal, deposit: Decimal): Contract
    - +processCheckout(aptId: int, managerId: int): void

C. ApartmentRepository

- **Purpose:** Handles persistent data access operations for apartment records.
- **Attributes:**
    - -db: DatabaseConnection
- **Methods:**
    - +findAll(filter: String): List<Apartment>
    - +findById(id: int): Apartment
    - +save(apt: Apartment): Apartment

    - +update(id: int, apt: Apartment): Apartment
    - +updateOccupancyStatus(id: int, status: String, ownerId: int): void
    - +getApartmentsWithAggregatedStats(filter): List<Apartment>

D. UserRepository

- **Purpose:** Handles persistent data access operations related to user accounts (tenants/residents).
- **Methods:**
    - +saveUser(user: User): User
    - +updateUserStatus(id: int, status: String): void
    - +findUserByPhone(phone: String): User

E. ContractRepository

- **Purpose:** Handles persistent data access operations for leasing contracts.
- **Methods:**
    - +saveContract(con: Contract): Contract
    - +terminateContract(id: int): void
    - +findActiveContractByApartmentId(aptId: int): Contract

F. CloudinaryService

- **Purpose:** Handles deletion of sensitive images (like CCCD) from external storage during checkout procedures.
- **Methods:**
    - +deleteImagesBatch(urls: List<String>): void

G. NotificationService

- **Purpose:** Dispatches push notifications to users.
- **Methods:**
    - +sendPushNotification(userId: int, title: String, body: String): void

H. Apartment (Entity)

- **Purpose:** Data model representing an apartment unit.
- **Attributes:**
    - +id: int
    - +unit_number: String
    - +floor: String
    - +owner_id: int
    - +status: String

I. Contract (Entity)

- **Purpose:** Data model representing a leasing agreement.

- **Attributes:**
    - +id: int
    - +apartment_id: int
    - +resident_id: int
    - +start_date: Date
    - +end_date: Date
    - +base_rent_snapshot: Decimal
    - +status: String

3.6.2.2 Flutter Client Class Specifications

A. ApartmentNotifier

- **Purpose**: A Riverpod notifier managing lists of apartments, filtering by status, and displaying details for property managers.
- **Attributes**:
    - apartmentAPIService: ApartmentAPIService
    - state: AsyncValue<List<Apartment>>
- **Methods**:
    - fetchApartments(): Future<void>
    - fetchApartmentDetails(id: int): Future<void>

B. TenancyCheckNotifier

- **Purpose**: A Riverpod notifier managing the check-in and check-out workflows for residents.
- **Attributes**:
    - apartmentAPIService: ApartmentAPIService
    - state: AsyncValue<void>
- **Methods**:
    - processCheckIn(apartmentId: int, residentId: int, start: DateTime, end: DateTime): Future<void>
    - processCheckOut(apartmentId: int): Future<void>

C. ApartmentAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Apartment REST API endpoints.

- **Attributes**:
    - dio: DioClient
- **Methods**:
    - getApartments(): Future<List<Apartment>>
    - getApartmentDetail(id: int): Future<Apartment>
    - checkIn(aptId: int, resId: int, start: DateTime, end: DateTime): Future<void>
    - checkOut(aptId: int): Future<void>

D. Apartment (Model)

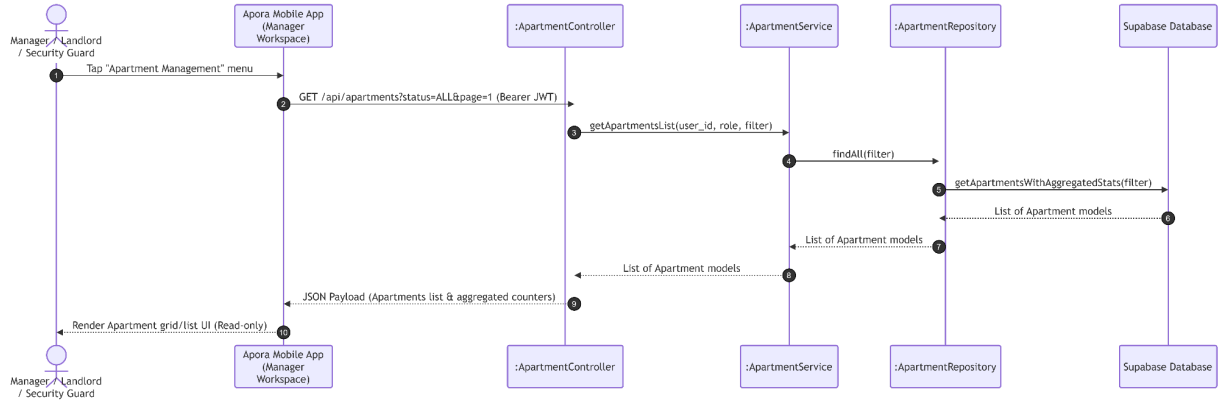
- **Purpose**: Client-side data model representing an apartment room entity with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - roomNumber: String
    - status: String
    - area: double
- **Methods**:
    - fromJson(json: Map<String, dynamic>): Apartment
    - toJson(): Map<String, dynamic>

E. ApartmentListScreen

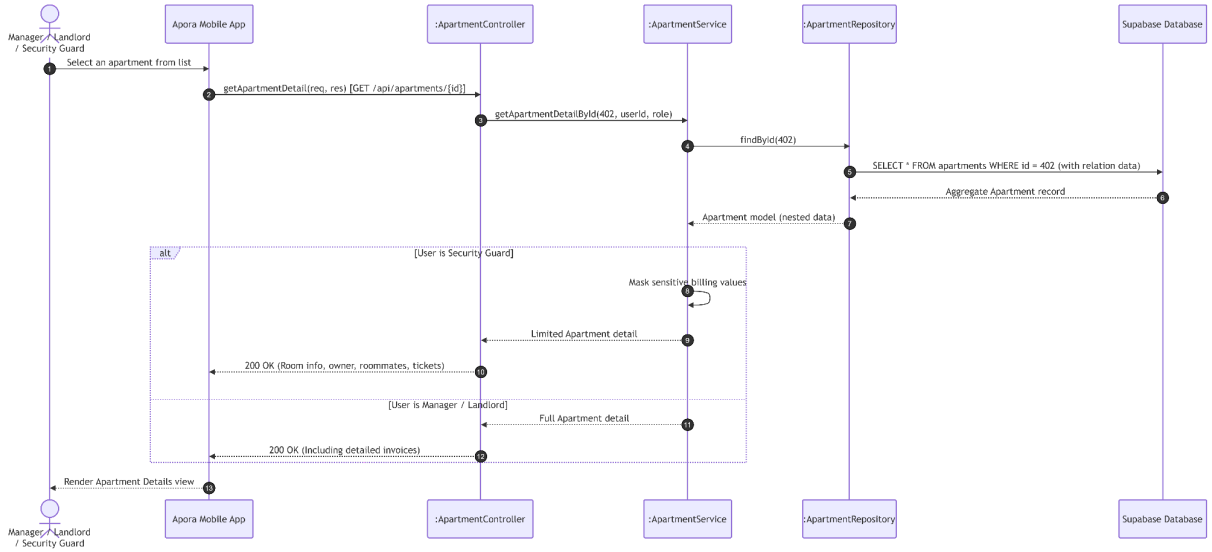
- **Purpose**: Flutter screen displaying building apartments, status badges, and managing room details.
- **Methods**:
    - build(context: BuildContext): Widget

3.6.3 Sequence Diagrams

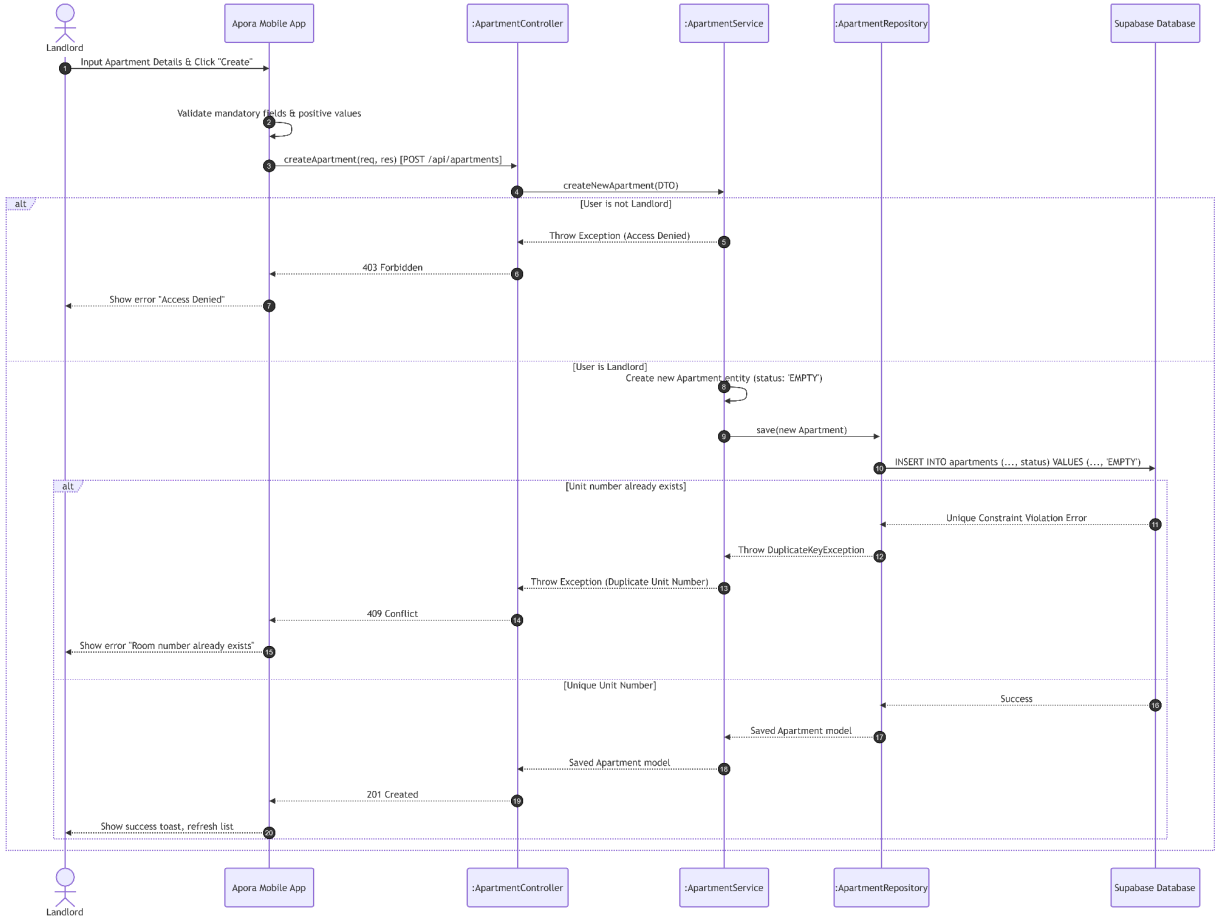
UC29: View Apartment List



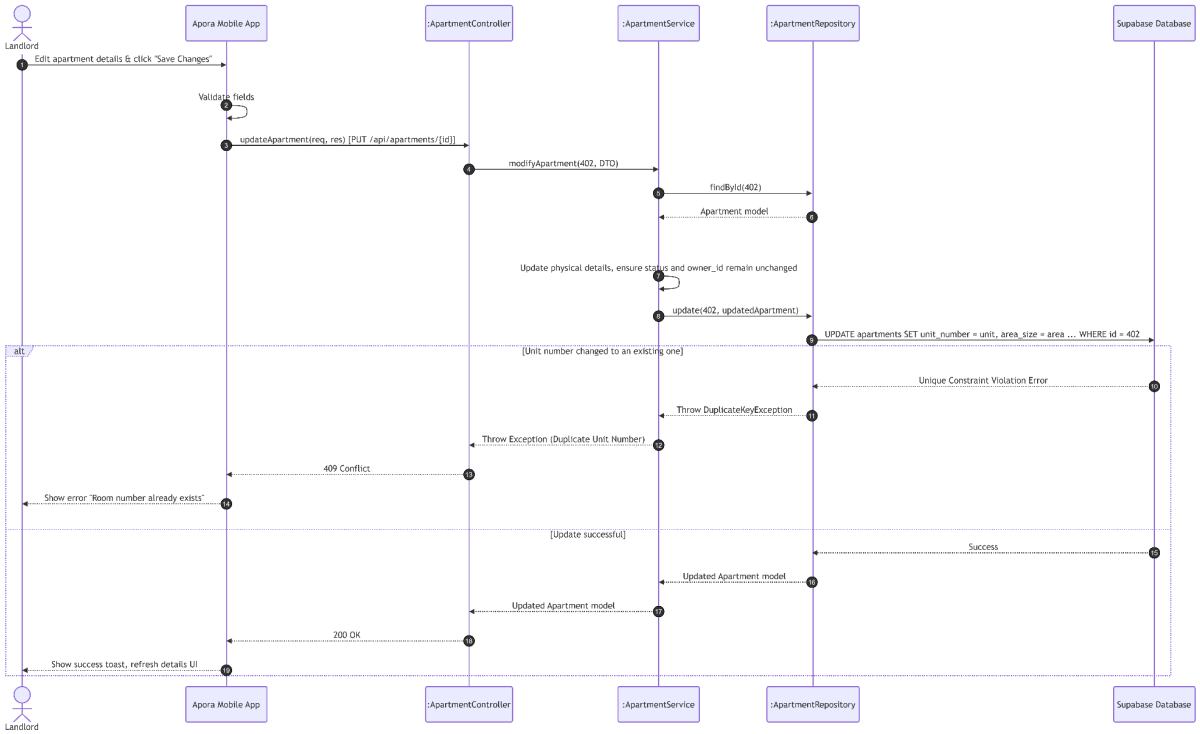
UC30: View Apartment Detail



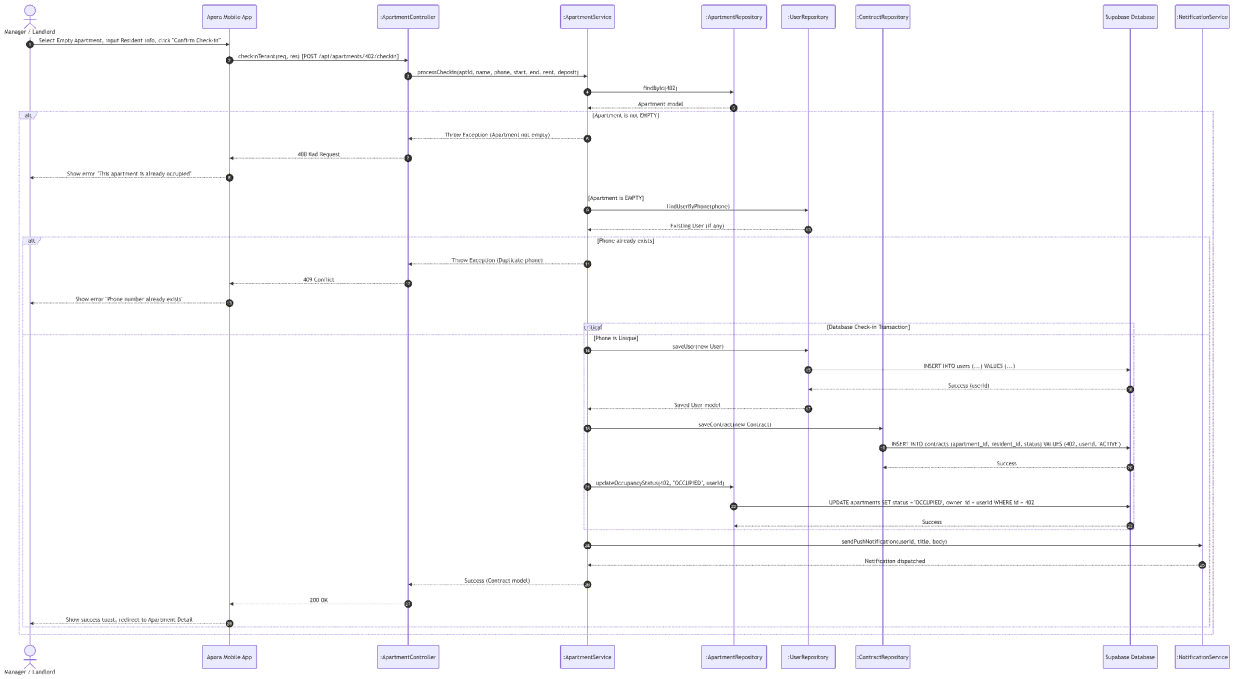
UC31: Create New Apartment



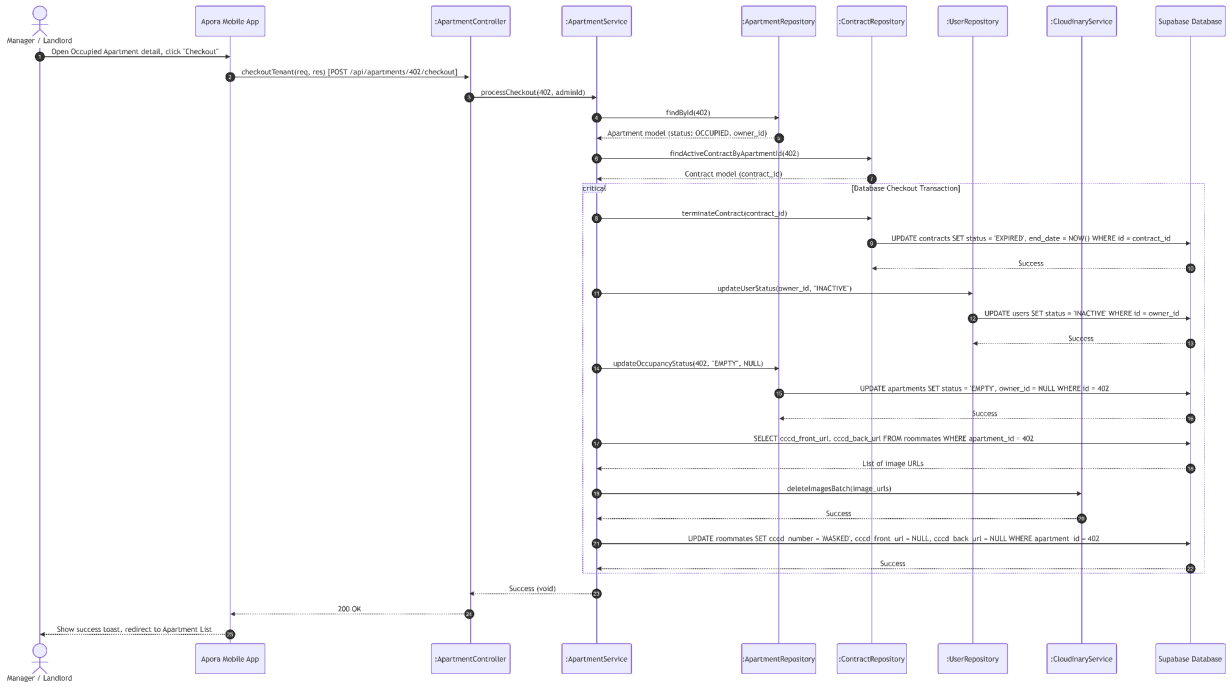
UC32: Update Apartment Information



UC33: Check in Apartment



UC34: Checkout Apartment

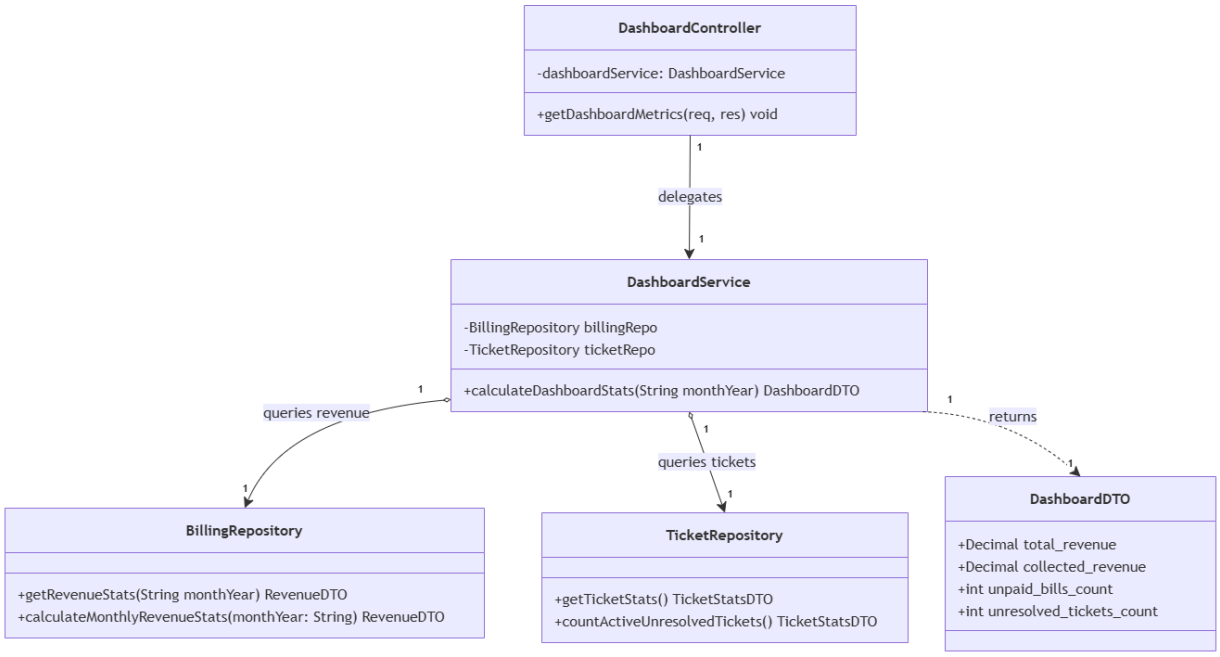


3.7 Module 7: Statistics & Dashboard (UC35)

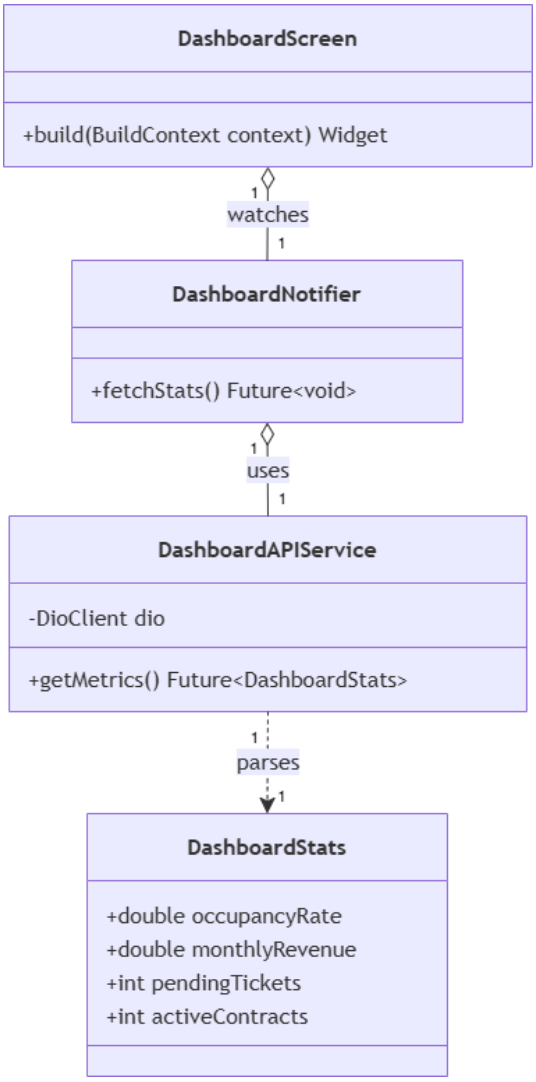
This module handles aggregated dashboard query calculations for financial and ticketing metrics.

3.7.1 Class Diagram

3.7.1.1. NextJS Backend Class Diagram



3.7.1.2. Flutter Client Class Diagram



3.7.2 Class Specifications

3.7.2.1 NextJS Backend Class Specifications

A. DashboardController

- **Purpose:** Exposes HTTP endpoints for retrieving aggregated dashboard metrics and analytical statistics.
- **Attributes:**
    - -dashboardService: DashboardService
- **Methods:**
    - +getDashboardMetrics(req, res): void

B. DashboardService

- **Purpose:** Orchestrates business logic for dashboard statistics calculation by querying financial and operational data from respective repositories.
- **Attributes:**
    - -billingRepo: BillingRepository
    - -ticketRepo: TicketRepository
- **Methods:**
    - +calculateDashboardStats(monthYear: String): DashboardDTO

C. BillingRepository

- **Purpose:** Handles persistent data access layer operations to fetch revenue and invoice statistics from the database.
- **Methods:**
    - +getRevenueStats(monthYear: String): RevenueDTO
    - +calculateMonthlyRevenueStats(monthYear: String): RevenueDTO

D. TicketRepository

- **Purpose:** Handles persistent data access layer operations to fetch aggregated ticket data and status counts from the database.
- **Methods:**
    - +getTicketStats(): TicketStatsDTO
    - +countActiveUnresolvedTickets(): TicketStatsDTO

E. DashboardDTO

- **Purpose:** Data Transfer Object holding the consolidated financial and operational metrics for dashboard visualization.
- **Attributes:**
    - +total_revenue: Decimal
    - +collected_revenue: Decimal
    - +unpaid_bills_count: int
    - +unresolved_tickets_count: int

3.7.2.2 Flutter Client Class Specifications

A. DashboardNotifier

- **Purpose**: A Riverpod notifier managing active dashboard indicators, occupancy rates, monthly revenues, and pending actions for landlords.
- **Attributes**:
    - dashboardAPIService: DashboardAPIService
    - state: AsyncValue<DashboardStats>
- **Methods**:

- fetchStats(): Future<void>

B. DashboardAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Dashboard metrics REST API endpoints.
- **Attributes**:
  - dio: DioClient
- **Methods**:
  - getMetrics(): Future<DashboardStats>

C. DashboardStats (Model)

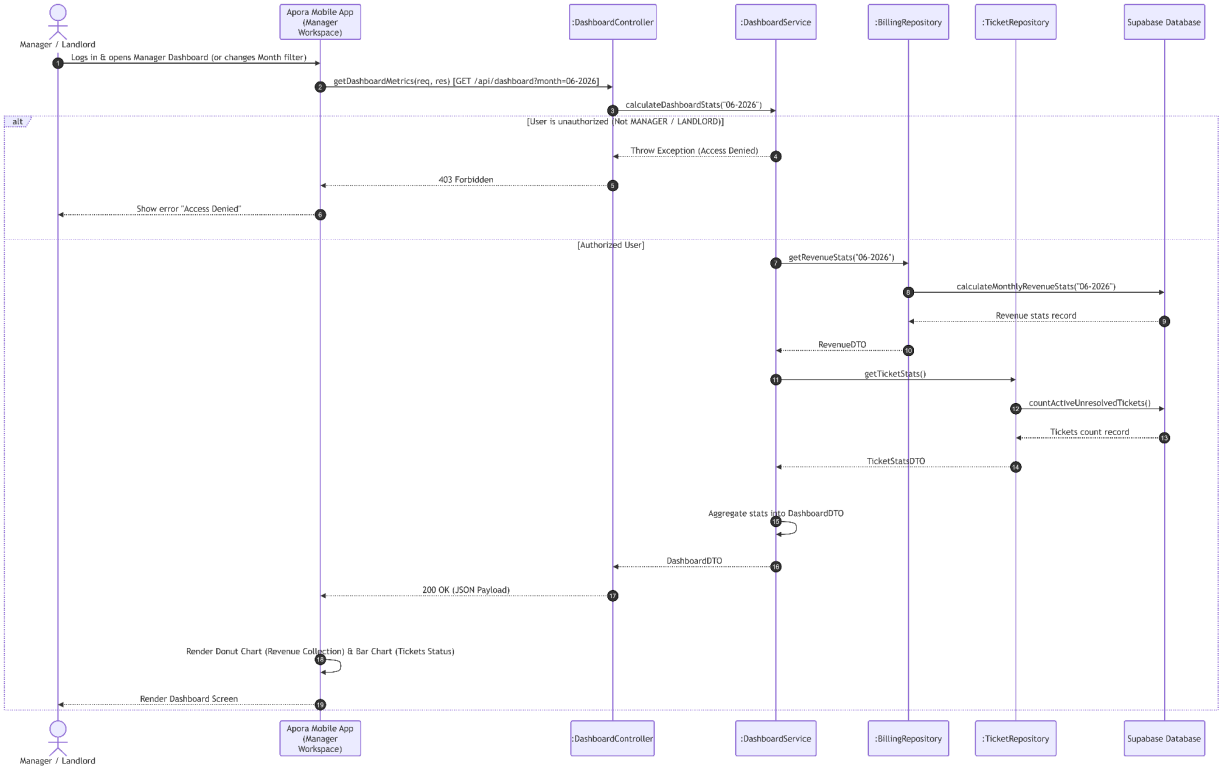
- **Purpose**: Client-side data model representing collected landlord KPIs with local JSON serialization capabilities.
- **Attributes**:
  - occupancyRate: double
  - monthlyRevenue: double
  - pendingTickets: int
  - activeContracts: int
- **Methods**:
  - fromJson(json: Map<String, dynamic>): DashboardStats
  - toJson(): Map<String, dynamic>

D. LandlordDashboardScreen

- **Purpose**: Flutter screen displaying business analytics charts, collected revenues, and operational status summaries for property landlords.
- **Methods**:
  - build(context: BuildContext): Widget

3.7.3 Sequence Diagrams

UC35: View Dashboard

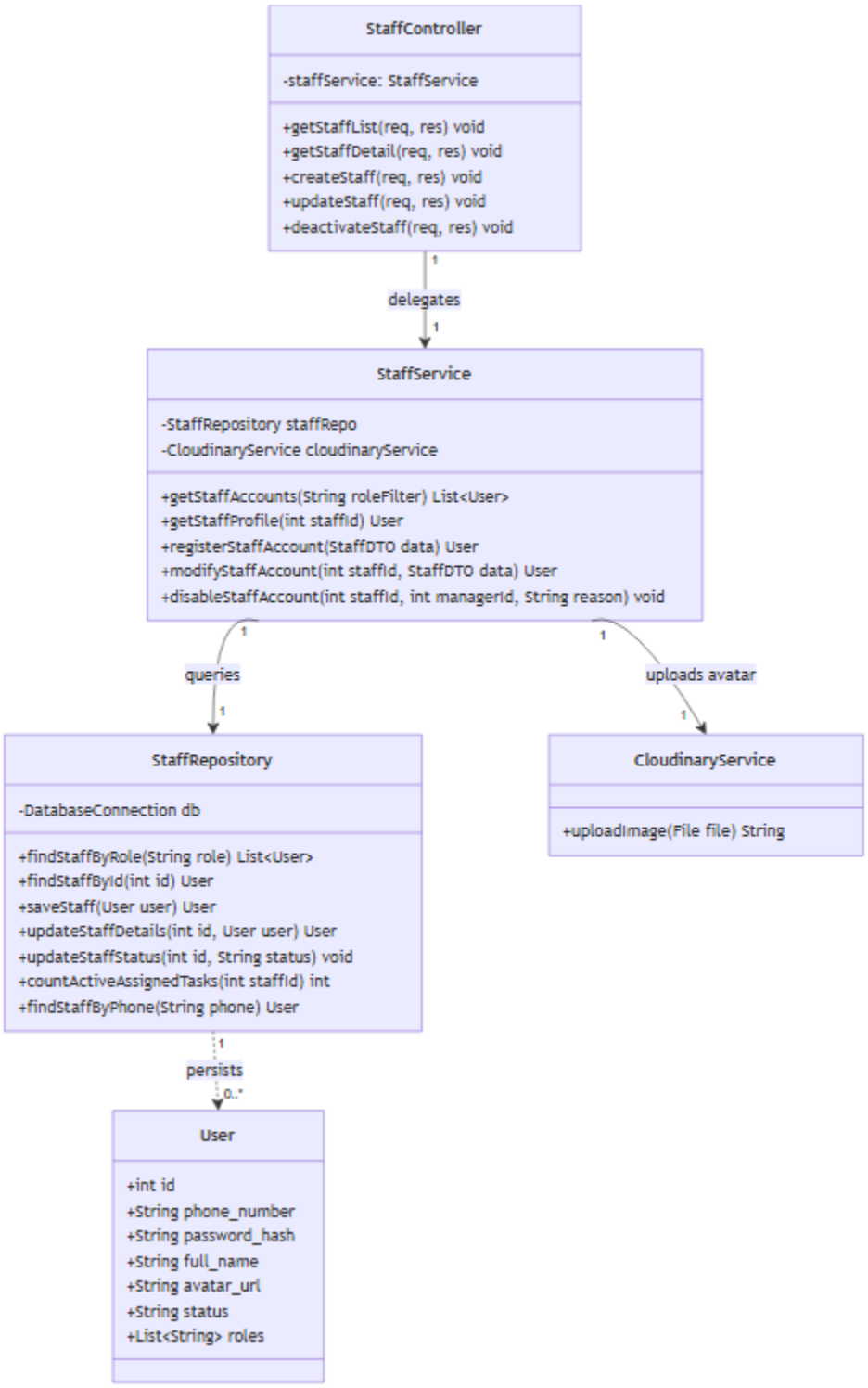


3.8 Module 8: Operational Staff Management (UC36 - UC40)

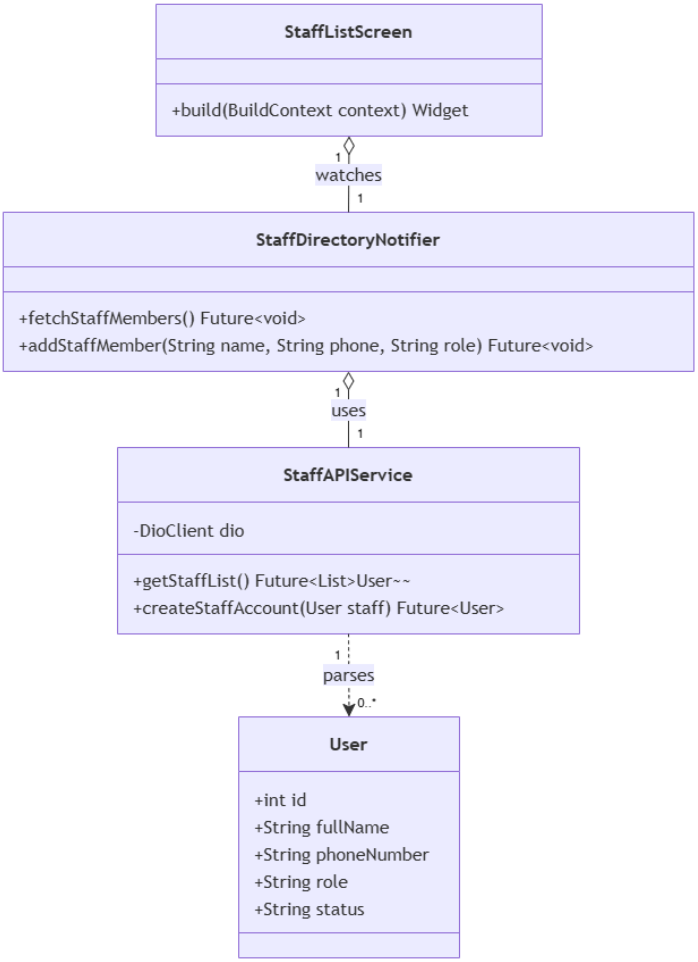
This module handles operational staff directories (Security Guard, Janitor, Technician), user creation, details modification, and account deactivation.

3.8.1 Class Diagram

3.8.1.1. NextJS Backend Class Diagram



3.8.1.2. Flutter Client Class Diagram



3.8.2 Class Specifications

3.8.2.1 NextJS Backend Class Specifications

A. StaffController

- **Purpose:** Exposes HTTP endpoints for handling staff-related operations such as retrieving lists, getting details, creating, updating, and deactivating staff accounts.
- **Attributes:**
    - staffService: StaffService
- **Methods:**
    - getStaffList(req, res): void
    - getStaffDetail(req, res): void
    - createStaff(req, res): void
    - updateStaff(req, res): void
    - deactivateStaff(req, res): void

B. StaffService

- **Purpose:** Encapsulates the core business logic for staff operations, acting as a bridge between the controller layer and data repositories or external services like image hosting.
- **Attributes:**
    - staffRepo: StaffRepository
    - cloudinaryService: CloudinaryService
- **Methods:**
    - getStaffAccounts(roleFilter: String): List<User>
    - getStaffProfile(staffId: int): User
    - registerStaffAccount(data: StaffDTO): User
    - modifyStaffAccount(staffId: int, data: StaffDTO): User
    - disableStaffAccount(staffId: int, managerId: int, reason: String): void

C. StaffRepository

- **Purpose:** Handles direct database interactions and CRUD operations for persisting and retrieving staff (User) records.
- **Attributes:**
    - db: DatabaseConnection
- **Methods:**
    - findStaffByRole(role: String): List<User>
    - findStaffById(id: int): User
    - saveStaff(user: User): User
    - updateStaffDetails(id: int, user: User): User
    - updateStaffStatus(id: int, status: String): void
    - countActiveAssignedTasks(staffId: int): int
    - findStaffByPhone(phone: String): User
    - assigned tasks to prevent task abandonment. If valid, sets status to INACTIVE.

3.8.2.2 Flutter Client Class Specifications

A. StaffDirectoryNotifier

- **Purpose**: A Riverpod notifier managing the operational staff directory list, account registrations, and deactivations.
- **Attributes**:
    - staffAPIService: StaffAPIService
    - state: AsyncValue<List<User>>
- **Methods**:
    - fetchStaffMembers(): Future<void>
    - addStaffMember(name: String, phone: String, role: String): Future<void>

B. StaffAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Staff directory REST API endpoints.
- **Attributes**:
    - dio: DioClient
- **Methods**:
    - getStaffList(): Future<List<User>>
    - createStaffAccount(staff: User): Future<User>

C. User (Model)

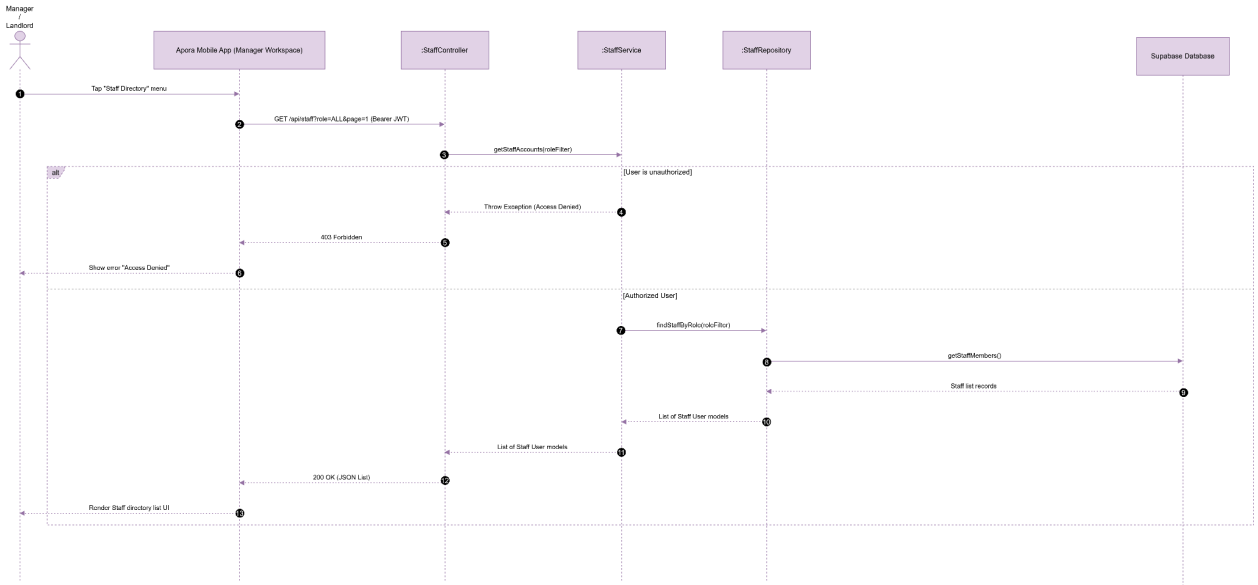
- **Purpose**: Client-side data model representing a staff user entity with local JSON serialization capabilities.
- **Attributes**:
    - id: int
    - fullName: String
    - phoneNumber: String
    - role: String
    - status: String
- **Methods**:
    - fromJson(json: Map<String, dynamic>): User
    - toJson(): Map<String, dynamic>

D. StaffListScreen

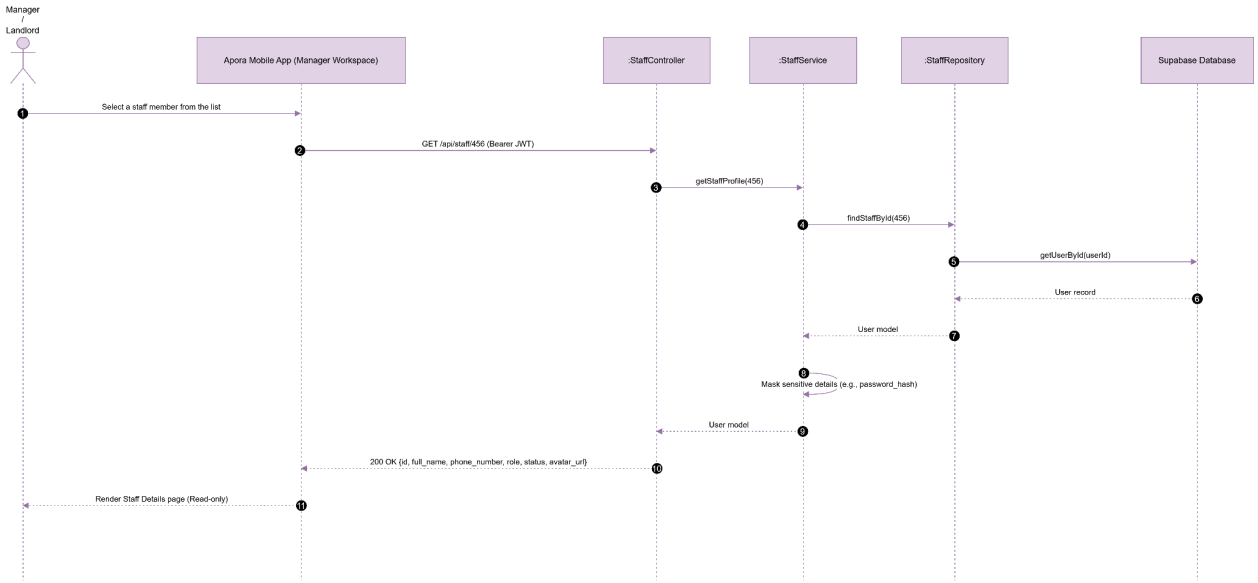
- **Purpose**: Flutter screen displaying operational staff accounts (Guard, Janitor, Technician) and managing creation forms.
- **Methods**:
    - build(context: BuildContext): Widget

3.8.3 Sequence Diagrams

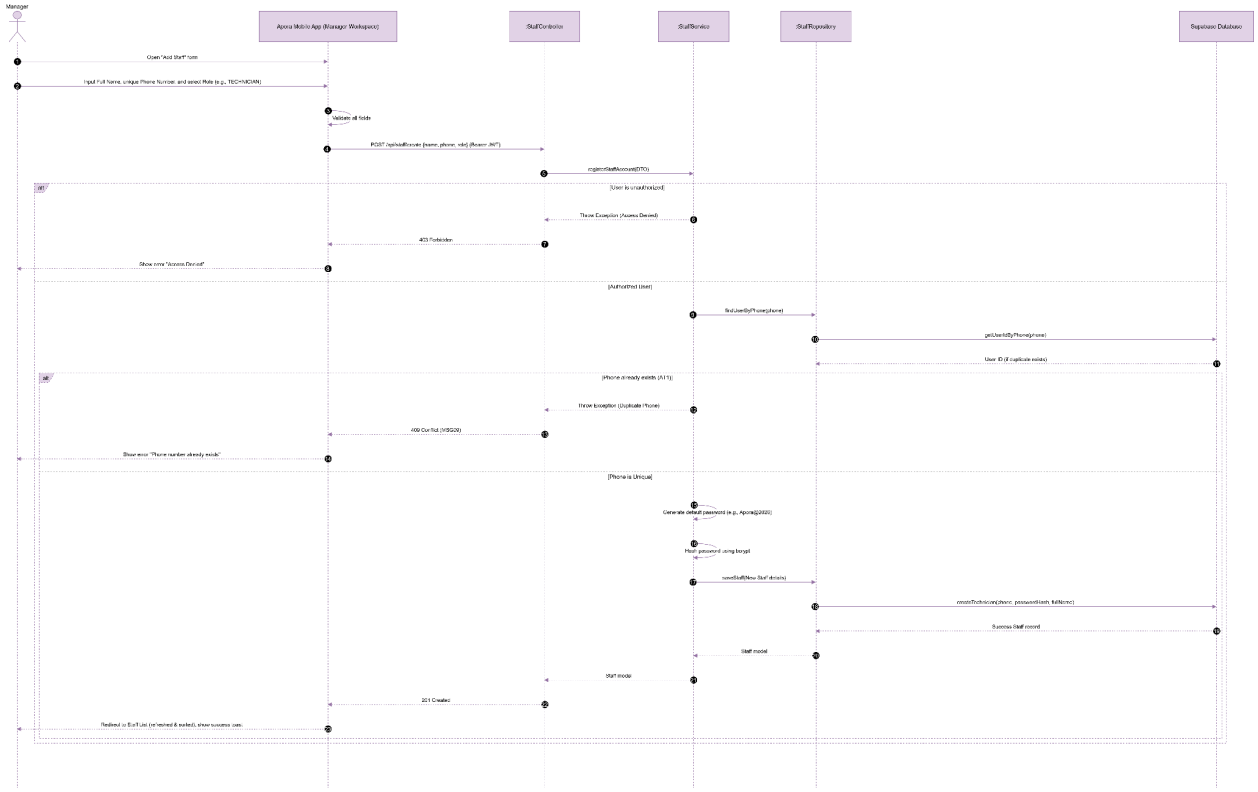
UC36 : View Staff List



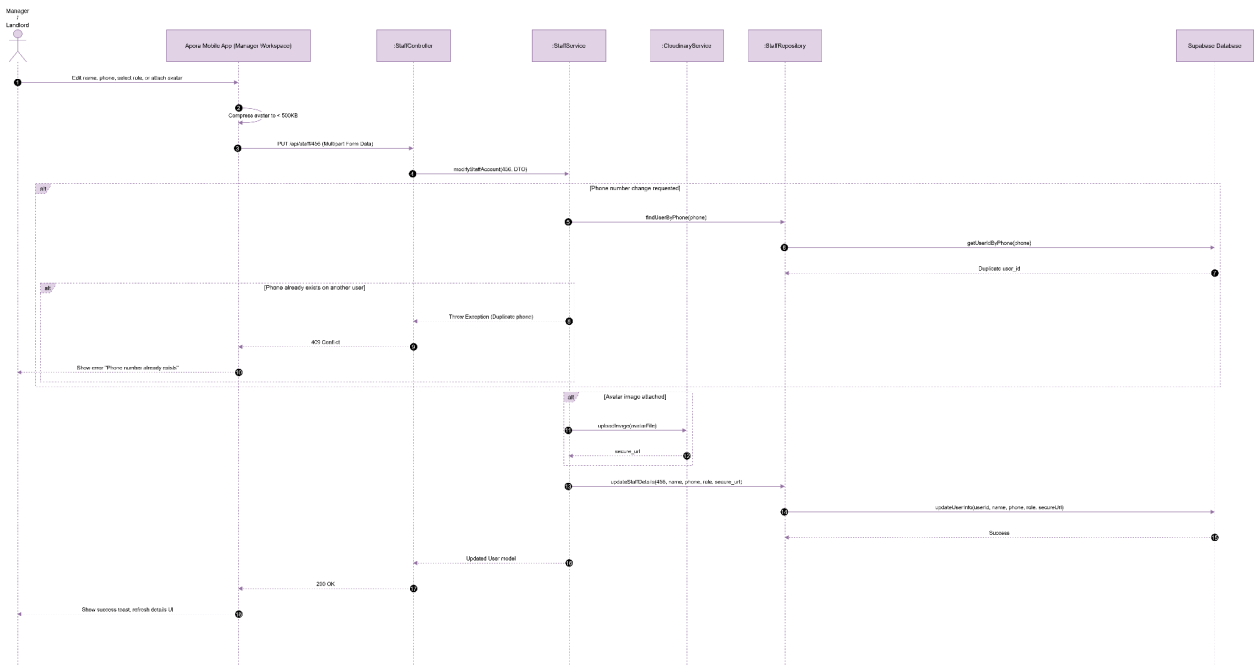
UC37 : View Staff Detail



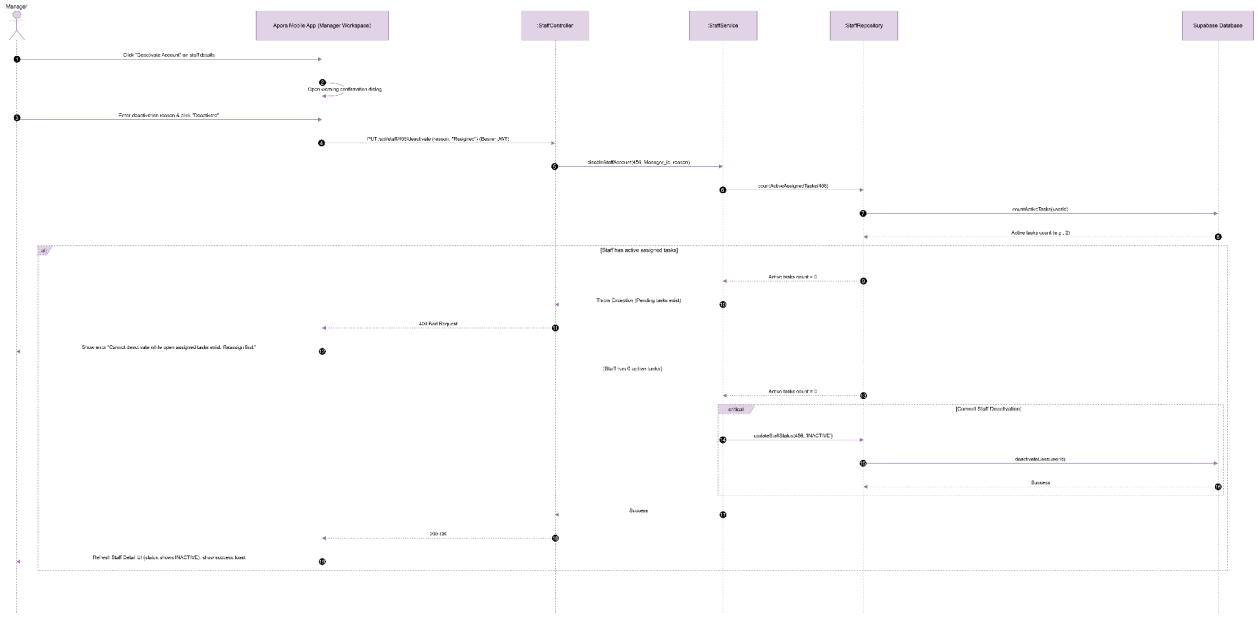
UC38 : Create Staff Account



UC39 : Update Staff Account



UC40 : Deactivate Staff Account

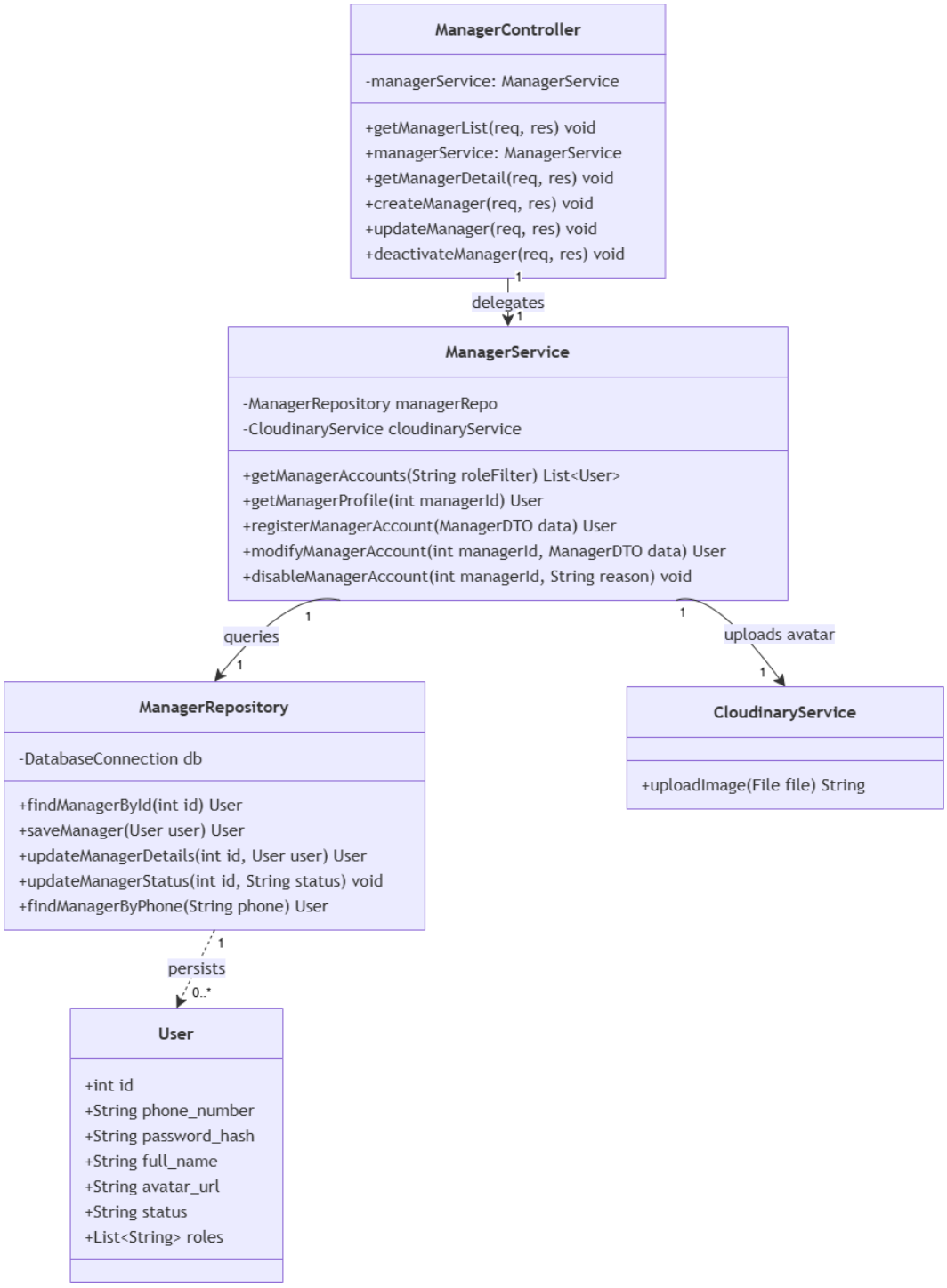


3.9 Module 9: Manager Management (UC41 - UC45)

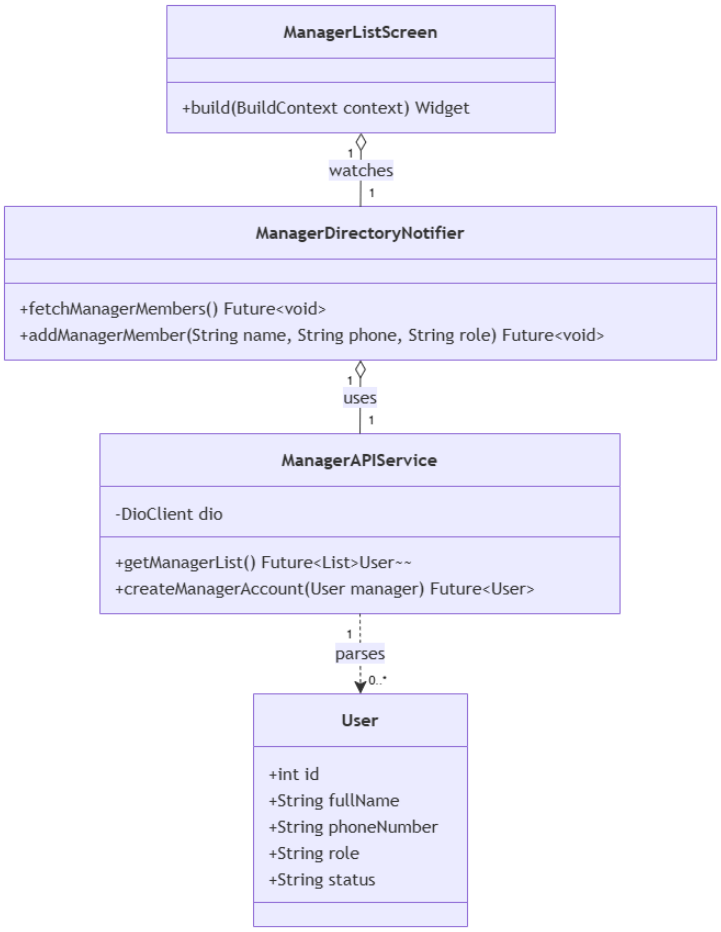
This module handles Manager directories and permissions. As Manager management is structurally identical to Staff management (CRUD operations on the same database user tables), it inherits the class design, methods, repository, and database structures from Module 8: Operational Staff Management, mapping variables and actions to the MANAGER role.

3.9.1 Class Diagram

3.9.1.1. NextJS Backend Class Diagram



3.9.1.2. Flutter Client Class Diagram



3.9.2 Class Specifications

3.9.2.1 NextJS Backend Class Specifications

A. ManagerController

- **Purpose:** Exposes HTTP endpoints for handling manager-related operations such as retrieving lists, getting details, creating, updating, and deactivating manager accounts.
- **Attributes:**
    - managerService: ManagerService
- **Methods:**
    - getManagerList(req, res): void
    - getManagerDetail(req, res): void
    - createManager(req, res): void
    - updateManager(req, res): void
    - deactivateManager(req, res): void

B. ManagerService

- **Purpose:** Encapsulates the core business logic for manager operations, acting as a bridge between the controller layer and data repositories or external services like image hosting.
- **Attributes:**
    - managerRepo: ManagerRepository
    - cloudinaryService: CloudinaryService
- **Methods:**
    - getManagerAccounts(roleFilter: String): List<User>
    - getManagerProfile(managerId: int): User
    - registerManagerAccount(data: ManagerDTO): User
    - modifyManagerAccount(managerId: int, data: ManagerDTO): User
    - disableManagerAccount(managerId: int, reason: String): void

C. ManagerRepository

- **Purpose:** Handles direct database interactions and CRUD operations for persisting and retrieving manager (User) records.
- **Attributes:**
    - db: DatabaseConnection
- **Methods:**
    - findManagerById(id: int): User
    - saveManager(user: User): User
    - updateManagerDetails(id: int, user: User): User
    - updateManagerStatus(id: int, status: String): void
    - findManagerByPhone(phone: String): User

3.9.2.2 Flutter Client Class Specifications

A. ManagerDirectoryNotifier

- **Purpose**: A Riverpod notifier managing property manager directories, adding new manager accounts, and handling deactivations.
- **Attributes**:
    - managerAPIService: ManagerAPIService
    - state: AsyncValue<List<User>>
- **Methods**:
    - fetchManagerMembers(): Future<void>
    - addManagerMember(name: String, phone: String, role: String): Future<void>

B. ManagerAPIService

- **Purpose**: Network service wrapper connecting the mobile client to Next.js Manager REST API endpoints.
- **Attributes**:

- dio: DioClient

- **Methods**:
  - getManagerList(): Future<List<User>>
  - createManagerAccount(manager: User): Future<User>

C. User (Model)

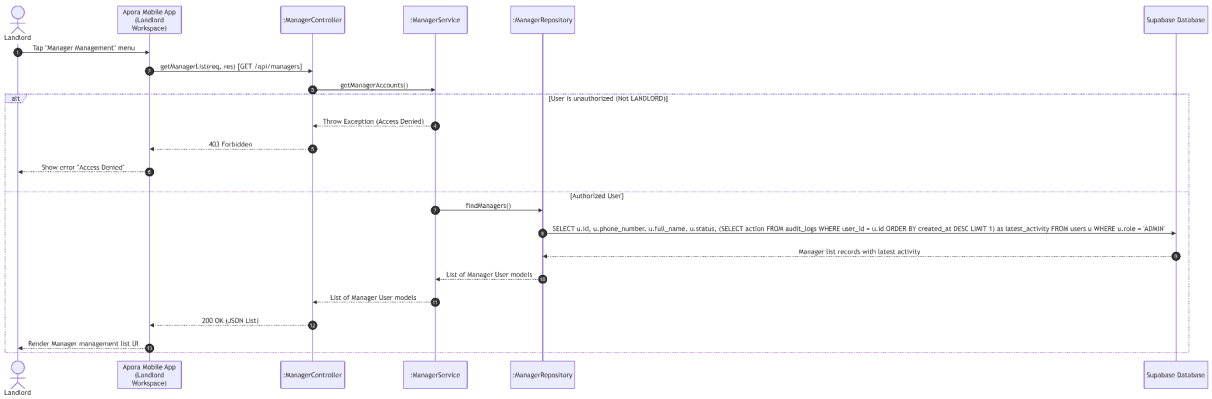
- **Purpose**: Client-side data model representing a manager user entity with local JSON serialization capabilities.
- **Attributes**:
  - id: int
  - fullName: String
  - phoneNumber: String
  - role: String
  - status: String
- **Methods**:
  - fromJson(json: Map<String, dynamic>): User
  - toJson(): Map<String, dynamic>

D. ManagerListScreen

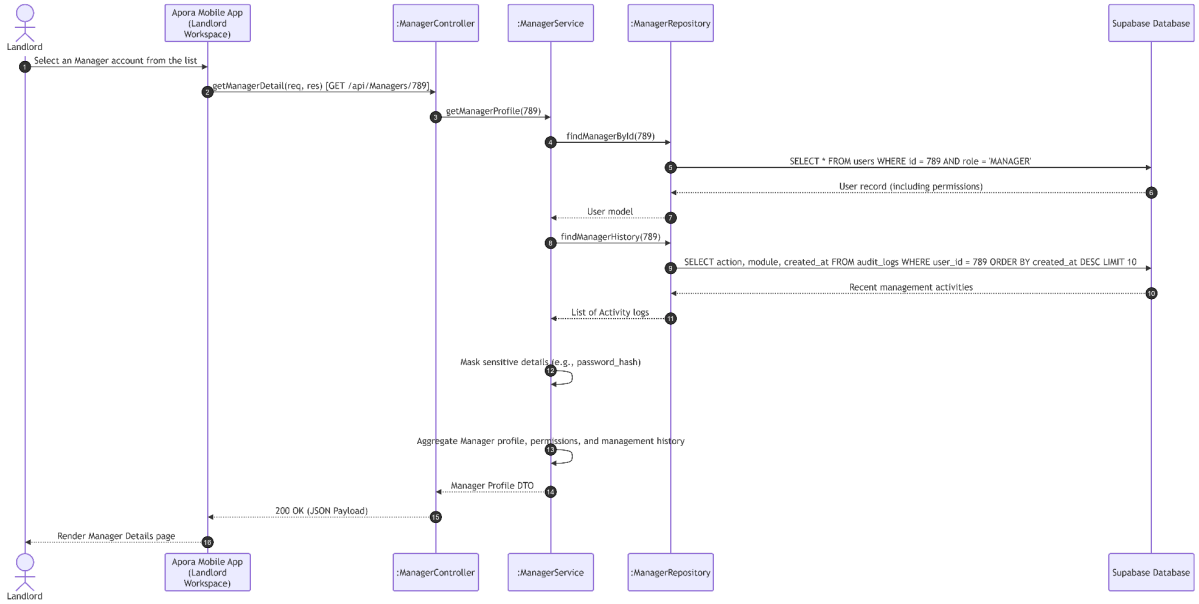
- **Purpose**: Flutter screen displaying building manager accounts and managing account registrations.
- **Methods**:
  - build(context: BuildContext): Widget

3.9.3 Sequence Diagrams

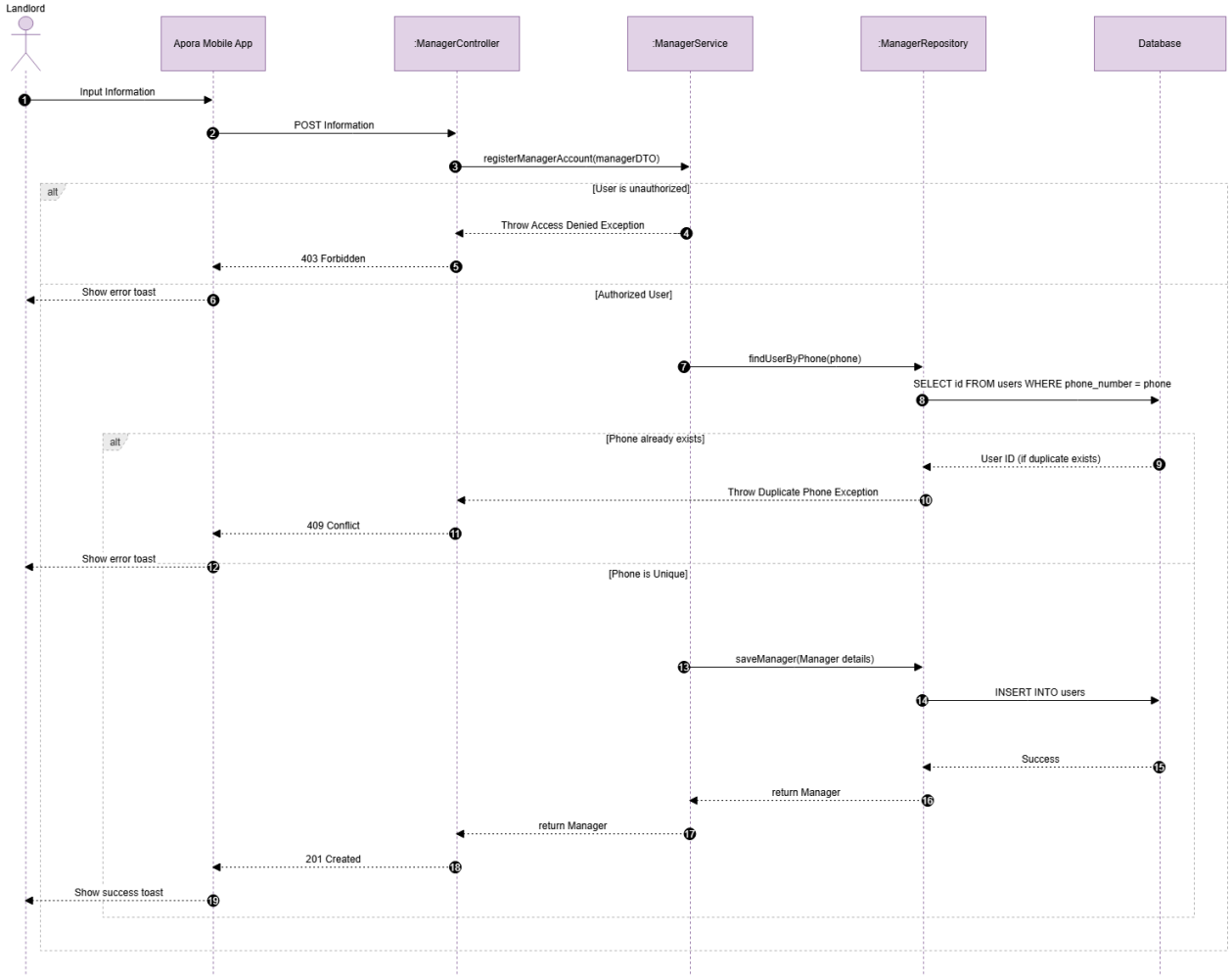
UC41: View Manager List



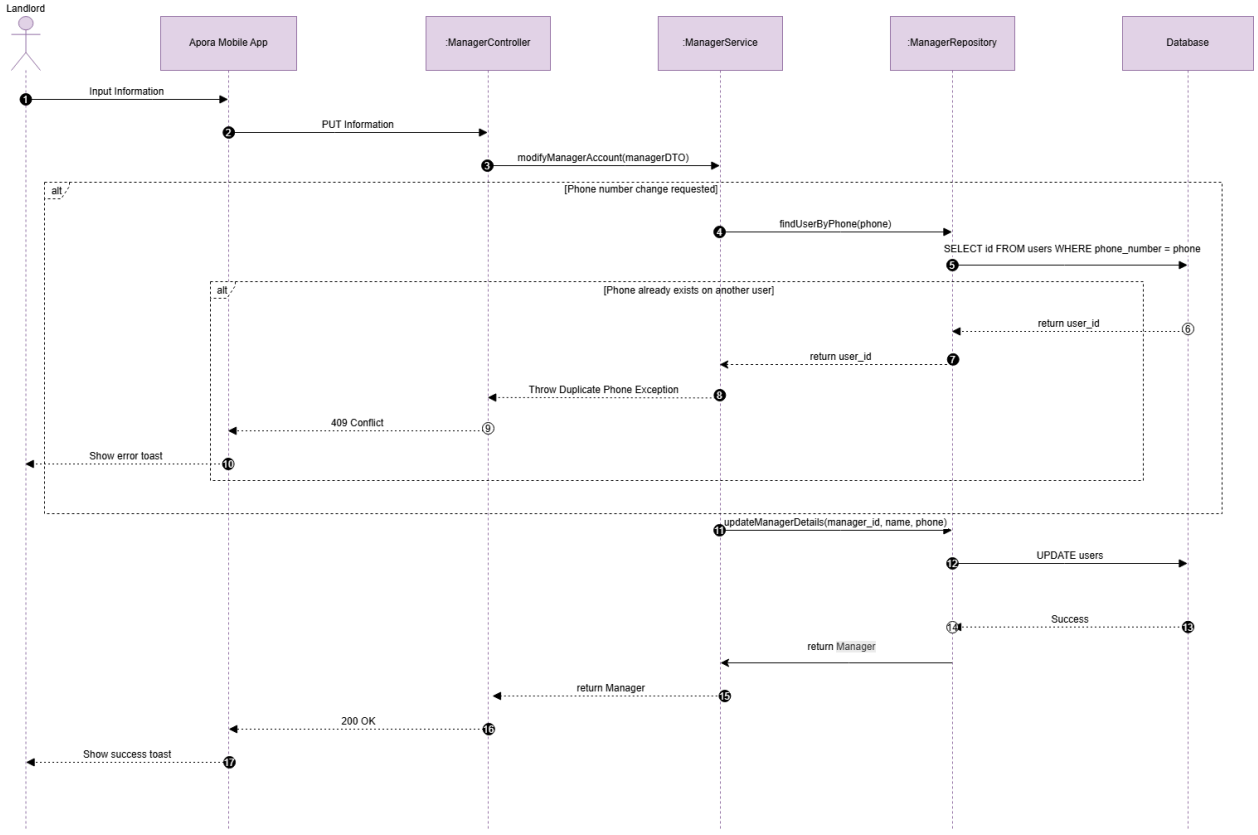
UC42: View Manager Detail



UC43: Create Manager Account



UC44: Update Manager Account



UC45: Deactivate Manager Account